# COMP102 Final Exam Workbook

Review Guide

SASE

UM6P Vanguard Center

June 30, 2026

UM6P University Mohammed VI Polytechnic

THE UM6P VANGUARD CENTER

#Empowering Minds.

# Contents

Chapter 1

# Python Foundations for Data Structures: Review Questions

This chapter reviews the first lecture of COMP102. The goal is not to memorize Python syntax. The goal is to connect Python behavior to representation, mutability, hashing, and cost.

Solutions are intentionally placed in the following chapter.

## 1.1 Conceptual model of Python objects

**Learning outcome 1.1: Explain what a Python object is conceptually**

Students should be able to explain that Python values are objects, that variable names refer to objects, and that assignment binds a name to an object instead of copying data automatically.

**Question 1 [short answer] Names and objects**

Consider the statement below.

```
data = [10, 20, 30]
```

Explain what is created and what the name `data` refers to. Your answer should mention the object type and the role of assignment.

**Question 2 [trace] Identity and equality**

What is printed by the following code?

```
a = [1, 2]
b = a
c = [1, 2]

print(a is b)
print(a == b)
print(a is c)
print(a == c)
```

**Question 3 [trace] Rebinding a name**

What is printed by the following code? Explain why.

```
x = ["old"]
y = x
x = ["new"]
print(y)
```

**Question 4 [classification] How objects appear**

For each line, classify the object creation or use as one of the following: literal, constructor, mutator method, accessor method.

```
a = {"python", "graphs"}
b = list("ada")
b.append("!")
n = len(b)
```

## 1.2 Mutability, aliasing, and rebinding

**Learning outcome 1.2: Distinguish mutable from immutable objects**

Students should be able to predict when an operation mutates an existing object and when it produces a new object. They should also be able to explain how aliasing creates side effects.

**Question 5 [trace] Aliasing through mutation**

What is printed by the following code?

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

**Question 6 [comparison] Mutation versus concatenation**

Compare the two code fragments below. For each fragment, state what `backup` refers to after the code runs.
Fragment A:

```
data = [10, 20]
backup = data
data.append(30)
```

Fragment B:

```
data = [10, 20]
backup = data
data = data + [30]
```

**Question 7 [trace] Nested aliasing**

What is printed by the following code?

```
grid = [[0, 0, 0]] * 2
grid[0][1] = 9
print(grid)
```

**Question 8 [classification] Mutable or immutable**

Classify each type as mutable or immutable: `int`, `str`, `tuple`, `list`, `dict`, `set`. For each mutable type, give one operation that can change the object in place.

## 1.3 Choosing among built-in structures

**Learning outcome 1.3: Choose between `list`, `tuple`, `dict`, and `set`**

Students should be able to select a built-in structure from the dominant constraint: order, indexing, fixed shape, mapping, uniqueness, or membership testing.

**Question 9 [design] One structure per situation**

Choose the most appropriate structure among `list`, `tuple`, `dict`, and `set` for each situation.

1. Store tasks in the order in which they were submitted. Duplicate tasks may appear.
2. Store a fixed RGB color such as `(255, 128, 0)`.
3. Store each student ID with the student's grade.
4. Track whether a graph node has already been visited.

**Question 10 [justification] List or tuple**

A program represents a 2D point as `[3, 5]`. Give one reason this may be a poor choice and one reason it may be acceptable.

**Question 11 [design critique] Membership at scale**

A website keeps one million registered usernames in a Python list and checks whether a new username already exists using `name in usernames`. Explain the likely problem and propose a better structure.

**Question 12 [nested structures] A small model**

Design a Python data representation for this situation:
Each course code maps to the set of student IDs enrolled in that course. The system frequently asks whether a given student is enrolled in a given course.
Give the structure and a small example with two courses.

## 1.4 Hidden costs in convenient syntax

**Learning outcome 1.4: Recognize hidden costs in Python operations**

Students should be able to identify operations that look simple but perform real work: slicing, membership testing in lists, concatenation, and insertion near the beginning of a list.

**Question 13 [concept] Slicing cost**

Consider the following code.

```
a = list(range(10))
b = a[2:7]
```

What does the slice create? Is `b` a view into `a`, or is it a separate list object?

**Question 14 [trace] Slice independence**

What is printed by the following code?

```
a = [10, 20, 30, 40, 50]
b = a[1:4]
b[0] = 99
print(a)
print(b)
```

**Question 15 [analysis] Repeated slicing**

A recursive function repeatedly splits a list using slices such as `items[:mid]` and `items[mid:]`. Why can this add significant overhead compared with passing index boundaries?

**Question 16 [cost ranking] Which operation is hiding work?**

For a list of length $n$, explain why each operation below may be more expensive than it first appears.

1. `x in data`
2. `data = data + [x]`

3. `data.insert(0, x)`

## 1.5 Hashing and hashability

**Learning outcome 1.5: Describe the intuition behind hashing**

Students should be able to explain that hash-based structures use a computed code to organize lookup, why collisions matter, and why dictionary keys and set elements must be hashable.

**Question 17 [short answer] Hashing intuition**

Explain the role of a hash function in a dictionary or set. Your answer should mention deterministic behavior, compatibility with equality, and lookup.

**Question 18 [error analysis] Unhashable key**

What happens here, and why?

```
d = {}
d[[1, 2]] = "hello"
```

**Question 19 [classification] Hashable or not**

For each object, state whether it can be used as a dictionary key in ordinary Python.

```
"ada"
42
(1, 2)
(1, [2, 3])
[1, 2]
```

**Question 20 [concept] Collisions**

What is a hash collision? Does the existence of collisions mean that dictionaries and sets are useless? Explain.

## 1.6 Complexity language

**Learning outcome 1.6: Use basic complexity language**

Students should be able to interpret $O(1)$, $O(n)$, and $O(n^2)$, distinguish best/average/-worst cases, and describe typical operation costs for common built-in structures.

**Question 21 [definition] Big-O interpretation**

Explain the difference between $O(1)$, $O(n)$, and $O(n^2)$. Do not use exact timings; describe growth as input size increases.

**Question 22 [cases] Search in a list**

Suppose `target in data` is evaluated on a list of length $n$. Describe the best case, average case, and worst case in terms of how many elements may need to be inspected.

**Question 23 [table completion] Typical operation costs**

Complete the following table using the usual average-case intuition discussed in the lecture.

| Operation | `list` | `tuple` | `set` | `dict` |
|---|---|---|---|---|
| Index by position | | | | |
| Membership test | | | | |
| Append/add/assign | | | | |
| Update by key or position | | | | |

**Question 24 [design reasoning] Dominant operation**

For each application, identify the dominant operation and choose a suitable structure.

1. A task scheduler repeatedly adds new tasks at the end and processes tasks in submission order.
2. A plagiarism checker repeatedly asks whether a paragraph fingerprint was already seen.
3. A grade book repeatedly retrieves a grade from a student ID.

## 1.7 Integrated practice

**Learning outcome 1.7: Connect data-structure choice to operation cost**

Students should be able to combine object semantics, mutability, hashing, and complexity to justify a representation choice.

**Question 25 [design] Remove duplicates but preserve order**

Given a list of names, design an approach that removes duplicates while preserving the first occurrence order. You may use more than one built-in structure. Explain why each structure is used.

**Question 26 [debugging] Visited nodes**

A graph traversal stores visited nodes in a list:

```
visited = []
for node in path:
    if node not in visited:
        visited.append(node)
```

For large paths, what is the cost issue? Rewrite the idea using a better structure for membership testing.

**Question 27 [trace] Combined behavior**

What is printed by the following code?

```
records = {"A": [10, 20], "B": [30]}
backup = records
records["A"].append(99)
records = {"C": [40]}
print(backup)
print(records)
```

**Question 28 [comparison] Two implementations**

Two students implement duplicate detection.
Implementation A:

```
def has_duplicate(items):
    seen = []
    for x in items:
        if x in seen:
            return True
        seen.append(x)
    return False
```

Implementation B:

```python
def has_duplicate(items):
    seen = set()
    for x in items:
        if x in seen:
            return True
        seen.add(x)
    return False
```

Compare the two implementations in terms of correctness assumptions and expected cost.

Chapter 2

# Solutions: Python Foundations for Data Structures

This chapter gives solutions to the review questions from the previous chapter. The solutions are intentionally concise. For exam review, students should first attempt the questions without looking at this chapter.

## 2.1 Solutions for conceptual object model questions

**Solution 1 [short answer]**

The expression `[10, 20, 30]` creates a new list object. The name `data` is then bound to that object. The name does not contain the list internally; it refers to the list object.

**Solution 2 [trace]**

The output is:

```
True
True
False
True
```

`a is b` is true because `b = a` creates a second reference to the same list object. `a is c` is false because `c` refers to a different list object. Equality `==` compares values, so both equality tests are true.

**Solution 3 [trace]**

The output is:

```
['old']
```

Initially `x` and `y` refer to the same list. The assignment `x = ["new"]` does not mutate the old list; it creates a new list and rebinds `x`. The name `y` still refers to the original list.

**Solution 4 [classification]**

1. `a = {"python", "graphs"}` uses a set literal.
2. `b = list("ada")` uses a constructor.
3. `b.append("!")` is a mutator method because it changes the list in place.
4. `n = len(b)` is an accessor-style operation because it inspects the object without changing it.

## 2.2 Solutions for mutability, aliasing, and rebinding questions

**Solution 5 [trace]**

The output is:

```
[1, 2, 3]
```

The names `a` and `b` refer to the same list object. `append` mutates that shared object in place.

**Solution 6 [comparison]**

In Fragment A, `backup` and `data` still refer to the same list object, which has been mutated to `[10, 20, 30]`.
In Fragment B, `backup` still refers to the original list `[10, 20]`. The expression `data + [30]` creates a new list, and `data` is rebound to that new list.

**Solution 7 [trace]**

The output is:

```
[[0, 9, 0], [0, 9, 0]]
```

The expression `[[0, 0, 0]] * 2` repeats references to the same inner list. Mutating one row mutates the shared inner list visible through both rows.

**Solution 8 [classification]**

`int`, `str`, and `tuple` are immutable. `list`, `dict`, and `set` are mutable.
Examples of in-place changes: `list.append(x)`, `dict[key] = value`, and `set.add(x)`.

## 2.3 Solutions for choosing built-in structures

**Solution 9 [design]**

1. `list`: order matters and duplicates are allowed.
2. `tuple`: the RGB value is a fixed-size record with fixed meaning.
3. `dict`: a student ID maps to an associated grade.
4. `set`: visited tracking is mainly a membership question.

**Solution 10 [justification]**

A list may be a poor choice because a point usually has fixed shape and fixed meaning; a tuple such as `(3, 5)` communicates that better. A list may be acceptable if the point is intentionally mutable, for example if an algorithm repeatedly updates coordinates in place.

**Solution 11 [design critique]**

The likely problem is that `name in usernames` on a list may require scanning many usernames, so the expected cost is linear in the number of stored names. A `set` is a better structure for repeated membership checks because average membership testing is constant time under normal hashing assumptions.

**Solution 12 [nested structures]**

A suitable representation is a dictionary from course code to a set of student IDs:

```
enrollment = {
    "COMP102": {1001, 1002, 1005},
    "MATH201": {1002, 1004},
}
```

The dictionary handles course lookup. The set handles fast membership testing inside each course.

## 2.4 Solutions for hidden cost questions

**Solution 13 [concept]**

The slice creates a new list object containing references to the selected elements. `b` is not a view into `a`. For a slice of length $k$, creating the slice performs work proportional to $k$.

**Solution 14 [trace]**

The output is:

```
[10, 20, 30, 40, 50]
[99, 30, 40]
```

The slice `a[1:4]` creates a separate list. Assigning `b[0] = 99` changes `b`, not `a`.

**Solution 15 [analysis]**

Each slice allocates a new list and copies references into it. In recursive or divide-and-conquer code, this extra copying can occur at many levels of recursion. Passing index boundaries avoids repeatedly constructing sublists.

**Solution 16 [cost ranking]**

1. `x in data`: the list may need to scan many elements, so membership is linear in the worst case.
2. `data = data + [x]`: concatenation creates a new list containing the old elements plus `x`; it is not the same as appending in place.
3. `data.insert(0, x)`: inserting at the beginning forces existing elements to shift right.

## 2.5 Solutions for hashing questions

**Solution 17 [short answer]**

A hash function computes a numerical code used to organize where an object should be stored or searched for in a hash-based structure. It should be deterministic during a run, equal objects must have equal hashes, and it should be fast enough that lookup remains efficient.

**Solution 18 [error analysis]**

The code raises a `TypeError` because a list is unhashable. A dictionary key must have a stable hash. Lists are mutable, so their contents can change after insertion, which would make hash-based placement unreliable.

**Solution 19 [classification]**

1. `"ada"`: hashable.
2. `42`: hashable.
3. `(1, 2)`: hashable, because its elements are hashable.
4. `(1, [2, 3])`: not hashable, because it contains a list.

5. `[1, 2]`: not hashable, because lists are mutable.

**Solution 20 [concept]**

A collision happens when two different objects produce the same hash value or are directed to the same bucket. Collisions do not make dictionaries and sets useless; real hash tables include collision-handling strategies. Collisions mainly affect performance, especially if many values cluster badly.

## 2.6 Solutions for complexity questions

**Solution 21 [definition]**

$O(1)$ means the cost stays bounded as input size grows. $O(n)$ means the cost grows roughly proportionally to the input size. $O(n^2)$ means the cost grows roughly with the square of the input size, so doubling $n$ can multiply the work by about four in the dominant term.

**Solution 22 [cases]**

Best case: the target is at the first inspected position, so only one element is checked.
Average case: if the target is present at a typical position, about half the list may be inspected; if it may be absent, the expected model depends on assumptions.
Worst case: the target is absent or at the last position, so all $n$ elements are inspected.

**Solution 23 [table completion]**

A typical table is:

| Operation | `list` | `tuple` | `set` | `dict` |
|---|---|---|---|---|
| Index by position | $O(1)$ | $O(1)$ | – | – |
| Membership test | $O(n)$ | $O(n)$ | avg. $O(1)$ | avg. $O(1)$ on keys |
| Append/add/assign | avg. $O(1)$ append | – | avg. $O(1)$ add | avg. $O(1)$ assignment |
| Update by key or position | $O(1)$ by index | – | – | avg. $O(1)$ by key |

**Solution 24 [design reasoning]**

1. Dominant operation: preserving order and appending. A `list` is suitable for the representation covered in this lecture. Later, queue-specific structures may be better for front removal.
2. Dominant operation: repeated membership testing and uniqueness. A `set` is suitable.

3. Dominant operation: key-based lookup from student ID to grade. A `dict` is suitable.

## 2.7 Solutions for integrated practice

**Solution 25 [design]**

Use both a list and a set. The list stores the output in first-occurrence order. The set stores values already seen for fast membership testing.

```
def unique_preserve_order(names):
    seen = set()
    result = []
    for name in names:
        if name not in seen:
            seen.add(name)
            result.append(name)
    return result
```

**Solution 26 [debugging]**

The issue is that `node not in visited` scans a list, so repeated checks can become costly. Use a set for membership:

```
visited = set()
for node in path:
    if node not in visited:
        visited.add(node)
```

If traversal order must also be preserved, keep a separate list for order.

**Solution 27 [trace]**

The output is:

```
{'A': [10, 20, 99], 'B': [30]}
{'C': [40]}
```

Initially `backup` and `records` refer to the same dictionary. Appending to `records["A"]` mutates the list stored inside that dictionary. Then `records = {"C": [40]}` rebinds `records` to a new dictionary, while `backup` still refers to the old mutated dictionary.

**Solution 28 [comparison]**

Both implementations can detect duplicates when the items are comparable for equality. Implementation B additionally assumes the items are hashable, because it inserts them into a set.

Expected cost differs sharply. Implementation A uses list membership, so each membership test can be linear in the number of previously seen items; in the no-duplicate case, the total work can become quadratic. Implementation B uses a set, so membership and insertion are average $O(1)$; the expected total cost is linear for hashable, well-behaved inputs.

Chapter 3

# Stacks & Queues: Review Questions

This chapter reviews Lecture 2 of COMP102. The central idea is restricted access: a structure is not only a container of values, but a contract about which operations are allowed and in which order values leave.

Solutions are intentionally placed in the following chapter.

## 3.1 Abstract data types

**Learning outcome 3.1: Explain what an abstract data type is**

Students should be able to explain that an abstract data type specifies state, operations, and behavioral rules, without committing to a particular memory representation or Python implementation.

**Question 1 [definition] ADT in your own words**

Define an abstract data type. Your answer should mention state, operations, behavioral rules, and the fact that an ADT does not prescribe a concrete implementation.

**Question 2 [comparison] ADT versus data structure**

For each item below, classify it as belonging mainly to the ADT side or the data-structure implementation side.

1. The operation `pop()` removes the most recently inserted item.
2. The stack is stored internally using a Python list.
3. `push(x)` adds `x` to the top.
4. `list.pop()` at the end is amortized $O(1)$.
5. The queue follows FIFO behavior.
6. The queue is implemented using `collections.deque`.

**Question 3 [short answer] Contract and implementation**

Explain the sentence: "An ADT is a contract; a data structure is a way to satisfy that contract." Use either a stack or a queue as your example.

**Question 4 [design critique] Too much access**

A programmer says: "I do not need a stack class. I can just use a list directly and let users call any list method they want." What risk does this create for the stack ADT?

## 3.2 Stack discipline and operations

**Learning outcome 3.2: Distinguish stack behavior by its access discipline**

Students should be able to describe LIFO behavior, identify the top of a stack, and explain the meaning of `push`, `pop`, `peek`, `is_empty`, and `length`.

**Question 5 [definition] LIFO discipline**

What does LIFO mean? Explain it using the sequence of inserted values `4`, `9`, `2`.

**Question 6 [operations] Stack operation meanings**

Give the meaning of each stack operation below.

1. `push(x)`
2. `pop()`
3. `peek()` or `top()`
4. `is_empty()`
5. `length()`

**Question 7 [trace] Basic stack trace**

Start with an empty stack. Trace the following operations. Give the final stack from bottom to top and list the returned values in order.

```
push(5)
push(8)
push(1)
pop()
push(3)
peek()
pop()
```

**Question 8 [trace] Python list as a stack**

What is printed by the following code?

```
stack = []
stack.append("A")
stack.append("B")
stack.append("C")
```

```
print(stack.pop())
print(stack[-1])
stack.append("D")
print(stack)
```

**Question 9 [concept] Peek versus pop**

A student calls `peek()` when they intended to remove the top item. What is the logical bug? Give a small example where this matters.

## 3.3 Queue discipline and operations

**Learning outcome 3.3: Distinguish queue behavior by its access discipline**

Students should be able to describe FIFO behavior, identify the front and rear of a queue, and explain the meaning of `enqueue`, `dequeue`, `front`, `is_empty`, and `length`.

**Question 10 [definition] FIFO discipline**

What does FIFO mean? Explain it using the sequence of inserted values `4`, `9`, `2`.

**Question 11 [operations] Queue operation meanings**

Give the meaning of each queue operation below.

1. `enqueue(x)`
2. `dequeue()`
3. `front()` or `peek()`
4. `is_empty()`
5. `length()`

**Question 12 [trace] Basic queue trace**

Start with an empty queue. Trace the following operations. Give the final queue from front to rear and list the returned values in order.

```
enqueue(5)
enqueue(8)
enqueue(1)
dequeue()
enqueue(3)
front()
dequeue()
```

**Question 13 [trace] Python deque as a queue**

What is printed by the following code?

```
from collections import deque

queue = deque()
queue.append("A")
queue.append("B")
queue.append("C")
print(queue.popleft())
print(queue[0])
queue.append("D")
print(list(queue))
```

**Question 14 [comparison] Same insertions, different removals**

Suppose values `10`, `20`, and `30` are inserted in that order. One removal is then performed.

1. Which value is removed from a stack?
2. Which value is removed from a queue?
3. Explain the difference using access discipline, not Python syntax.

## 3.4 Tracing states after operation sequences

**Learning outcome 3.4: Trace the state of a stack or queue after operations**

Students should be able to update the state of a stack or queue step by step and distinguish returned values from remaining contents.

**Question 15 [table completion] Stack trace table**

Complete the table. The stack content should be written from bottom to top.

| Step | Stack content | Returned value |
|---|---|---|
| start | `[]` | – |
| `push(2)` | | |
| `push(4)` | | |
| `pop()` | | |
| `push(9)` | | |
| `push(6)` | | |
| `pop()` | | |
| `peek()` | | |

**Question 16 [table completion] Queue trace table**

Complete the table. The queue content should be written from front to rear.

| Step | Queue content | Returned value |
|---|---|---|
| start | `[]` | – |
| `enqueue(2)` | | |
| `enqueue(4)` | | |
| `dequeue()` | | |
| `enqueue(9)` | | |
| `enqueue(6)` | | |
| `dequeue()` | | |
| `front()` | | |

**Question 17 [error analysis] Removing from an empty structure**

What should a reasonable stack or queue implementation do when `pop()` or `dequeue()` is called on an empty structure? Explain why silently returning an arbitrary value is a bad idea.

**Question 18 [reverse trace] Recovering possible operations**

A stack starts empty and ends as `[3, 7]` from bottom to top. During the process, exactly one value was removed, and that removed value was `5`. Give one possible sequence of stack operations that produces this result.

## 3.5 Python implementation choices and complexity

**Learning outcome 3.5: Choose an appropriate Python implementation and justify it**

Students should be able to justify why a Python list is usually a good stack implementation and why `collections.deque` is usually a better queue implementation than a list with `pop(0)`.

**Question 19 [implementation choice] Best tool for the ADT**

Choose a suitable Python tool for each ADT and justify the choice.

1. Stack
2. Queue

Your answer should mention which end or ends need to be efficient.

### Question 20 [code reading] Stack implementation with a list

The code below uses a Python list as a stack.

```
stack = []
stack.append(4)
stack.append(9)
x = stack.pop()
top = stack[-1]
```

Identify which lines correspond to `push`, `pop`, and `peek`. What are the final values of `x`, `top`, and `stack`?

### Question 21 [code reading] Queue implementation with deque

The code below uses `deque` as a queue.

```
from collections import deque

queue = deque()
queue.append(4)
queue.append(9)
x = queue.popleft()
front = queue[0]
```

Identify which lines correspond to `enqueue`, `dequeue`, and `front`. What are the final values of `x`, `front`, and `list(queue)`?

### Question 22 [design critique] Correct but poor queue

A student implements a queue using this pattern.

```
queue = []
queue.append(x)    # enqueue
item = queue.pop(0)   # dequeue
```

Explain why this is behaviorally correct but usually a poor implementation choice.

### Question 23 [table completion] Operation costs

Complete the table using the cost intuition from the lecture.

| ADT operation | Good Python operation | Typical cost |
|---|---|---|
| Stack `push` | | |
| Stack `pop` | | |
| Queue `enqueue` | | |
| Queue `dequeue` | | |
| Naive queue `dequeue` with list | | |

## 3.6 Encapsulating restricted access

**Learning outcome 3.6: Reason about restricted access as a design choice**

Students should be able to explain why stacks and queues are often wrapped in classes: the class hides representation and exposes only the operations that preserve the intended discipline.

**Question 24 [short answer] Why wrap in a class?**

Explain why wrapping a stack or queue in a class can be better than exposing the raw `list` or `deque` directly.

**Question 25 [debugging] Broken stack discipline**

A program uses a list to represent a stack, but one part of the code does this:

```
stack.insert(0, "urgent")
```

Explain how this can violate the stack abstraction, even if the Python code itself runs.

**Question 26 [implementation] Implement `MyStack`**

Implement a class `MyStack` with the following methods:

- `push(value)`
- `pop()`
- `peek()`
- `is_empty()`
- `__len__()`

Use a Python list internally. On `pop()` or `peek()` from an empty stack, raise `IndexError`.

**Question 27 [implementation] Implement `MyQueue`**

Implement a class `MyQueue` with the following methods:

- `enqueue(value)`
- `dequeue()`
- `front()`
- `is_empty()`
- `__len__()`

Use `collections.deque` internally. On `dequeue()` or `front()` from an empty queue, raise `IndexError`.

## 3.7 Use cases and integrated reasoning

**Learning outcome 3.7: Recognize common algorithmic and real-world use cases**

Students should be able to recognize when LIFO or FIFO behavior is the key requirement in systems and algorithms such as undo history, function calls, DFS, BFS, task scheduling, buffering, and balanced-parentheses checking.

**Question 28 [classification] Stack or queue?**

For each situation, choose stack, queue, or neither. Justify briefly.

1. Browser back button.
2. Printer jobs handled in arrival order.
3. Checking whether parentheses are balanced.
4. Breadth-first search.
5. Depth-first search.
6. Mapping student IDs to grades.

**Question 29 [algorithm design] Balanced parentheses**

Describe an algorithm that uses a stack to check whether a string containing only `(`, `)`, `[`, `]`, `{`, and `}` is balanced. You do not need to write full Python code, but your description must be precise enough to implement.

**Question 30 [algorithm reasoning] DFS versus BFS**

Both DFS and BFS can explore a graph. Which one naturally uses a stack, and which one naturally uses a queue? Explain how the access discipline affects the exploration order.

**Question 31 [integrated trace] Undo and redo stacks**

A text editor keeps two stacks: `undo` and `redo`. New actions are pushed onto `undo`. Undoing an action pops from `undo` and pushes that action onto `redo`. Redoing an action pops from `redo` and pushes it back onto `undo`.
Start with both stacks empty and perform:

```
type_A
type_B
undo
undo
redo
type_C
```

Assume that typing a new action clears the redo stack. Give the final contents of `undo` and `redo` from bottom to top.

**Question 32 [integrated critique] Choosing between correctness and performance**

A program passes all small tests using a list-based queue with `pop(0)`, but times out on large inputs. Explain why this is not a contradiction. What should be changed?

Chapter 4

# Solutions: Stacks & Queues

This chapter gives solutions to the review questions from the previous chapter. The solutions are intentionally concise. For exam review, students should first attempt the questions without looking at this chapter.

## 4.1 Solutions for abstract data type questions

**Solution 1 [definition]**

An abstract data type specifies a family of values or states, the operations allowed on those states, and the rules those operations must obey. It describes behavior, not memory layout. For example, a stack specifies operations such as `push`, `pop`, and `peek`, together with LIFO behavior, without saying whether the implementation uses a list, an array, or linked nodes.

**Solution 2 [comparison]**

1. ADT side: this is part of the stack behavior.
2. Implementation side: this is a concrete Python representation.
3. ADT side: this is part of the stack interface.
4. Implementation side: this is a performance property of a Python operation.
5. ADT side: FIFO is the queue behavioral rule.
6. Implementation side: `deque` is a concrete Python tool.

**Solution 3 [short answer]**

For a queue, the contract says that items are enqueued at the rear and dequeued from the front, so the oldest waiting item leaves first. A data structure such as `collections.deque` satisfies that contract by providing efficient operations at both ends. The ADT says what must happen; the data structure decides how it happens and how much it costs.

**Solution 4 [design critique]**

The risk is that users can perform operations outside the stack interface, such as `insert(0, x)`, indexing into the middle, or deleting arbitrary elements. The program may still run, but it no longer clearly obeys LIFO behavior. The raw list exposes too many operations

for the abstraction.

## 4.2 Solutions for stack discipline and operations

**Solution 5 [definition]**

LIFO means last in, first out. If `4`, then `9`, then `2` are pushed onto a stack, the next item removed by `pop()` is `2`, because it was inserted most recently.

**Solution 6 [operations]**

1. `push(x)` adds `x` to the top of the stack.
2. `pop()` removes and returns the top item.
3. `peek()` or `top()` returns the top item without removing it.
4. `is_empty()` returns whether the stack contains no elements.
5. `length()` returns the number of elements in the stack.

**Solution 7 [trace]**

The operations evolve as follows:

| Step | Stack | Returned |
|---|---|---|
| `push(5)` | `[5]` | – |
| `push(8)` | `[5, 8]` | – |
| `push(1)` | `[5, 8, 1]` | – |
| `pop()` | `[5, 8]` | `1` |
| `push(3)` | `[5, 8, 3]` | – |
| `peek()` | `[5, 8, 3]` | `3` |
| `pop()` | `[5, 8]` | `3` |

Final stack: `[5, 8]`. Returned values: `1`, `3`, `3`.

**Solution 8 [trace]**

The output is:

```
C
B
['A', 'B', 'D']
```

`pop()` removes the last list element, while `stack[-1]` inspects the current top without removing it.

**Solution 9 [concept]**

The bug is that `peek()` only inspects the top item; it does not remove it. For example, if the stack is `["A", "B"]`, then `peek()` returns `"B"`, but the stack remains `["A", "B"]`. If the program expected `"B"` to be consumed, later operations will see it again.

## 4.3 Solutions for queue discipline and operations

**Solution 10 [definition]**

FIFO means first in, first out. If `4`, then `9`, then `2` are enqueued, the next item removed by `dequeue()` is `4`, because it has been waiting the longest.

**Solution 11 [operations]**

1. `enqueue(x)` adds `x` at the rear of the queue.
2. `dequeue()` removes and returns the front item.
3. `front()` or `peek()` returns the front item without removing it.
4. `is_empty()` returns whether the queue contains no elements.
5. `length()` returns the number of elements in the queue.

**Solution 12 [trace]**

The operations evolve as follows:

| Step | Queue | Returned |
|---|---|---|
| `enqueue(5)` | `[5]` | – |
| `enqueue(8)` | `[5, 8]` | – |
| `enqueue(1)` | `[5, 8, 1]` | – |
| `dequeue()` | `[8, 1]` | `5` |
| `enqueue(3)` | `[8, 1, 3]` | – |
| `front()` | `[8, 1, 3]` | `8` |
| `dequeue()` | `[1, 3]` | `8` |

Final queue: `[1, 3]`. Returned values: `5`, `8`, `8`.

**Solution 13 [trace]**

The output is:

```
A
B
['B', 'C', 'D']
```

`popleft()` removes from the front. `queue[0]` then inspects the new front.

**Solution 14 [comparison]**

1. A stack removes `30`, because `30` was inserted last.
2. A queue removes `10`, because `10` was inserted first.
3. The stack follows LIFO behavior, while the queue follows FIFO behavior.

## 4.4 Solutions for tracing questions

**Solution 15 [table completion]**

| Step | Stack content | Returned value |
|---|---|---|
| start | `[]` | – |
| `push(2)` | `[2]` | – |
| `push(4)` | `[2, 4]` | – |
| `pop()` | `[2]` | `4` |
| `push(9)` | `[2, 9]` | – |
| `push(6)` | `[2, 9, 6]` | – |
| `pop()` | `[2, 9]` | `6` |
| `peek()` | `[2, 9]` | `9` |

**Solution 16 [table completion]**

| Step | Queue content | Returned value |
|---|---|---|
| start | `[]` | – |
| `enqueue(2)` | `[2]` | – |
| `enqueue(4)` | `[2, 4]` | – |
| `dequeue()` | `[4]` | `2` |
| `enqueue(9)` | `[4, 9]` | – |
| `enqueue(6)` | `[4, 9, 6]` | – |
| `dequeue()` | `[9, 6]` | `4` |
| `front()` | `[9, 6]` | `9` |

**Solution 17 [error analysis]**

A reasonable implementation should raise an error such as `IndexError`. The operation has no valid item to return. Silently returning an arbitrary value hides a bug and can make the later program state invalid or misleading.

**Solution 18 [reverse trace]**

One valid sequence is:

```
push(3)
push(5)
pop()
push(7)
```

The removed value is `5`. The final stack from bottom to top is `[3, 7]`.

## 4.5 Solutions for Python implementation choices and complexity

**Solution 19 [implementation choice]**

1. A stack can be implemented well with a Python `list`, using `append()` for push and `pop()` at the end for pop. Both operate at the same end, which is efficient for lists.
2. A queue is usually better implemented with `collections.deque`, using `append()` at the rear and `popleft()` at the front. A queue needs efficient operations at opposite ends.

**Solution 20 [code reading]**

The `append` lines are pushes. The `stack.pop()` line is pop. The `stack[-1]` line is peek. After the code runs: `x == 9`, `top == 4`, and `stack == [4]`.

**Solution 21 [code reading]**

The `append` lines are enqueues. The `queue.popleft()` line is dequeue. The `queue[0]` line is front.
After the code runs: `x == 4`, `front == 9`, and `list(queue) == [9]`.

**Solution 22 [design critique]**

The behavior is correct because `pop(0)` removes the oldest item, so it matches FIFO order. The implementation is usually poor because removing index `0` from a list shifts all remaining elements one position to the left. That makes each dequeue linear in the queue length.

**Solution 23 [table completion]**

| ADT operation | Good Python operation | Typical cost |
|---|---|---|
| Stack `push` | `list.append(x)` | amortized $O(1)$ |
| Stack `pop` | `list.pop()` at the end | amortized $O(1)$ |
| Queue `enqueue` | `deque.append(x)` | amortized $O(1)$ |
| Queue `dequeue` | `deque.popleft()` | amortized $O(1)$ |
| Naive queue `dequeue` with list | `list.pop(0)` | $O(n)$ |

## 4.6 Solutions for encapsulating restricted access

**Solution 24 [short answer]**

A class hides the representation and exposes only the ADT operations. For a stack, users can call `push`, `pop`, and `peek`, but they cannot casually insert in the middle or remove from the wrong end. This protects the intended behavior and allows the implementation to change later without changing user code.

**Solution 25 [debugging]**

The code runs, but it inserts an item at the beginning of the underlying list rather than using the top operation. If the rest of the program treats the end of the list as the top, `"urgent"` will not be the next value popped. This mixes raw list behavior with stack behavior and breaks the abstraction.

**Solution 26 [implementation]**

One implementation is:

```
class MyStack:
    def __init__(self):
        self._items = []

    def push(self, value):
        self._items.append(value)

    def pop(self):
        if self.is_empty():
            raise IndexError("pop from empty stack")
        return self._items.pop()

    def peek(self):
        if self.is_empty():
            raise IndexError("peek from empty stack")
        return self._items[-1]
```

```
    def is_empty(self):
        return len(self._items) == 0

    def __len__(self):
        return len(self._items)
```

**Solution 27 [implementation]**

One implementation is:

```
from collections import deque

class MyQueue:
    def __init__(self):
        self._items = deque()

    def enqueue(self, value):
        self._items.append(value)

    def dequeue(self):
        if self.is_empty():
            raise IndexError("dequeue from empty queue")
        return self._items.popleft()

    def front(self):
        if self.is_empty():
            raise IndexError("front from empty queue")
        return self._items[0]

    def is_empty(self):
        return len(self._items) == 0

    def __len__(self):
        return len(self._items)
```

## 4.7 Solutions for use cases and integrated reasoning

**Solution 28 [classification]**

1. Browser back button: stack, because the most recent page is returned to first.
2. Printer jobs handled in arrival order: queue, because the oldest waiting job is processed first.
3. Balanced parentheses: stack, because the most recent unmatched opening symbol must be matched first.
4. Breadth-first search: queue, because nodes are explored in discovery order by distance

layer.

5. Depth-first search: stack, because the most recently discovered unfinished path is continued first.
6. Mapping student IDs to grades: neither stack nor queue; a dictionary is the natural structure.

**Solution 29 [algorithm design]**

Use a stack of opening symbols. Scan the string left to right. When an opening symbol is seen, push it. When a closing symbol is seen, the stack must not be empty; pop the top opening symbol and check that it matches the closing symbol. At the end, the string is balanced exactly when the stack is empty. Any mismatch or premature closing symbol means the string is not balanced.

**Solution 30 [algorithm reasoning]**

DFS naturally uses a stack. The most recently discovered unfinished node or path is explored next, so the search goes deep before returning. BFS naturally uses a queue. The earliest discovered nodes are explored first, so the search expands layer by layer.

**Solution 31 [integrated trace]**

Trace:

1. `type_A`: `undo = [A]`, `redo = []`
2. `type_B`: `undo = [A, B]`, `redo = []`
3. `undo`: `undo = [A]`, `redo = [B]`
4. `undo`: `undo = []`, `redo = [B, A]`
5. `redo`: `undo = [A]`, `redo = [B]`
6. `type_C`: `undo = [A, C]`, `redo = []`

Final contents from bottom to top: `undo = [A, C]` and `redo = []`.

**Solution 32 [integrated critique]**

There is no contradiction. Small tests may not expose poor asymptotic behavior. A list-based queue with `pop(0)` is behaviorally correct, but each dequeue shifts the remaining elements, so large inputs can become too slow. Replace the list queue with `collections.deque` and use `popleft()` for dequeue.

Chapter 5

# Linked Lists: Review Questions

This chapter reviews Lecture 3 of COMP102 and the linked-list worksheet used during mentoring. The focus is not only on writing methods, but on predicting how references change. For linked lists, pointer order is part of correctness.

Solutions are intentionally placed in the following chapter.

## 5.1 Arrays, linked lists, and motivation

**Learning outcome 5.1: Compare Python lists and linked lists**

Students should be able to explain the difference between contiguous storage and node-based storage, and connect that difference to operation costs.

**Question 1 [concept] Python list versus linked list**

What is the key difference between how a Python list and a linked list store their elements? Your answer should mention memory layout and references.

**Question 2 [cost explanation] Insertion at the beginning**

Inserting at the beginning of a Python list costs $O(n)$, while inserting at the head of a singly linked list costs $O(1)$. Explain why.

**Question 3 [table completion] Operation costs**

Complete the following table with $O(1)$, $O(n)$, or amortized $O(1)$, assuming a singly linked list with only a `head` pointer.

| Operation | Python list | Singly linked list |
|---|---|---|
| Access item by index | | |
| Insert at head | | |
| Insert at tail | | |
| Delete at head | | |
| Search for a value | | |

**Question 4 [scenario] Choosing a representation**

A playlist application frequently supports the following actions:

1. move to the next song,
2. move to the previous song,
3. insert a new song right after the current song,
4. jump directly to song number 42.

Which of these actions fit a linked-list representation well, and which one exposes a weakness of linked lists?

## 5.2 Nodes and references

**Learning outcome 5.2: Explain how nodes and references form a linked list**

Students should be able to describe the structure of a node, draw references between nodes, and reason about how the chain is maintained.

**Question 5 [fill in the blanks] Node structure**

Fill in the blanks.

1. A singly linked-list node contains _____ and _____.
2. The last node has `next` equal to _____.
3. The first node of the list is referenced by _____.

**Question 6 [code reading] Manual linking**

Consider the following code.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

a = Node(10)
b = Node(20)
c = Node(30)
a.next = b
b.next = c
```

What are the values of the following expressions?

1. `a.data`
2. `a.next.data`
3. `a.next.next.data`
4. `c.next`

**Question 7 [concept] Scattered memory**

Nodes in a linked list do not need to be side-by-side in memory. Then why can we still traverse the list from beginning to end?

**Question 8 [error analysis] Lost old head**

A student writes this version of `insert_at_head`:

```
def insert_at_head(self, data):
    new_node = Node(data)
    self.head = new_node
    new_node.next = self.head
    self.size += 1
```

Explain precisely what goes wrong.

## 5.3 Traversal, iteration, and basic inspection

**Learning outcome 5.3: Traverse and inspect a singly linked list**

Students should be able to walk through a linked list using a `current` reference, implement display and iteration, and understand why access by index requires traversal.

**Question 9 [implementation] Display method**

Write a `display(self)` method that prints a singly linked list as:

```
10 -> 20 -> 30 -> None
```

Assume each node has fields `data` and `next`.

**Question 10 [implementation] Iterator method**

Write an `__iter__(self)` method for a singly linked list. It should allow:

```
for value in linked_list:
    print(value)
```

Use `yield`.

**Question 11 [code reading] Membership using iteration**

Suppose `__iter__` has been implemented. Explain how this method works and give its time complexity.

```
def __contains__(self, target):
    for data in self:
        if data == target:
            return True
```

```
    return False
```

**Question 12 [trace] Worksheet trace**

Start with an empty singly linked list. Apply the operations below. Fill in the list state after each operation, written as `data -> data -> None`.

| Operation | List state |
|---|---|
| start | |
| `insert_at_head(30)` | |
| `insert_at_head(20)` | |
| `insert_at_head(10)` | |
| `insert_at_tail(40)` | |
| `delete_at_head()` | |
| `delete_at_tail()` | |

What is the final value of `head.data`? During `delete_at_tail()`, how many links must be followed after starting from `head`?

## 5.4 Insertion in singly linked lists

**Learning outcome 5.4: Implement insertion in a singly linked list**

Students should be able to insert at the head, at the tail, and at a given index, while preserving the chain of references.

**Question 13 [implementation] Insert at head**

Implement `insert_at_head(self, data)` for a singly linked list with fields `head` and `size`. The method should run in $O(1)$.

**Question 14 [implementation] Insert at tail**

Implement `insert_at_tail(self, data)` for a singly linked list with fields `head` and `size`. Handle the empty-list case.

**Question 15 [concept] Why check current.next?**

In `insert_at_tail`, why is the loop condition usually:

```
while current.next is not None:
    current = current.next
```

instead of:

```
while current is not None:
```

```
    current = current.next
```

**Question 16 [implementation] Insert at index**

Implement `insert_at_index(self, index, data)` for a singly linked list. Use zero-based indexing. Raise `IndexError` if `index < 0` or `index > self.size`. Reuse `insert_at_head` when `index == 0`.

**Question 17 [pointer order] Inserting in the middle**

Suppose a list contains:

```
A -> B -> C -> None
```

We want to insert `X` at index 2, between `B` and `C`. Which two pointer assignments are needed, and why must they be done in that order?

## 5.5 Deletion and search in singly linked lists

**Learning outcome 5.5: Implement deletion and search in a singly linked list**

Students should be able to delete from the head, tail, and a given index; search for values; and handle empty and single-node edge cases.

**Question 18 [implementation] Delete at head**

Implement `delete_at_head(self)`. It should remove and return the first element, update `size`, and raise `IndexError` if the list is empty.

**Question 19 [implementation] Delete at tail**

Implement `delete_at_tail(self)`. It should remove and return the last element, update `size`, and correctly handle the empty-list and single-node cases.

**Question 20 [implementation] Delete at index**

Implement `delete_at_index(self, index)`. It should remove and return the element at position `index`. Raise `IndexError` for invalid indices.

**Question 21 [implementation] Search**

Implement `search(self, target)`. It should return the index of `target`, or `-1` if `target` is absent.

**Question 22 [debugging] NoneType error**

A student gets the error:

```
AttributeError: 'NoneType' object has no attribute 'next'
```

while writing a linked-list method. Give two common causes of this error in linked-list code.

## 5.6 Reverse and two-pointer patterns

**Learning outcome 5.6: Reverse and analyze singly linked lists**

Students should be able to reverse a linked list in-place and apply slow/fast pointer reasoning to find the middle or detect a cycle.

**Question 23 [trace] Reverse trace**

Trace the in-place reverse algorithm on:

```
1 -> 2 -> 3 -> None
```

Use the variables `prev`, `current`, and `next_node`. Show the values of the three variables at each iteration.

**Question 24 [implementation] Reverse method**

Write a `reverse(self)` method for a singly linked list. It must relink the existing nodes and must not create a new linked list.

**Question 25 [complexity] Reverse complexity**

What are the time and space complexities of the in-place linked-list reversal method? Justify briefly.

**Question 26 [two pointers] Find the middle**

The following strategy uses two pointers: `slow` moves one step at a time, while `fast` moves two steps at a time. Explain why this finds the middle of a linked list in one pass. What value does it return for the list `10 -> 20 -> 30 -> 40 -> 50 -> None`?

**Question 27 [concept] Cycle detection**

Explain the idea behind Floyd's cycle-detection algorithm. Why does the meeting of `slow` and `fast` imply that there is a cycle?

## 5.7 Doubly linked lists

**Learning outcome 5.7: Explain and manipulate doubly linked lists**

Students should be able to describe the role of `prev` and `next`, maintain `head` and `tail`, and update both directions correctly.

**Question 28 [definition] Doubly linked-list node**

What fields should a doubly linked-list node contain? How is this different from a singly linked-list node?

**Question 29 [invariants] Head and tail invariants**

For a non-empty doubly linked list with `head` and `tail`, which reference conditions should always be true at the two ends of the list?

**Question 30 [implementation] DLL insert at tail**

Implement `insert_at_tail(self, data)` for a doubly linked list with fields `head`, `tail`, and `size`. The method should run in $O(1)$.

**Question 31 [implementation] DLL delete at tail**

Implement `delete_at_tail(self)` for a doubly linked list. It should run in $O(1)$ and correctly handle the empty and single-node cases.

**Question 32 [pointer updates] Insert between two DLL nodes**

A doubly linked list contains:

```
A <-> B <-> C
```

We want to insert `X` between `B` and `C`. List the four pointer updates needed.

**Question 33 [debugging] Broken backward links**

A doubly linked list has `A.next == B`, but `B.prev != A`. Explain why this is a bug and give one operation where it could cause incorrect behavior.

## 5.8 Complexity, use cases, and pitfalls

**Learning outcome 5.8: Choose linked lists deliberately and avoid common pitfalls**

Students should be able to compare linked lists with arrays, identify appropriate use cases, and recognize pointer-order and edge-case bugs.

**Question 34 [comparison] Full complexity comparison**

Classify each statement as true or false. Correct the false statements.

1. Access by index is $O(1)$ in a singly linked list.
2. Insert at head is $O(1)$ in a singly linked list.
3. Delete at tail is $O(1)$ in a singly linked list with only a head pointer.
4. Delete at tail is $O(1)$ in a doubly linked list with a tail pointer.
5. Search is $O(n)$ in Python lists, singly linked lists, and doubly linked lists.

**Question 35 [design choice] When to use linked lists**

Give two situations where a linked list is a reasonable choice, and two situations where a Python list or array-like structure is usually a better choice.

**Question 36 [production Python] collections.deque**

Python provides `collections.deque`. Which linked-list-like operations does it support efficiently, and why is it usually preferable to writing your own linked list in production code?

**Question 37 [testing] Edge-case test design**

For linked-list insertion and deletion methods, why should you test with lists of size 0, 1, and 2? Give one bug each case can reveal.

**Question 38 [exit ticket] Pointer order and tail deletion**

Answer both questions.

1. What happens if you swap the pointer order in `insert_at_head`?
2. Why is delete-at-tail $O(n)$ in a singly linked list?

Chapter 6

# Solutions: Linked Lists

This chapter gives solutions to the review questions from the previous chapter. The solutions are intentionally concise. For exam review, students should first attempt the questions without looking at this chapter.

## 6.1 Solutions for arrays, linked lists, and motivation

**Solution 1 [concept]**

A Python list stores references to its elements in a contiguous array-like block. This gives fast index-based access because the location of index $i$ can be computed directly. A linked list stores elements in separate nodes. Each node stores data and a reference to the next node. The nodes may be scattered in memory, and traversal works by following references.

**Solution 2 [cost explanation]**

In a Python list, inserting at index 0 forces all existing elements to shift one position to the right, so the cost grows with the number of elements: $O(n)$. In a singly linked list, inserting at the head only requires creating a node, linking it to the old head, and updating `head`. No existing nodes shift, so the operation is $O(1)$.

**Solution 3 [table completion]**

| Operation | Python list | Singly linked list |
|---|---|---|
| Access item by index | $O(1)$ | $O(n)$ |
| Insert at head | $O(n)$ | $O(1)$ |
| Insert at tail | amortized $O(1)$ | $O(n)$ |
| Delete at head | $O(n)$ | $O(1)$ |
| Search for a value | $O(n)$ | $O(n)$ |

For the singly linked list, insert at tail is $O(n)$ here because the structure has only a `head` pointer. With an additional `tail` pointer, it can be $O(1)$.

**Solution 4 [scenario]**

Moving to the next song fits a linked list because it is just following `current.next`. Moving to the previous song fits a doubly linked list because it is following `current.prev`. Inserting right after the current song also fits well because only a small number of references must be updated. Jumping directly to song number 42 exposes a weakness: linked lists do not have random access, so the program must traverse from some known node, usually the head or current node.

## 6.2 Solutions for nodes and references

**Solution 5 [fill in the blanks]**

1. A singly linked-list node contains **data** and a reference to the **next node**.
2. The last node has `next` equal to `None`.
3. The first node of the list is referenced by `head`.

**Solution 6 [code reading]**

1. `a.data` is `10`.
2. `a.next.data` is `20`.
3. `a.next.next.data` is `30`.
4. `c.next` is `None`.

**Solution 7 [concept]**

Traversal works because each node stores a reference to the next node. The nodes do not need to be contiguous. Starting at `head`, we repeatedly assign `current = current.next`. The chain ends when `current` becomes `None`.

**Solution 8 [error analysis]**

After `self.head = new_node`, the old head is no longer referenced by `self.head`. Then `new_node.next = self.head` sets `new_node.next` to itself, because `self.head` is already `new_node`. The old list is lost, and the new node forms a self-loop. The correct order is:

```
new_node.next = self.head
self.head = new_node
```

## 6.3 Solutions for traversal, iteration, and basic inspection

**Solution 9 [implementation]**

```
def display(self):
    current = self.head
    elements = []
    while current is not None:
        elements.append(str(current.data))
        current = current.next
    print(" -> ".join(elements) + " -> None")
```

This traverses the list once, so the time complexity is $O(n)$. It also builds a temporary Python list of strings, so the auxiliary space is $O(n)$.

**Solution 10 [implementation]**

```
def __iter__(self):
    current = self.head
    while current is not None:
        yield current.data
        current = current.next
```

The method produces one value at a time. It does not build a full Python list before iteration begins.

**Solution 11 [code reading]**

The method loops over the linked list using `__iter__`. If it finds `target`, it returns `True`; if traversal finishes without a match, it returns `False`. The worst-case time complexity is $O(n)$, because the target may be absent or located at the last node. The extra space is $O(1)$.

**Solution 12 [trace]**

| Operation | List state |
|---|---|
| start | `None` |
| `insert_at_head(30)` | `30 -> None` |
| `insert_at_head(20)` | `20 -> 30 -> None` |
| `insert_at_head(10)` | `10 -> 20 -> 30 -> None` |
| `insert_at_tail(40)` | `10 -> 20 -> 30 -> 40 -> None` |
| `delete_at_head()` | `20 -> 30 -> 40 -> None` |
| `delete_at_tail()` | `20 -> 30 -> None` |

The final value of `head.data` is 20. During `delete_at_tail()` on `20 -> 30 -> 40 -> None`, starting at `head`, we follow one link from 20 to 30 and stop at the second-to-last node. More generally, delete-at-tail must traverse to the predecessor of the tail.

## 6.4 Solutions for insertion in singly linked lists

**Solution 13 [implementation]**

```
def insert_at_head(self, data):
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node
    self.size += 1
```

The method is $O(1)$ because it performs a fixed number of reference assignments.

**Solution 14 [implementation]**

```
def insert_at_tail(self, data):
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
    else:
        current = self.head
        while current.next is not None:
            current = current.next
        current.next = new_node

    self.size += 1
```

The method is $O(n)$ because it may need to walk to the last node.

**Solution 15 [concept]**

We stop when `current.next is None` because we need `current` to be the last node. Then we can attach the new node by writing `current.next = new_node`. If the loop continued while `current is not None`, then `current` would eventually become `None`; there would be no node left on which to set `next`.

**Solution 16 [implementation]**

```
def insert_at_index(self, index, data):
    if index < 0 or index > self.size:
        raise IndexError("Index out of range")

    if index == 0:
        self.insert_at_head(data)
        return

    new_node = Node(data)
    current = self.head

```

```
    for _ in range(index - 1):
        current = current.next

    new_node.next = current.next
    current.next = new_node
    self.size += 1
```

If `index == self.size`, this also inserts at the tail because `current` becomes the old last node and `current.next` is `None`.

**Solution 17 [pointer order]**

The predecessor is `B`. The two assignments are:

```
new_node.next = prev.next
prev.next = new_node
```

where `prev` is the node containing `B`. The first assignment makes `X.next` point to `C`. The second makes `B.next` point to `X`. If we update `B.next` first without saving the old value, we lose the direct reference to `C`.

## 6.5 Solutions for deletion and search in singly linked lists

**Solution 18 [implementation]**

```
def delete_at_head(self):
    if self.head is None:
        raise IndexError("List is empty")

    data = self.head.data
    self.head = self.head.next
    self.size -= 1
    return data
```

The method is $O(1)$.

**Solution 19 [implementation]**

```
def delete_at_tail(self):
    if self.head is None:
        raise IndexError("List is empty")

    if self.head.next is None:
        data = self.head.data
        self.head = None
        self.size -= 1
        return data

```

```
    current = self.head
    while current.next.next is not None:
        current = current.next

    data = current.next.data
    current.next = None
    self.size -= 1
    return data
```

The method is $O(n)$ because it must find the second-to-last node.

**Solution 20 [implementation]**

```
def delete_at_index(self, index):
    if index < 0 or index >= self.size:
        raise IndexError("Index out of range")

    if index == 0:
        return self.delete_at_head()

    current = self.head
    for _ in range(index - 1):
        current = current.next

    data = current.next.data
    current.next = current.next.next
    self.size -= 1
    return data
```

The key is to stop at the predecessor, then bypass the target node.

**Solution 21 [implementation]**

```
def search(self, target):
    current = self.head
    index = 0

    while current is not None:
        if current.data == target:
            return index
        current = current.next
        index += 1

    return -1
```

The method is $O(n)$ in the worst case.

**Solution 22 [debugging]**

Two common causes are:

1. forgetting to handle the empty-list case before accessing `self.head.next`;
2. using the wrong traversal stop condition, so `current` becomes `None` and the code still tries to access `current.next`.

A related cause is failing to handle the single-node case in `delete_at_tail`, where `current.next.next` may be invalid.

## 6.6 Solutions for reverse and two-pointer patterns

**Solution 23 [trace]**

For the list `1 -> 2 -> 3 -> None`, initially:

```
prev = None, current = 1
```

A compact trace is:

| Iteration | next_node | Reversed link | State after advancing |
|---|---|---|---|
| 1 | 2 | `1.next = None` | `prev = 1, current = 2` |
| 2 | 3 | `2.next = 1` | `prev = 2, current = 3` |
| 3 | None | `3.next = 2` | `prev = 3, current = None` |

At the end, `head` becomes `prev`, so the list is `3 -> 2 -> 1 -> None`.

**Solution 24 [implementation]**

```
def reverse(self):
    prev = None
    current = self.head

    while current is not None:
        next_node = current.next
        current.next = prev
        prev = current
        current = next_node

    self.head = prev
```

The method relinks the existing nodes. It does not create a second linked list.

**Solution 25 [complexity]**

The time complexity is $O(n)$ because every node is visited exactly once. The auxiliary space complexity is $O(1)$ because the method uses only a fixed number of references: `prev`, `current`, and `next_node`.

**Solution 26 [two pointers]**

When `fast` moves two steps for every one step of `slow`, `slow` has covered about half the list by the time `fast` reaches the end. For `10 -> 20 -> 30 -> 40 -> 50 -> None`, the method returns `30`. It runs in $O(n)$ time and $O(1)$ space.

**Solution 27 [concept]**

Floyd's algorithm uses two references. `slow` moves one step at a time and `fast` moves two steps at a time. If there is no cycle, `fast` eventually reaches `None`. If there is a cycle, both references eventually move inside the cycle; because `fast` gains one node per iteration relative to `slow`, it must eventually catch up and meet `slow`.

## 6.7 Solutions for doubly linked lists

**Solution 28 [definition]**

A doubly linked-list node contains `data`, `prev`, and `next`. A singly linked-list node only needs `data` and `next`. The extra `prev` reference allows backward traversal, but it also increases memory overhead and creates more pointer updates to maintain.

**Solution 29 [invariants]**

For a non-empty doubly linked list:

1. `head.prev` should be `None`;
2. `tail.next` should be `None`;
3. following `next` links from `head` should eventually reach `tail`;
4. following `prev` links from `tail` should eventually reach `head`.

For an empty list, both `head` and `tail` should be `None`.

**Solution 30 [implementation]**

```
def insert_at_tail(self, data):
    new_node = DNode(data)

    if self.tail is None:
        self.head = new_node
        self.tail = new_node
    else:
        new_node.prev = self.tail
        self.tail.next = new_node
        self.tail = new_node

    self.size += 1
```

The method is $O(1)$ because the tail pointer gives direct access to the last node.

**Solution 31 [implementation]**

```
def delete_at_tail(self):
    if self.tail is None:
        raise IndexError("List is empty")

    data = self.tail.data

    if self.head == self.tail:
        self.head = None
        self.tail = None
    else:
        self.tail = self.tail.prev
        self.tail.next = None

    self.size -= 1
    return data
```

This is $O(1)$ because `tail.prev` gives direct access to the predecessor of the old tail.

**Solution 32 [pointer updates]**

Let `new_node` contain `X`. The four updates are:

```
new_node.prev = B
new_node.next = C
B.next = new_node
C.prev = new_node
```

All four links are needed: two links connect `X` to its neighbors, and two links connect the neighbors back to `X`.

**Solution 33 [debugging]**

It is a bug because the forward and backward views of the list disagree. If `A.next == B`, then moving backward from `B` should return to `A`. This can break operations such as `delete_at_tail`, backward traversal, undo/redo navigation, or inserting a node before `B`.

## 6.8 Solutions for complexity, use cases, and pitfalls

**Solution 34 [comparison]**

1. False. Access by index is $O(n)$ in a singly linked list because traversal is required.
2. True.
3. False. Delete at tail is $O(n)$ in a singly linked list with only a head pointer because we must find the second-to-last node.
4. True.
5. True.

**Solution 35 [design choice]**

A linked list is reasonable when insertions or deletions near the head are frequent, or when implementing structures such as stacks and queues where random access is not needed. A Python list or array-like structure is usually better when fast index-based access matters, when memory overhead should be low, or when cache locality is important.

**Solution 36 [production Python]**

`collections.deque` supports efficient operations at both ends, such as `appendleft`, `append`, `popleft`, and `pop`. It is usually preferable in production because it is implemented, tested, optimized, and maintained by Python. In this course, implementing linked lists manually is still useful because it teaches references, pointer updates, and complexity tradeoffs.

**Solution 37 [testing]**

Testing size 0 catches missing empty-list checks. Testing size 1 catches bugs where `head` and `tail`, or `head.next`, are handled incorrectly. Testing size 2 catches off-by-one traversal errors and incorrect predecessor logic. These cases are small, but they expose most pointer mistakes.

**Solution 38 [exit ticket]**

1. If the pointer order in `insert_at_head` is swapped, the old head may be lost, or the new node may point to itself. The correct order is to link `new_node.next` to the old `head` first, then update `head`.
2. Delete-at-tail is $O(n)$ in a singly linked list because the tail node does not store a reference to its predecessor. To remove the tail safely, we must traverse from `head` to the second-to-last node.

Chapter 7

# Recursion and Problem Decomposition: Review Questions

This chapter reviews Lecture 4 of COMP102 and the recursion coding lab. The goal is not to memorize recursive templates. The goal is to recognize when a problem can be reduced to smaller instances, identify the base case, and explain how recursive answers are combined.

Solutions are intentionally placed in the following chapter.

## 7.1 Recursive thinking and base cases

**Learning outcome 7.1: Explain recursion as problem decomposition**

Students should be able to explain recursion as solving a problem by reducing it to a smaller instance of the same problem, with a clear base case and progress toward that base case.

**Question 1 [concept] What makes a problem recursive?**

In your own words, explain what it means for a problem to have a recursive structure. Your answer should mention:

1. a smaller instance of the same problem,
2. a base case,
3. progress toward stopping.

**Question 2 [recipe] Three-part recipe on a palindrome**

Use the recursive recipe to explain how to check whether `"racecar"` is a palindrome. Your answer should identify:

1. the base case,
2. the smaller subproblem after comparing the two ends,
3. how the current answer depends on the smaller answer.

**Question 3 [code reading] Palindrome code**

Consider the function below.

```
def is_palindrome(s):
    if len(s) <= 1:
        return True
    if s[0] != s[-1]:
        return False
    return is_palindrome(s[1:-1])
```

Answer the following questions.

1. What is the base case?
2. What is the smaller subproblem?
3. What does the function return for `"level"`?
4. What does the function return for `"python"`?

**Question 4 [debugging] A recursive function that never stops**

A student writes:

```
def countdown(n):
    if n == 0:
        return "done"
    return countdown(n)
```

Explain why this function does not terminate for `countdown(5)`. Then write a corrected version.

**Question 5 [concept] Recursion limit**

Python has a recursion limit and provides `sys.setrecursionlimit(...)`.
Explain:

1. why Python has a recursion limit,
2. why increasing it is not a fix for bad recursion,
3. one situation where increasing it may be reasonable,
4. one situation where increasing it is a bad idea.

## 7.2 Recursive functions on integers, strings, and lists

**Learning outcome 7.2: Write simple recursive functions**

Students should be able to write recursive functions on integers, strings, and lists by identifying a base case, a smaller subproblem, and a rule for building the answer.

### Question 6 [implementation] Sum of digits

Implement `digit_sum(n)` recursively. The function receives a nonnegative integer `n` and returns the sum of its decimal digits.
Examples:

```
digit_sum(5073)   # 15
digit_sum(8)      # 8
```

### Question 7 [implementation] Product of digits

Implement `digit_product(n)` recursively. The function receives a nonnegative integer `n` and returns the product of its decimal digits.
Rules:

- Do not use loops.
- Treat `0` as a special case: `digit_product(0)` should return `0`.

Examples:

```
digit_product(5073)   # 0
digit_product(248)    # 64
digit_product(9)      # 9
```

### Question 8 [implementation] Recursive string reverse

Implement `reverse_string(s)` recursively. The function should return the reverse of `s`.
Examples:

```
reverse_string("python")   # "nohtyp"
reverse_string("abcde")    # "edcba"
```

### Question 9 [implementation] Remove a character recursively

Implement `remove_char(s, c)` recursively. The function should return a new string obtained by removing every occurrence of character `c` from `s`.
Examples:

```
remove_char("banana", "a")      # "bnn"
remove_char("recursion", "r")   # "ecusion"
```

### Question 10 [implementation] Is the list sorted?

Implement `is_sorted(a, i=0)` recursively. It should return `True` if the list is sorted in nondecreasing order, and `False` otherwise.
Examples:

```
is_sorted([1, 2, 2, 4, 7])   # True
```

```
is_sorted([1, 5, 3, 4, 7])  # False
```

**Question 11 [implementation] Count occurrences**

Implement `count_occurrences(a, x, i=0)` recursively. It should return how many times `x` appears in the list `a`.
Example:

```
count_occurrences([4, 1, 4, 2, 4, 5, 1, 4], 4)  # 4
```

**Question 12 [implementation] Maximum of a list**

Implement `max_in_list(a, i=0)` recursively. The list `a` is non-empty. Do not use loops.
Examples:

```
max_in_list([3, 7, 2, 9, 4])  # 9
max_in_list([-5, -2, -9])     # -2
```

**Question 13 [trace] Tracing a recursive list function**

Trace the first three calls of `max_in_list([3, 7, 2, 9, 4])` if `i` represents the current index.
For each call, write:

1. the value of `i`,
2. the remaining part of the list being considered,
3. what the recursive call is expected to return.

## 7.3 Branching recursion and counting

**Learning outcome 7.3: Analyze branching recursive problems**

Students should be able to identify recursive problems that branch into multiple smaller cases and reason about the cost of exploring those cases.

**Question 14 [implementation] Climbing stairs**

A student can climb a staircase by taking either 1 step or 2 steps at a time. Implement `count_ways(n)` recursively, returning the number of different ways to reach the top.
Examples:

```
count_ways(1)  # 1
count_ways(2)  # 2
count_ways(4)  # 5
```

For `n = 4`, the ways are `1111`, `112`, `121`, `211`, and `22`.

**Question 15 [analysis] Why branching recursion can be slow**

The simple recursive solution for `count_ways(n)` calls `count_ways(n - 1)` and `count_ways(n - 2)`.
Explain why this can become much slower than a linear recursive function such as `digit_sum(n)`.

**Question 16 [recurrence] Counting combinations**

Suppose we want to compute the number of ways to choose $k$ items from $n$ items.
Explain the recurrence:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

Your explanation should use the idea of choosing one special item $x$: either $x$ is included, or $x$ is excluded.

**Question 17 [implementation] Recursive choose**

Implement `choose(n, k)` recursively using the recurrence from the previous question.
Use the base cases:

- choosing 0 items has exactly 1 way;
- choosing all $n$ items has exactly 1 way.

## 7.4 Recursive generation

**Learning outcome 7.4: Generate outputs recursively**

Students should be able to solve recursive generation problems by maintaining a partial answer, branching over choices, and stopping when the partial answer is complete.

**Question 18 [implementation] Generate binary strings**

Implement `generate_binary(n, current="")` recursively. It should print all binary strings of length `n`, one per line.
Example for `n = 3`:

```
000
001
010
011
100
101
110
```

```
111
```

**Question 19 [analysis] Binary-string recursion tree**

For `generate_binary(n)`, answer the following.

1. What does `current` represent?
2. What is the base case?
3. Why are there $2^n$ printed strings?
4. What is the recursion-stack depth?

**Question 20 [implementation] Generate all subsequences by printing**

Implement `generate_subsequences(s, i=0, current="")` recursively. The function should print all subsequences of `s`. At each character, branch into two choices: exclude the character or include it.
For `"abc"`, one valid recursive order is:

```
""
"c"
"b"
"bc"
"a"
"ac"
"ab"
"abc"
```

**Question 21 [implementation] Generate all subsequences as a list**

Implement `generate_subsequences_list(s)` recursively. It should return the subsequences in this exact order:

```
generate_subsequences_list("abc")
# ["", "c", "b", "bc", "a", "ac", "ab", "abc"]
```

Do not use loops to generate the recursive branches.

## 7.5 Exponentiation, binary search, and divide and conquer

**Learning outcome 7.5: Connect recursive decomposition to efficiency**

Students should be able to compare weak and strong decompositions, explain why halving gives logarithmic behavior, and distinguish plain recursion from divide and conquer.

**Question 22 [implementation] Simple recursive exponentiation**

Implement `power(base, exp)` recursively for nonnegative integer `exp`, using the idea:

$$base^{exp} = base \cdot base^{exp-1}.$$

State its time complexity in terms of `exp`.

**Question 23 [implementation] Fast exponentiation**

Implement `fast_power(base, exp)` recursively using halving.
Use the ideas:

$$base^{exp} = (base^{exp/2})^2 \quad \text{if } exp \text{ is even,}$$

$$base^{exp} = base \cdot (base^{\lfloor exp/2 \rfloor})^2 \quad \text{if } exp \text{ is odd.}$$

State its time complexity.

**Question 24 [trace] Binary search trace**

Trace recursive binary search for target `19` in the sorted list:

```
[3, 6, 9, 12, 19, 25, 31]
```

Start with `left = 0` and `right = 6`. For each recursive call, write `left`, `right`, `mid`, and the decision made.

**Question 25 [implementation] Recursive binary search**

Implement:

```
def binary_search(a, target, left, right):
    pass
```

It should return the index of `target`, or `-1` if the target is absent. State the base case.

**Question 26 [precondition] Binary search on unsorted data**

A student applies binary search to this list:

```
[12, 3, 25, 9, 19, 6, 31]
```

Explain why the algorithm is no longer reliable, even if the code itself is syntactically correct.

**Question 27 [concept] Recursion versus divide and conquer**

Explain the difference between recursion and divide and conquer.
Your answer should:

1. define recursion,

2. define divide and conquer,
3. give one example of plain recursion,
4. give one example of divide and conquer,
5. explain what the combine step means.

**Question 28 [implementation] Merge step**

Implement `merge(left, right)`, where `left` and `right` are already sorted lists. The function should return one sorted list containing all elements.

**Question 29 [analysis] Merge sort**

Explain why merge sort follows the divide-conquer-combine pattern. Also state its time complexity and the reason the complexity contains a $\log n$ factor.

**Question 30 [challenge] Majority element by divide and conquer**

A majority element is a value that appears strictly more than $n/2$ times.
Describe a divide-and-conquer strategy for finding whether a list has a majority element. Your answer should mention:

1. how to divide the list,
2. what each recursive call returns,
3. how to combine the two candidates,
4. why counting candidates in the full range is necessary.

## 7.6 Classical recursion and final pitfalls

**Learning outcome 7.6: Recognize classical recursive patterns and common bugs**

Students should be able to explain Tower of Hanoi, identify recursive bugs, and decide when recursion is appropriate or when another approach is safer.

**Question 31 [trace] Tower of Hanoi trace**

For Tower of Hanoi with `n = 2`, source rod `A`, auxiliary rod `B`, and destination rod `C`, write the sequence of moves.

**Question 32 [implementation] Tower of Hanoi as returned moves**

Implement `hanoi_moves(n, source="A", auxiliary="B", destination="C")` recursively. It should return a list of strings such as `"A -> C"`. Assume `n >= 1`.
Examples:

```
hanoi_moves(1)  # ["A -> C"]
hanoi_moves(2)  # ["A -> B", "A -> C", "B -> C"]
```

**Question 33 [analysis] Why Hanoi is exponential**

Explain why solving Tower of Hanoi for $n$ disks requires solving the problem for $n-1$ disks twice, with one move of the largest disk between them. What is the number of moves as a function of $n$?

**Question 34 [debugging] Find the recursion bugs**

For each function below, identify the bug and explain how to fix it.

```
def f1(n):
    return 1 + f1(n - 1)


def f2(n):
    if n == 0:
        return 0
    return f2(n + 1)


def f3(s):
    if s == "":
        return ""
    return s[0] + f3(s[1:])
```

**Question 35 [design] Correct but not efficient**

Give an example of a recursive solution that is correct but inefficient. Explain whether the issue is correctness, recursion depth, repeated work, or a combination of these.

Chapter 8

# Solutions: Recursion and Problem Decomposition

This chapter gives solutions to the review questions from the previous chapter. The solutions are concise but complete enough to check the recursive structure: base case, smaller subproblem, progress, and combination.

## 8.1 Solutions for recursive thinking and base cases

**Solution 1 [concept]**

A problem has recursive structure when solving it can be reduced to solving a smaller instance of the same kind of problem. A recursive solution needs a base case: a smallest case that can be answered immediately. Each recursive call must make progress toward that base case; otherwise the recursion may never stop.

**Solution 2 [recipe]**

For `"racecar"`, compare the first and last characters. Both are `"r"`, so the question reduces to whether `"aceca"` is a palindrome. Then compare `"a"` with `"a"`, reducing to `"cec"`. Then compare `"c"` with `"c"`, reducing to `"e"`. A one-character string is the base case and is a palindrome. Since every outer comparison matched, `"racecar"` is a palindrome.

**Solution 3 [code reading]**

1. The base case is `len(s) <= 1`.
2. The smaller subproblem is `is_palindrome(s[1:-1])`, the inside substring.
3. `is_palindrome("level")` returns `True`.
4. `is_palindrome("python")` returns `False`, because `"p" != "n"` immediately.

**Solution 4 [debugging]**

The function has a base case, but the recursive call does not make progress. For `countdown(5)`, it calls `countdown(5)` again with the same argument, so `n` never reaches `0`.

A corrected version is:

```
def countdown(n):
    if n == 0:
        return "done"
    return countdown(n - 1)
```

**Solution 5 [concept]**

Python has a recursion limit because every recursive call uses a stack frame. Without a limit, runaway recursion could consume memory and crash the interpreter. Increasing the limit does not fix bad recursion: if the function has no reachable base case or does not make progress, it will still fail later.
Increasing the limit may be reasonable for a logically correct recursive algorithm on a very deep input, such as a deep tree traversal. It is a bad idea when the function is actually broken, for example when it repeatedly calls itself with the same argument.

## 8.2 Solutions for recursive functions on integers, strings, and lists

**Solution 6 [implementation]**

```
def digit_sum(n):
    if n < 10:
        return n
    return n % 10 + digit_sum(n // 10)
```

The base case is a one-digit number. The smaller subproblem is `n // 10`, which removes the last digit.

**Solution 7 [implementation]**

```
def digit_product(n):
    if n == 0:
        return 0
    if n < 10:
        return n
    return (n % 10) * digit_product(n // 10)
```

The special case `n == 0` is needed because the product of the digits of `0` should be `0`, not `1`. For `248`, the computation is `8 * digit_product(24)`, then `8 * 4 * digit_product(2)`, so the result is `64`.

**Solution 8 [implementation]**

```python
def reverse_string(s):
    if len(s) <= 1:
        return s
    return reverse_string(s[1:]) + s[0]
```

The smaller subproblem is the string without its first character. The first character is placed after the reversed smaller string.

**Solution 9 [implementation]**

```python
def remove_char(s, c):
    if s == "":
        return ""

    rest = remove_char(s[1:], c)
    if s[0] == c:
        return rest
    return s[0] + rest
```

At each step, the function decides whether to keep or remove the current first character, then recursively processes the rest.

**Solution 10 [implementation]**

```python
def is_sorted(a, i=0):
    if i >= len(a) - 1:
        return True
    if a[i] > a[i + 1]:
        return False
    return is_sorted(a, i + 1)
```

The base case is reaching the last index or beyond. The function checks one adjacent pair, then moves the index forward.

**Solution 11 [implementation]**

```python
def count_occurrences(a, x, i=0):
    if i == len(a):
        return 0
    if a[i] == x:
        return 1 + count_occurrences(a, x, i + 1)
    return count_occurrences(a, x, i + 1)
```

The function reduces the unsolved part of the list by increasing `i`.

**Solution 12 [implementation]**

```
def max_in_list(a, i=0):
    if i == len(a) - 1:
        return a[i]

    max_rest = max_in_list(a, i + 1)
    if a[i] > max_rest:
        return a[i]
    return max_rest
```

The base case is the last element: the maximum of a one-element suffix is that element.

**Solution 13 [trace]**

For `max_in_list([3, 7, 2, 9, 4])`:

1. Call 1: `i = 0`. The suffix is `[3, 7, 2, 9, 4]`. It expects the recursive call to return the maximum of `[7, 2, 9, 4]`.
2. Call 2: `i = 1`. The suffix is `[7, 2, 9, 4]`. It expects the recursive call to return the maximum of `[2, 9, 4]`.
3. Call 3: `i = 2`. The suffix is `[2, 9, 4]`. It expects the recursive call to return the maximum of `[9, 4]`.

Eventually the base case returns `4`, then the calls compare upward and return `9`.

## 8.3 Solutions for branching recursion and counting

**Solution 14 [implementation]**

```
def count_ways(n):
    if n == 0:
        return 1
    if n < 0:
        return 0
    return count_ways(n - 1) + count_ways(n - 2)
```

An alternative is to use `count_ways(1) = 1` and `count_ways(2) = 2` as base cases. The version above treats reaching exactly zero remaining steps as one valid completed path.

**Solution 15 [analysis]**

`digit_sum(n)` makes one recursive call per digit, so the call chain is linear in the number of digits. The naive `count_ways(n)` makes two recursive calls from most states. Many subproblems are recomputed repeatedly, such as `count_ways(3)` appearing inside both `count_ways(4)` and `count_ways(5)`. This creates exponential growth without memoization.

**Solution 16 [recurrence]**

Choose one special item $x$. Every valid group of size $k$ either includes $x$ or excludes $x$. If it includes $x$, we still need to choose $k-1$ items from the remaining $n-1$. If it excludes $x$, we still need to choose $k$ items from the remaining $n-1$. These cases are complete and non-overlapping, so their counts add:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

**Solution 17 [implementation]**

```
def choose(n, k):
    if k == 0 or k == n:
        return 1
    return choose(n - 1, k - 1) + choose(n - 1, k)
```

A more defensive version could also handle invalid values such as `k < 0` or `k > n` by returning `0`.

## 8.4 Solutions for recursive generation

**Solution 18 [implementation]**

```
def generate_binary(n, current=""):
    if len(current) == n:
        print(current)
        return

    generate_binary(n, current + "0")
    generate_binary(n, current + "1")
```

**Solution 19 [analysis]**

1. `current` is the partial binary string built so far.
2. The base case is `len(current) == n`; at that point the string is complete.
3. There are $2^n$ outputs because each of the $n$ positions has two choices: `0` or `1`.
4. The recursion-stack depth is $O(n)$, because each path chooses one bit per level.

**Solution 20 [implementation]**

```
def generate_subsequences(s, i=0, current=""):
    if i == len(s):
        print(repr(current))
        return
```

```
    generate_subsequences(s, i + 1, current)          # exclude
    generate_subsequences(s, i + 1, current + s[i])   # include
```

Using `repr` makes the empty string visible as `''`. If the worksheet expects raw printing, replace `print(repr(current))` with `print(current)`.

**Solution 21 [implementation]**

```
def generate_subsequences_list(s):
    if s == "":
        return [""]

    without_first = generate_subsequences_list(s[1:])
    with_first = [s[0] + item for item in without_first]
    return without_first + with_first
```

This returns `["", "c", "b", "bc", "a", "ac", "ab", "abc"]` for `"abc"`. The list comprehension is used to build the include branch from the recursively generated exclude branch. If loops are forbidden even inside comprehensions, this helper can be written recursively as well.

## 8.5 Solutions for exponentiation, binary search, and divide and conquer

**Solution 22 [implementation]**

```
def power(base, exp):
    if exp == 0:
        return 1
    return base * power(base, exp - 1)
```

The time complexity is $O(exp)$, because the exponent decreases by 1 at each call.

**Solution 23 [implementation]**

```
def fast_power(base, exp):
    if exp == 0:
        return 1

    half = fast_power(base, exp // 2)
    if exp % 2 == 0:
        return half * half
    return base * half * half
```

The time complexity is $O(\log exp)$, because the exponent is roughly halved at each recursive call.

**Solution 24 [trace]**

The list is `[3, 6, 9, 12, 19, 25, 31]`.

1. `left = 0`, `right = 6`, `mid = 3`, `a[mid] = 12`. Since `19 > 12`, search the right half.
2. `left = 4`, `right = 6`, `mid = 5`, `a[mid] = 25`. Since `19 < 25`, search the left half.
3. `left = 4`, `right = 4`, `mid = 4`, `a[mid] = 19`. The target is found at index `4`.

**Solution 25 [implementation]**

```
def binary_search(a, target, left, right):
    if left > right:
        return -1

    mid = (left + right) // 2

    if a[mid] == target:
        return mid
    if target < a[mid]:
        return binary_search(a, target, left, mid - 1)
    return binary_search(a, target, mid + 1, right)
```

The base case is an empty interval: `left > right`.

**Solution 26 [precondition]**

Binary search relies on sorted order. When the middle element is inspected, the algorithm decides that one half can be discarded. That decision is valid only if all elements on one side are known to be smaller or larger in the sorted sense. On unsorted data, discarding half the list can discard the target by mistake.

**Solution 27 [concept]**

Recursion means solving a problem by reducing it to smaller instances of the same problem. Divide and conquer is a specific recursive strategy: divide the input into subproblems, solve the subproblems recursively, and combine their answers.
Plain recursion example: reversing a string by reversing `s[1:]` and appending `s[0]`.
Divide-and-conquer example: merge sort, where the list is split into halves, each half is sorted recursively, and the two sorted halves are merged. The combine step is the work that turns the subproblem answers into the full answer.

**Solution 28 [implementation]**

```
def merge(left, right):
    result = []
    i = 0
    j = 0

```

```
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

**Solution 29 [analysis]**

Merge sort divides the list into two halves, recursively sorts each half, then combines the two sorted halves using `merge`. The $\log n$ factor comes from repeatedly halving the list until sublists have size 1. At each level, the merge work across all sublists is $O(n)$. Therefore the total time is $O(n \log n)$.

**Solution 30 [challenge]**

A divide-and-conquer majority strategy is:

1. Split the current range into left and right halves.
2. Recursively find a majority candidate in each half.
3. If both halves return the same candidate, keep it.
4. Otherwise, count both candidates in the full current range and return the one that appears more than half the range length, if any.

Counting in the full range is necessary because a value that is a majority in one half is not automatically a majority in the combined range.
A typical implementation uses helpers like:

```
def count_in_range(a, value, left, right):
    count = 0
    for i in range(left, right + 1):
        if a[i] == value:
            count += 1
    return count


def majority_element(a, left, right):
    if left == right:
        return a[left]

    mid = (left + right) // 2
    left_candidate = majority_element(a, left, mid)
    right_candidate = majority_element(a, mid + 1, right)

```

```
    if left_candidate == right_candidate:
        return left_candidate

    length = right - left + 1
    left_count = count_in_range(a, left_candidate, left, right)
    right_count = count_in_range(a, right_candidate, left, right)

    if left_count > length // 2:
        return left_candidate
    if right_count > length // 2:
        return right_candidate
    return None
```

The caller can print `NO MAJORITY` when the result is `None`.

## 8.6 Solutions for classical recursion and final pitfalls

**Solution 31 [trace]**

For `n = 2`, the moves are:

```
A -> B
A -> C
B -> C
```

First move the smaller disk from `A` to `B`, then move the large disk from `A` to `C`, then move the smaller disk from `B` to `C`.

**Solution 32 [implementation]**

```
def hanoi_moves(n, source="A", auxiliary="B", destination="C"):
    if n == 1:
        return [f"{source} -> {destination}"]

    moves = []
    moves += hanoi_moves(n - 1, source, destination, auxiliary)
    moves.append(f"{source} -> {destination}")
    moves += hanoi_moves(n - 1, auxiliary, source, destination)
    return moves
```

**Solution 33 [analysis]**

To move $n$ disks from source to destination, first move the top $n-1$ disks to the auxiliary rod. Then move the largest disk once to the destination. Finally, move the $n-1$ disks from the auxiliary rod to the destination. This creates the recurrence:

$$T(n) = 2T(n-1) + 1.$$

With $T(1) = 1$, the number of moves is $2^n - 1$. The time complexity is $O(2^n)$.

**Solution 34 [debugging]**

1. `f1` has no base case, so it never stops. Add a base case such as `if n == 0: return 0`.
2. `f2` has a base case, but the recursive call moves away from it when `n` is positive. Use `f2(n - 1)` instead of `f2(n + 1)`.
3. `f3` terminates, but it does not reverse the string. It returns the original string, because it keeps `s[0]` before the recursive result. To reverse, use `return f3(s[1:]) + s[0]`.

**Solution 35 [design]**

The naive recursive staircase solution is correct but inefficient because it recomputes the same subproblems many times. The issue is not the base case or termination; the function does stop for valid nonnegative `n`. The problem is repeated work, leading to exponential time. Memoization or dynamic programming can keep the recursive idea while avoiding recomputation.

Chapter 9

# Binary Trees: Review Questions

This chapter reviews Lecture 5 of COMP102 and the binary-trees worksheet. The focus is on branching structure: terminology, node representation, depth and height, recursive depth-first traversals, breadth-first traversal, and small recursive algorithms on trees.

Solutions are intentionally placed in the following chapter.

## 9.1 Binary-tree structure and terminology

**Learning outcome 9.1: Define binary trees and identify their parts**

Students should be able to define a binary tree, identify its main parts, and use vocabulary such as root, child, parent, sibling, leaf, internal node, ancestor, descendant, and subtree.

**Question 1 [concept] From linear data to branching data**

Most structures seen earlier in the course are linear: arrays, stacks, queues, and linked lists all have some notion of a next element.
Explain why a binary tree is different. Your answer should mention:

1. branching,
2. hierarchy,
3. why expressions such as $(3 + 5) \times (10 - 2)$ naturally fit a tree.

**Question 2 [terminology] Parts of a binary tree**

Use the tree below.

```
      A
     / \
    B   C
   / \   \
  D   E   F
```

Answer the following.

1. What is the root?
2. What are the leaves?
3. What are the internal nodes?

4. What is the parent of `E`?
5. What is the sibling of `D`?
6. What are the descendants of `B`?
7. What is the subtree rooted at `B`?

**Question 3 [concept] What binary does not mean**

A student says: "A binary tree is a tree where the values are ordered, with smaller values on the left and larger values on the right."
Explain what is wrong with this statement. Then state what the word *binary* means in *binary tree*.

**Question 4 [relation queries] Recovering relations from child statements**

Suppose a binary tree is described by these statements:

- `A` has left child `B`.
- `A` has right child `C`.
- `B` has left child `D`.
- `B` has right child `E`.
- `C` has left child `F`.

Answer the queries.

1. Who is the parent of `F`?
2. Who is the grandparent of `D`?
3. Who is the sibling of `D`?
4. Who is the uncle or aunt of `D`?
5. Draw the tree or describe it in one sentence.

**Question 5 [classification] Leaves and internal nodes**

For each node in the tree below, classify it as a leaf or an internal node.

```
      R
     / \
    S   T
       / \
      U   V
         /
        W
```

Then answer: can a node with exactly one child be an internal node?

## 9.2 Representation with nodes and references

**Learning outcome 9.2: Represent binary trees using Python objects**

Students should be able to implement a minimal `Node` class and build small binary trees by linking `left` and `right` references.

**Question 6 [implementation] Minimal binary-tree node**

Write a minimal Python `Node` class for a binary tree. Each node should store:

1. a value,
2. a reference to the left child,
3. a reference to the right child.

The constructor should allow this usage:

```
root = Node("A")
root.left = Node("B")
root.right = Node("C")
```

**Question 7 [implementation] Build a fixed tree**

Using your `Node` class, implement `build_sample_tree()` so that it returns the root of this tree:

```
      A
     / \
    B   C
   / \   \
  D   E   F
```

Rules:

- Return the root node, not a list and not a dictionary.
- `root.value` should be `"A"`.
- `root.left.right.value` should be `"E"`.
- `root.right.right.value` should be `"F"`.

**Question 8 [code reading] Following object references**

Assume `root = build_sample_tree()` from the previous question.
What does each expression refer to?

1. `root.value`
2. `root.left.value`
3. `root.left.right.value`
4. `root.right.left`
5. `root.right.right.value`

**Question 9 [implementation] Build an expression tree**

Build the expression tree for:

$$(3 + 5) \times (10 - 2).$$

The root should contain `"*"`. The left child of the root should contain `"+"`, and the right child should contain `"-"`. The leaves should contain `3`, `5`, `10`, and `2`.
Write the Python linking code.

## 9.3 Depth and height

**Learning outcome 9.3: Compute depth and height**

Students should be able to compute the depth of a node, the height of a node, and explain the difference between the two measurements.

**Question 10 [concept] Depth versus height**

In one or two sentences, explain the difference between depth and height.
Your answer must mention:

1. where depth is measured from,
2. where height is measured toward,
3. the depth of the root,
4. the height of a leaf.

**Question 11 [calculation] Depth and height on a tree**

Use the tree below.

```
      A
     / \
    B   C
   / \   \
  D   E   F
```

Compute:

1. `depth(A)`
2. `depth(E)`
3. `depth(F)`
4. `height(D)`
5. `height(B)`
6. the height of the whole tree

**Question 12 [calculation] An unbalanced tree**

Use the tree below.

```
    A
   /
  B
   \
    C
     \
      D
```

Compute:

1. `depth(D)`
2. `height(D)`
3. `height(C)`
4. `height(B)`
5. `height(A)`

**Question 13 [implementation] Recursive height function**

Use the convention:

- `height(None)` returns `-1`;
- a leaf has height `0`;
- the height of a non-empty tree is the number of edges on the longest downward path from that node to a leaf.

Implement:

```python
def height(root):
    pass
```

Do not use loops or global variables.

## 9.4 Depth-first traversals

**Learning outcome 9.4: Trace and implement DFS traversals**

Students should be able to trace preorder, inorder, and postorder traversals and explain how they differ by the moment when the current node is visited.

**Question 14 [trace] Trace DFS traversals**

Use the tree below.

```
    A
   / \
  B   C
```

```
    / \   \
   D   E   F
```

Write the output order for each traversal.

1. Preorder: Node, Left, Right
2. Inorder: Left, Node, Right
3. Postorder: Left, Right, Node

### Question 15 [concept] When is the root visited?

For the tree in the previous question, explain where `A` appears in each DFS traversal and why.
Your answer should mention:

1. preorder,
2. inorder,
3. postorder.

### Question 16 [implementation] Recursive DFS functions

Implement the following traversal functions. Each function should return a list of node values, not print them.

```python
def preorder(root):
    pass

def inorder(root):
    pass

def postorder(root):
    pass
```

Rules:

- If the tree is empty, return `[]`.
- Use recursion.
- Do not use a global variable.

### Question 17 [expression tree] Expression notation

Use the expression tree below.

```
      *
     / \
    +   -
   / \ / \
  3  5 10 2
```

Answer the following.

1. Write the usual infix expression.
2. Write the preorder output.
3. Write the inorder output.
4. Write the postorder output.
5. Which traversal corresponds most closely to prefix notation?
6. Which traversal corresponds most closely to postfix notation?

## 9.5 Breadth-first traversal

**Learning outcome 9.5: Trace BFS and explain queue usage**

Students should be able to trace breadth-first traversal, also called level-order traversal, and explain why it naturally uses a queue.

**Question 18 [trace] Trace BFS**

Use the tree below.

```
      A
     / \
    B   C
   / \   \
  D   E   F
```

Write the breadth-first traversal order. Then write one sentence explaining the main idea of BFS.

**Question 19 [queue trace] BFS queue states**

Trace the queue used by BFS on the tree below. At each step, show the node removed from the front and the queue after adding its children.

```
      A
     / \
    B   C
   / \   \
  D   E   F
```

Use this table format:

| Step | Removed | Queue after adding children |
|---|---|---|
| 1 | | |
| 2 | | |
| 3 | | |
| 4 | | |
| 5 | | |
| 6 | | |

**Question 20 [implementation] Level-order traversal**

Implement `level_order(root)` so that it returns a list of values in BFS order.
Rules:

- If the tree is empty, return `[]`.
- Return a list; do not print.
- Use a queue-like process.

You may use `collections.deque`.

**Question 21 [comparison] DFS versus BFS**

Compare DFS and BFS on a binary tree.
Your answer should mention:

1. how DFS explores,
2. how BFS explores,
3. the typical tool used by DFS,
4. the typical tool used by BFS,
5. one DFS output and the BFS output for the sample tree.

## 9.6 Recursive algorithms on trees

**Learning outcome 9.6: Write recursive algorithms on binary trees**

Students should be able to write recursive tree functions by solving the left subtree, solving the right subtree, and combining those answers with the current node.

**Question 22 [implementation] Count all nodes**

Implement `count_nodes(root)` recursively.
Rules:

- If the tree is empty, return `0`.
- Do not use loops.
- Do not use a global variable.

**Question 23 [implementation] Count leaves**

Implement `count_leaves(root)` recursively.
A leaf is a node with no left child and no right child.
Rules:

- If the tree is empty, return `0`.
- A single-node tree has one leaf.
- Do not use loops or global variables.

**Question 24 [analysis] Explain recursive tree counting**

Explain the recursive idea behind either `count_nodes` or `count_leaves`.
Your answer must mention:

1. the base case,
2. what is computed on the left subtree,
3. what is computed on the right subtree,
4. how the two results are combined.

**Question 25 [reflection] Tree recursion versus linear recursion**

Recursive functions on strings or lists often move from index `i` to `i + 1`. Binary-tree recursion usually makes one recursive call on the left subtree and one recursive call on the right subtree.
Explain why tree recursion is different from linear recursion. Use one example function from this chapter.

## 9.7 Common pitfalls

**Learning outcome 9.7: Recognize common binary-tree mistakes**

Students should be able to recognize common mistakes involving terminology, traversal order, empty trees, and confusing binary trees with binary search trees.

**Question 26 [debugging] Traversal bug**

A student claims this function is preorder traversal:

```
def preorder(root):
    if root is None:
        return []
    return preorder(root.left) + preorder(root.right) + [root.value]
```

Explain why the claim is wrong. What traversal is this actually implementing?

**Question 27 [debugging] Forgetting the empty tree**

A student writes:

```
def count_nodes(root):
    return 1 + count_nodes(root.left) + count_nodes(root.right)
```

Explain what error will occur on an empty child and fix the function.

**Question 28 [mixed review] Final binary-tree check**

Answer briefly.

1. Why are preorder, inorder, and postorder all DFS traversals?
2. Why is BFS naturally associated with a queue?
3. Why does inorder traversal of a general binary tree not necessarily produce sorted values?
4. Why is drawing the tree usually the safest first step for relation and traversal questions?

Chapter 10

# Solutions: Binary Trees

This chapter gives solutions to the review questions from the previous chapter. The solutions emphasize structure, traversal order, and recursive reasoning on left and right subtrees.

## 10.1 Solutions for binary-tree structure and terminology

**Solution 1 [concept]**

A binary tree is different from a linear structure because a node can branch to a left child and a right child. This creates a hierarchy rather than one simple next-element chain. Expressions such as $(3 + 5) \times (10 - 2)$ fit a tree because the multiplication depends on two subexpressions, and each subexpression depends on its own operands.

**Solution 2 [terminology]**

1. The root is `A`.
2. The leaves are `D`, `E`, and `F`.
3. The internal nodes are `A`, `B`, and `C`. Here `C` is internal because it has one child.
4. The parent of `E` is `B`.
5. The sibling of `D` is `E`.
6. The descendants of `B` are `D` and `E`.
7. The subtree rooted at `B` contains `B`, `D`, and `E`, with `D` as the left child and `E` as the right child.

**Solution 3 [concept]**

The statement confuses a binary tree with a binary search tree. In a binary tree, *binary* only means that each node has at most two children, usually called left and right. It says nothing about sorted values. Ordering constraints are introduced later for binary search trees.

**Solution 4 [relation queries]**

The described tree is:

```
1         A
```

```
      / \
     B   C
    / \ /
   D  E F
```

1. The parent of `F` is `C`.
2. The grandparent of `D` is `A`.
3. The sibling of `D` is `E`.
4. The uncle or aunt of `D` is `C`, because `C` is the sibling of `D`'s parent `B`.
5. Drawing helps because relationship questions are path questions. Once the parent-child links are visible, the answers are direct.

**Solution 5 [classification]**

Leaves: `S`, `U`, and `W`. Internal nodes: `R`, `T`, and `V`. A node with exactly one child is still an internal node because it is not a leaf. A leaf has no children.

## 10.2 Solutions for representation with nodes and references

**Solution 6 [implementation]**

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

Each node stores its value and two child references. If a child is missing, the corresponding reference remains `None`.

**Solution 7 [implementation]**

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None


def build_sample_tree():
    root = Node("A")
    root.left = Node("B")
    root.right = Node("C")

    root.left.left = Node("D")
    root.left.right = Node("E")
```

```
    root.right.right = Node("F")

    return root
```

**Solution 8 [code reading]**

1. `root.value` is `"A"`.
2. `root.left.value` is `"B"`.
3. `root.left.right.value` is `"E"`.
4. `root.right.left` is `None`, because `C` has no left child in this tree.
5. `root.right.right.value` is `"F"`.

**Solution 9 [implementation]**

```
root = Node("*")
root.left = Node("+")
root.right = Node("-")

root.left.left = Node(3)
root.left.right = Node(5)

root.right.left = Node(10)
root.right.right = Node(2)
```

The operators are internal nodes and the numbers are leaves.

## 10.3 Solutions for depth and height

**Solution 10 [concept]**

Depth is measured from the root down to a node. The root has depth `0`. Height is measured from a node down to its deepest leaf. A leaf has height `0`.

**Solution 11 [calculation]**

1. `depth(A) = 0`
2. `depth(E) = 2`
3. `depth(F) = 2`
4. `height(D) = 0`
5. `height(B) = 1`
6. The height of the whole tree is `2`.

**Solution 12 [calculation]**

For the unbalanced tree:

1. `depth(D) = 3`
2. `height(D) = 0`
3. `height(C) = 1`
4. `height(B) = 2`
5. `height(A) = 3`

**Solution 13 [implementation]**

```
def height(root):
    if root is None:
        return -1

    left_height = height(root.left)
    right_height = height(root.right)
    return 1 + max(left_height, right_height)
```

For a leaf, both child heights are `-1`, so the function returns `1 + max(-1, -1) = 0`.

## 10.4 Solutions for depth-first traversals

**Solution 14 [trace]**

For the sample tree:

1. Preorder: `A, B, D, E, C, F`
2. Inorder: `D, B, E, A, C, F`
3. Postorder: `D, E, B, F, C, A`

**Solution 15 [concept]**

In preorder, `A` appears first because the node is visited before its subtrees. In inorder, `A` appears after the entire left subtree and before the right subtree. In postorder, `A` appears last because both subtrees are visited before the node itself.

**Solution 16 [implementation]**

```
def preorder(root):
    if root is None:
        return []
    return [root.value] + preorder(root.left) + preorder(root.right)


def inorder(root):
```

```
    if root is None:
        return []
    return inorder(root.left) + [root.value] + inorder(root.right)


def postorder(root):
    if root is None:
        return []
    return postorder(root.left) + postorder(root.right) + [root.value]
```

These are DFS traversals because each one fully explores subtrees before moving back up.

**Solution 17 [expression tree]**

1. The usual infix expression is $(3 + 5) \times (10 - 2)$.
2. Preorder: `*, +, 3, 5, -, 10, 2`
3. Inorder: `3, +, 5, *, 10, -, 2`. With parentheses, this is $(3 + 5) * (10 - 2)$.
4. Postorder: `3, 5, +, 10, 2, -, *`
5. Preorder corresponds most closely to prefix notation.
6. Postorder corresponds most closely to postfix notation.

## 10.5 Solutions for breadth-first traversal

**Solution 18 [trace]**

The breadth-first traversal order is:

```
A, B, C, D, E, F
```

BFS visits the tree level by level: first the root, then all nodes at depth `1`, then all nodes at depth `2`, and so on.

**Solution 19 [queue trace]**

| Step | Removed | Queue after adding children |
|---|---|---|
| 1 | `A` | `[B, C]` |
| 2 | `B` | `[C, D, E]` |
| 3 | `C` | `[D, E, F]` |
| 4 | `D` | `[E, F]` |
| 5 | `E` | `[F]` |
| 6 | `F` | `[]` |

The traversal output is the order of removed nodes: `A, B, C, D, E, F`.

**Solution 20 [implementation]**

```python
from collections import deque


def level_order(root):
    if root is None:
        return []

    result = []
    q = deque([root])

    while q:
        node = q.popleft()
        result.append(node.value)

        if node.left is not None:
            q.append(node.left)
        if node.right is not None:
            q.append(node.right)

    return result
```

**Solution 21 [comparison]**

DFS goes deep along a branch before returning to explore another branch. Recursive preorder on the sample tree gives `A, B, D, E, C, F`. BFS visits level by level and gives `A, B, C, D, E, F`. DFS is naturally implemented with recursion or a stack. BFS is naturally implemented with a queue because nodes discovered earlier should be processed earlier.

## 10.6 Solutions for recursive algorithms on trees

**Solution 22 [implementation]**

```python
def count_nodes(root):
    if root is None:
        return 0
    return 1 + count_nodes(root.left) + count_nodes(root.right)
```

The `1` counts the current node. The two recursive calls count the left and right subtrees.

**Solution 23 [implementation]**

```python
def count_leaves(root):
    if root is None:
        return 0
```

```
4
5       if root.left is None and root.right is None:
6           return 1
7
8       return count_leaves(root.left) + count_leaves(root.right)
```

A leaf contributes `1`; an internal node contributes the number of leaves in its two subtrees.

**Solution 24 [analysis]**

For `count_nodes`, the base case is the empty tree, which contributes `0`. The function recursively counts the nodes in the left subtree and the nodes in the right subtree. It combines them by adding `1` for the current node:

$$1 + \text{left count} + \text{right count}.$$

The same left/right/combine structure appears in many tree algorithms.

**Solution 25 [reflection]**

Linear recursion on a string or list often has one smaller subproblem, such as the suffix starting at `i + 1`. Tree recursion usually has two smaller subproblems: the left subtree and the right subtree. For example, `height(root)` computes the height of the left subtree and the height of the right subtree, then combines them with `1 + max(left_height, right_height)`. The empty tree is the base case because recursive calls eventually reach missing children.

## 10.7 Solutions for common pitfalls

**Solution 26 [debugging]**

The function is not preorder because it visits the root after both subtrees. Preorder should return `[root.value] + preorder(root.left) + preorder(root.right)`. The given function implements postorder traversal: Left, Right, Node.

**Solution 27 [debugging]**

The function will eventually call `count_nodes(None)` for a missing child. Then it tries to access `root.left` when `root` is `None`, causing an error such as `AttributeError: 'NoneType' object has no attribute 'left'`.
Correct version:

```
def count_nodes(root):
    if root is None:
        return 0
    return 1 + count_nodes(root.left) + count_nodes(root.right)
```

**Solution 28 [mixed review]**

1. Preorder, inorder, and postorder are all DFS traversals because they explore down subtrees before moving sideways to unrelated branches.
2. BFS is associated with a queue because it processes the oldest discovered node first and appends newly discovered children to the back.
3. Inorder traversal of a general binary tree is not necessarily sorted because a general binary tree has no ordering rule on values.
4. Drawing the tree makes parent-child links, levels, and traversal order visible. It reduces mistakes in relation and traversal questions.

Chapter 11

# Binary Search Trees and AVL Trees: Review Questions

This chapter reviews Lecture 6 of COMP102 and the BST/AVL mentoring worksheet. The focus is on the ordering invariant of binary search trees, search and insertion, sorted inorder traversal, validation with bounds, deletion, height-based complexity, and AVL rotations.

Solutions are intentionally placed in the following chapter.

## 11.1 From binary trees to binary search trees

**Learning outcome 11.1: Define the binary search tree property**

Students should be able to define the BST ordering invariant and distinguish an ordinary binary tree from a binary search tree.

**Question 1 [concept] Binary tree versus BST**

Explain the difference between an ordinary binary tree and a binary search tree.
Your answer should mention:

1. the shape restriction of a binary tree,
2. the ordering rule of a BST,
3. why the words *left* and *right* do not automatically imply sorted values in an ordinary binary tree.

**Question 2 [definition] The strict BST property**

State the strict BST property used in this course.
Then answer: under this strict convention, are duplicate values allowed? Explain briefly.

**Question 3 [classification] Valid or invalid BSTs**

For each tree below, say whether it is a valid strict BST. If it is invalid, identify the node that violates the rule and explain why.
**Tree A**

```
1        8
```

```
      / \
     3   10
    / \    \
   1   6    14
```

**Tree B**

```
      10
     /  \
    5    15
     \
      12
```

**Tree C**

```
    5
   / \
  3   5
```

**Question 4 [concept] The global invariant**

A student validates a BST by checking only this at every node:

```
if root.left and root.left.value >= root.value:
    return False
if root.right and root.right.value <= root.value:
    return False
```

Explain why this is insufficient. Use the following tree in your explanation:

```
      10
     /  \
    5    15
     \
      12
```

## 11.2 Searching, insertion, and inorder traversal

**Learning outcome 11.2: Trace search and insertion in a BST**

Students should be able to use the BST property to trace search and insertion one comparison at a time.

**Question 5 [trace] Trace BST search**

Use this BST.

```
      8
     / \
```

```
      3   10
     / \    \
    1   6    14
```

Trace the search for each target.

1. Search for `6`. Give the visited nodes and the comparison at each step.
2. Search for `7`. Give the visited nodes and explain where the search fails.
3. Why does the BST search not need to inspect both subtrees at each node?

### Question 6 [implementation] BST search

Implement `contains(root, x)` for a strict BST.
Rules:

- Return `True` if `x` appears in the BST.
- Return `False` if the tree is empty or if `x` is absent.
- Use the BST ordering rule to choose only one direction at each node.

```python
class Node:
    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left = left
        self.right = right


def contains(root, x):
    pass
```

### Question 7 [trace] Build a BST by insertion

Starting from an empty tree, insert the values in this order:

$$8,\ 3,\ 10,\ 1,\ 6,\ 14,\ 4,\ 7.$$

Answer the following.

1. Draw the final BST.
2. Write the inorder traversal of the final tree.
3. Where are `4` and `7` attached, and why?

### Question 8 [implementation] Insertion, building, and inorder

Complete the following functions.

```python
class Node:
    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left = left
```

```
        self.right = right


def insert(root, x):
    """Insert x into the BST and return the root.
    Duplicates are ignored.
    """
    pass


def build_bst(values):
    """Build a BST by inserting values from left to right."""
    pass


def inorder(root):
    """Return the inorder traversal as a list."""
    pass
```

Your code should satisfy:

```
root = build_bst([8, 3, 10, 1, 6, 14, 4, 7])
print(inorder(root))
# [1, 3, 4, 6, 7, 8, 10, 14]

root = build_bst([5, 5, 3, 7, 3])
print(inorder(root))
# [3, 5, 7]
```

**Learning outcome 11.3: Explain why inorder traversal of a BST is sorted**

Students should be able to connect the traversal rule *left, node, right* to the BST invariant.

**Question 9 [concept] Why inorder becomes sorted**

For an arbitrary binary tree, inorder traversal is just a traversal order. For a BST, inorder traversal returns values in sorted order.
Explain why. Your answer must mention:

1. the left subtree,
2. the root value,
3. the right subtree,
4. the BST ordering property.

**Question 10 [code reading] Reconnect the recursive result**

Consider this incorrect insertion pattern:

```
if x < root.value:
```

```
    insert(root.left, x)      # wrong
elif x > root.value:
    insert(root.right, x)     # wrong
```

Explain why the return value of the recursive call must be assigned back:

```
root.left = insert(root.left, x)
root.right = insert(root.right, x)
```

Use insertion into an empty child position in your explanation.

## 11.3 BST validation and basic operations

**Learning outcome 11.4: Check whether a binary tree is a valid BST**

Students should be able to validate a BST using lower and upper bounds rather than only direct-child comparisons.

**Question 11 [implementation] Validate with bounds**

Implement `is_bst(root)` using lower and upper bounds.
Rules:

- An empty tree is a valid BST.
- The convention is strict: `low < node.value < high` whenever bounds exist.
- The left subtree receives the current node value as its new upper bound.
- The right subtree receives the current node value as its new lower bound.

```
def is_bst(root):
    """Return True if root is a valid strict BST."""
    def check(node, low, high):
        pass

    return check(root, None, None)
```

**Question 12 [trace] Bounds on a counterexample**

For this tree:

```
      10
     /  \
    5    15
     \
      12
```

Trace the bounds passed to node `12` in a correct `is_bst` implementation. Why does the check reject it?

**Question 13 [implementation] Minimum and maximum**

Assume `root` is a non-empty BST. Implement:

- `find_min(root)`, which returns the smallest value;
- `find_max(root)`, which returns the largest value.

Then explain why each operation costs $O(h)$, where $h$ is the height of the tree.

```
def find_min(root):
    pass


def find_max(root):
    pass
```

## 11.4 Height, complexity, and degenerate trees

**Learning outcome 11.5: Connect BST performance to height**

Students should be able to explain why search, insertion, minimum, and maximum depend on tree height.

**Question 14 [concept] Cost depends on height**

Let $h$ be the height of a BST with $n$ nodes.
Answer the following.

1. Why is search $O(h)$?
2. Why is insertion $O(h)$?
3. Why is inorder traversal $O(n)$ instead of $O(h)$?
4. What is the difference between a balanced BST and a degenerate BST?

**Question 15 [trace] Bad insertion order**

Starting from an empty BST, insert the sorted values:

$$1,\ 2,\ 3,\ 4,\ 5,\ 6.$$

Answer the following.

1. Draw the final shape.
2. What is its height if height is counted in edges?
3. Why is this BST similar to a linked list?
4. What is the worst-case search cost in this tree?

**Question 16 [comparison] Binary tree, BST, AVL tree**

Complete the comparison.

| Structure | Rule | Search cost |
|---|---|---|
| Binary tree | | |
| BST | | |
| AVL tree | | |

## 11.5 BST deletion

**Learning outcome 11.6: Delete from a BST while preserving the invariant**

Students should be able to identify the three deletion cases and explain why the inorder successor repairs the two-child case.

**Question 17 [concept] The three BST deletion cases**

Explain the three deletion cases in a BST:

1. deleting a leaf,
2. deleting a node with one child,
3. deleting a node with two children.

For the two-child case, explain why replacing the deleted value with the inorder successor preserves the BST order.

**Question 18 [trace] Delete the root with two children**

Use this BST:

```
      5
     / \
    3   8
   / \ / \
  2  4 6  9
```

Trace `delete(5)` using the inorder successor strategy.

1. Which deletion case is this?
2. What is the inorder successor of `5`?
3. What value replaces the root?
4. From where is the old successor removed?
5. What is the inorder traversal after deletion?

**Question 19 [implementation] BST deletion**

Implement `delete(root, x)` using the inorder successor for the two-child case.

```
def delete(root, x):
    """Delete x from the BST and return the new root."""
    pass
```

You may write a helper function to find the minimum node in a subtree.

## 11.6 AVL trees and rotations

**Learning outcome 11.7: Define AVL balance factor and recognize imbalance cases**

Students should be able to compute balance factors, identify the four AVL insertion cases, and choose the appropriate rotation repair.

**Question 20 [concept] AVL balance factor**

Define the AVL balance factor of a node.
Then answer:

1. Which balance-factor values are allowed in an AVL tree?
2. What does a balance factor of `2` mean?
3. What does a balance factor of `-2` mean?

**Question 21 [calculation] Compute balance factors**

Use the tree below. For this question, use the lecture convention: height of a leaf is `0`.

```
      30
     /  \
    20   40
   /  \
  10  25
```

Compute:

1. the height of `10`, `25`, and `40`;
2. the balance factor of `20`;
3. the balance factor of `30`;
4. whether the tree is AVL-balanced.

**Question 22 [classification] Four AVL insertion cases**

For each insertion sequence, identify the imbalance case and the rotation repair. Then state the final root after rebalancing.

1. `30, 20, 10`
2. `10, 20, 30`
3. `30, 10, 20`
4. `10, 30, 20`

### Question 23 [tracing] AVL insertion trace with rotations

Starting from an empty AVL tree, insert the following values in order:

$$30,\ 20,\ 10,\ 40,\ 50,\ 25,\ 5,\ 15,\ 13$$

Use the AVL rule:

$$\text{balance factor} = \text{height(left)} - \text{height(right)}$$

A node is balanced if its balance factor is $-1$, 0, or 1.
For each insertion:

1. Draw the tree after insertion and rebalancing.
2. If a rotation is needed, identify:
   - the first unbalanced node,
   - the imbalance case: LL, RR, LR, or RL,
   - the rotation or rotations applied.
3. Give the final AVL tree.
4. Give the inorder traversal of the final tree.

### Question 24 [concept] What rotations must preserve

A rotation is a local rewrite of part of the tree.
Explain why a valid AVL rotation must preserve:

1. the BST ordering property,
2. the same set of values,
3. the inorder traversal order.

Then explain what problem rotations are meant to fix.

### Question 25 [implementation] Repair AVL insertion

Complete the missing parts of AVL insertion. For this coding exercise, use the common implementation convention: `height(None) = 0` and a leaf has stored height `1`.

```
class AVLNode:
    def __init__(self, value, left=None, right=None, height=1):
        self.value = value
        self.left = left
        self.right = right
        self.height = height
```

```


def height(node):
    return 0 if node is None else node.height


def update_height(node):
    if node is not None:
        node.height = 1 + max(height(node.left), height(node.right))


def balance_factor(node):
    return 0 if node is None else height(node.left) - height(node.right)


def right_rotate(y):
    x = y.left
    t2 = x.right
    x.right = y
    y.left = t2
    update_height(y)
    update_height(x)
    return x


def left_rotate(x):
    y = x.right
    t2 = y.left
    y.left = x
    x.right = t2
    update_height(x)
    update_height(y)
    return y


def avl_insert(root, x):
    if root is None:
        return AVLNode(x)

    if x < root.value:
        root.left = avl_insert(root.left, x)
    elif x > root.value:
        root.right = avl_insert(root.right, x)
    else:
        return root

    # TODO: update height
    # TODO: compute balance

    # TODO: Left-left
    # TODO: Right-right
```

```
    # TODO: Left-right
    # TODO: Right-left

    return root
```

**Question 26 [reflection] BST versus AVL**

Explain the progression:

$$\text{binary tree} \rightarrow \text{BST} \rightarrow \text{AVL tree}.$$

Your answer should explain what each step adds and why AVL trees are needed even after we already have BSTs.

Chapter 12

# Solutions: Binary Search Trees and AVL Trees

This chapter gives solutions to the review questions from the previous chapter. The solutions emphasize the BST invariant, height-based complexity, correct validation, and AVL rotation logic.

## 12.1 Solutions for BST definitions and validity

**Solution 1 [concept]**

An ordinary binary tree is a tree where each node has at most two children, usually called left and right. A binary search tree is a binary tree with an ordering invariant: all values in the left subtree of a node are smaller than the node value, and all values in the right subtree are larger. In an ordinary binary tree, left and right are only structural positions; they do not imply any ordering of values.

**Solution 2 [definition]**

The strict BST property is:

$$\text{all values in the left subtree} < \text{node value} < \text{all values in the right subtree.}$$

Under this strict convention, duplicates are not allowed. A duplicate would violate either the strict left inequality or the strict right inequality.

**Solution 3 [classification]**

1. Tree A is a valid strict BST. Every value in the left subtree of `8` is smaller than `8`, and every value in the right subtree is larger than `8`. The same rule holds inside each subtree.
2. Tree B is not a valid BST. Node `12` is inside the left subtree of `10`, so it must be smaller than `10`. But `12 > 10`.
3. Tree C is not a valid strict BST because it contains a duplicate `5`. The right child must be strictly greater than the root, not equal to it.

**Solution 4 [concept]**

Checking only direct children misses violations caused by ancestors. In the example, `12` is a valid right child of `5` locally because `12 > 5`. However, `12` is still inside the full left subtree of `10`, so it must also satisfy `12 < 10`. It does not. Correct validation must carry lower and upper bounds imposed by all ancestors.

## 12.2 Solutions for searching, insertion, and inorder traversal

**Solution 5 [trace]**

1. Search for `6`: compare with `8`; since `6 < 8`, go left. Compare with `3`; since `6 > 3`, go right. Compare with `6`; found. Visited path: `8 -> 3 -> 6`.
2. Search for `7`: compare with `8`; go left. Compare with `3`; go right. Compare with `6`; since `7 > 6`, go right. The right child is `None`, so the value is absent. Visited path: `8 -> 3 -> 6 -> None`.
3. The search does not inspect both subtrees because the BST property rules out one side at every comparison. If the target is smaller than the current value, it cannot appear in the right subtree. If it is larger, it cannot appear in the left subtree.

**Solution 6 [implementation]**

```
class Node:
    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left = left
        self.right = right


def contains(root, x):
    if root is None:
        return False

    if x == root.value:
        return True
    if x < root.value:
        return contains(root.left, x)
    return contains(root.right, x)
```

The recursive call follows only the subtree where `x` is allowed to appear.

**Solution 7 [trace]**

The final BST is:

```
       8
      / \
```

```
      3   10
     / \    \
    1   6    14
       / \
      4   7
```

The inorder traversal is:

```
[1, 3, 4, 6, 7, 8, 10, 14]
```

Node `4` is attached as the left child of `6`: `4 < 8`, `4 > 3`, then `4 < 6`. Node `7` is attached as the right child of `6`: `7 < 8`, `7 > 3`, then `7 > 6`.

**Solution 8 [implementation]**

```
class Node:
    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left = left
        self.right = right


def insert(root, x):
    """Insert x into the BST and return the root.
    Duplicates are ignored.
    """
    if root is None:
        return Node(x)

    if x < root.value:
        root.left = insert(root.left, x)
    elif x > root.value:
        root.right = insert(root.right, x)

    return root


def build_bst(values):
    """Build a BST by inserting values from left to right."""
    root = None
    for x in values:
        root = insert(root, x)
    return root


def inorder(root):
    """Return the inorder traversal as a list."""
    if root is None:
        return []
    return inorder(root.left) + [root.value] + inorder(root.right)
```

**Solution 9 [concept]**

Inorder traversal visits `left subtree -> root -> right subtree`. In a BST, every value in the left subtree is smaller than the root, and every value in the right subtree is larger than the root. So the inorder order is: smaller values first, then the root value, then larger values. Recursively, each subtree is also output in sorted order.

**Solution 10 [code reading]**

If the recursive call inserts into an empty child position, it returns a new `Node`. If that returned node is ignored, the parent never receives the new child reference. The call may create a node, but nothing in the tree points to it. Assigning `root.left = insert(root.left, x)` or `root.right = insert(root.right, x)` reconnects the modified subtree back to the current node.

## 12.3 Solutions for BST validation and basic operations

**Solution 11 [implementation]**

```
def is_bst(root):
    """Return True if root is a valid strict BST."""
    def check(node, low, high):
        if node is None:
            return True

        if low is not None and node.value <= low:
            return False
        if high is not None and node.value >= high:
            return False

        return (check(node.left, low, node.value) and
                check(node.right, node.value, high))

    return check(root, None, None)
```

The bounds encode the constraints imposed by all ancestors, not only the parent.

**Solution 12 [trace]**

At the root `10`, the valid range is initially `(-infinity, +infinity)`. When the algorithm moves to the left child `5`, the upper bound becomes `10`. When it then moves to the right child `12`, the range is `(5, 10)`. Node `12` violates the upper bound because `12 >= 10`. Therefore the tree is rejected.

**Solution 13 [implementation]**

```
def find_min(root):
    current = root
    while current.left is not None:
        current = current.left
    return current.value


def find_max(root):
    current = root
    while current.right is not None:
        current = current.right
    return current.value
```

The minimum follows one chain of left pointers; the maximum follows one chain of right pointers. Each chain has length at most the tree height $h$, so each operation costs $O(h)$.

## 12.4 Solutions for height, complexity, and degenerate trees

**Solution 14 [concept]**

1. Search is $O(h)$ because each comparison moves one level down the tree, and at most one root-to-leaf path is followed.
2. Insertion is $O(h)$ because it first searches for the correct empty child position, which is also along one root-to-leaf path.
3. Inorder traversal is $O(n)$ because it must visit every node once, regardless of the height.
4. A balanced BST has small height, about $O(\log n)$. A degenerate BST is shaped like a chain, so its height is $O(n)$.

**Solution 15 [trace]**

Inserting `1, 2, 3, 4, 5, 6` creates:

```
1
 \
  2
   \
    3
     \
      4
       \
        5
         \
          6
```

The height is `5` edges. The tree is similar to a linked list because every node has only one

child and there is only one path to follow. Worst-case search is $O(n)$, because searching for a missing or deepest value may inspect all nodes.

**Solution 16 [comparison]**

| Structure | Rule | Search cost |
|---|---|---|
| Binary tree | no ordering rule | $O(n)$ |
| BST | left subtree $<$ node $<$ right subtree | $O(h)$ |
| AVL tree | BST rule plus controlled balance | $O(\log n)$ |

The progression is: structure, then ordering, then balance.

## 12.5 Solutions for BST deletion

**Solution 17 [concept]**

1. If the node is a leaf, remove it by returning `None` in its place.
2. If the node has one child, replace the node by that child.
3. If the node has two children, replace its value with the inorder successor, then delete that successor from the right subtree.

The inorder successor is the smallest value in the right subtree. It is larger than everything in the left subtree and no larger replacement from the right subtree would be smaller. Therefore it fits exactly where the deleted value was.

**Solution 18 [trace]**

1. Deleting `5` is the two-child case because the root has left child `3` and right child `8`.
2. The inorder successor of `5` is `6`, the minimum value in the right subtree.
3. The root value changes from `5` to `6`.
4. The old successor node `6` is removed from the left side of the subtree rooted at `8`.
5. The inorder traversal after deletion is `[2, 3, 4, 6, 8, 9]`.

**Solution 19 [implementation]**

```
def delete(root, x):
    """Delete x from the BST and return the new root."""
    if root is None:
        return None

    if x < root.value:
        root.left = delete(root.left, x)
        return root

    if x > root.value:
```

```
        root.right = delete(root.right, x)
        return root

    # Now root.value == x.

    # Case 1: no child.
    if root.left is None and root.right is None:
        return None

    # Case 2: one child.
    if root.left is None:
        return root.right
    if root.right is None:
        return root.left

    # Case 3: two children.
    successor = root.right
    while successor.left is not None:
        successor = successor.left

    root.value = successor.value
    root.right = delete(root.right, successor.value)
    return root
```

## 12.6 Solutions for AVL trees and rotations

**Solution 20 [concept]**

The AVL balance factor of a node is:

$$\text{height(left subtree)} - \text{height(right subtree)}.$$

Allowed AVL balance factors are $-1$, 0, and 1. A balance factor of `2` means the node is too left-heavy. A balance factor of `-2` means the node is too right-heavy.

**Solution 21 [calculation]**

The leaves `10`, `25`, and `40` all have height `0`. Node `20` has left height `0` and right height `0`, so its balance factor is `0`. Node `30` has left height `1` and right height `0`, so its balance factor is `1`. Every balance factor is in $\{-1, 0, 1\}$, so the tree is AVL-balanced.

**Solution 22 [classification]**

1. `30, 20, 10`: Left-left case. Repair with a right rotation at `30`. Final root: `20`.
2. `10, 20, 30`: Right-right case. Repair with a left rotation at `10`. Final root: `20`.
3. `30, 10, 20`: Left-right case. Repair with a left rotation at `10`, then a right rotation

at `30`. Final root: `20`.

4. `10, 30, 20`: Right-left case. Repair with a right rotation at `30`, then a left rotation at `10`. Final root: `20`.

**Solution 23 [tracing]**

We insert the values one by one.
**Insert 30.**
The tree is empty, so 30 becomes the root.

```
30
```

No rotation is needed.
**Insert 20.**
Since $20 < 30$, insert 20 as the left child of 30.

```
  30
 /
20
```

No rotation is needed.
**Insert 10.**
Since $10 < 30$ and $10 < 20$, insert 10 as the left child of 20.
Before rebalancing:

```
    30
   /
  20
 /
10
```

The first unbalanced node is 30.
The insertion happened in the left subtree of the left child of 30, so this is a **Left-Left** case.
Repair: perform a **right rotation** at 30.
After rotation:

```
  20
 /  \
10   30
```

**Insert 40.**
Since $40 > 20$ and $40 > 30$, insert 40 as the right child of 30.

```
  20
 /  \
10   30
       \
       40
```

No rotation is needed.
**Insert 50.**

Since $50 > 20$, $50 > 30$, and $50 > 40$, insert 50 as the right child of 40.
Before rebalancing:

```
   20
  /  \
10   30
       \
       40
         \
         50
```

The first unbalanced node is 30.
The insertion happened in the right subtree of the right child of 30, so this is a **Right-Right** case.
Repair: perform a **left rotation** at 30.
After rotation:

```
    20
   /  \
 10    40
      /  \
    30   50
```

**Insert 25.**
Since $25 > 20$, $25 < 40$, and $25 < 30$, insert 25 as the left child of 30.
Before rebalancing:

```
    20
   /  \
 10    40
      /  \
    30   50
   /
 25
```

The first unbalanced node is 20.
The insertion happened in the left subtree of the right child of 20, so this is a **Right-Left** case.
Repair:

1. Right rotation at 40.
2. Left rotation at 20.

After the right rotation at 40:

```
    20
   /  \
 10    30
      /  \
    25    40
            \
            50
```

After the left rotation at 20:

```
      30
     /  \
   20    40
  /  \     \
10   25    50
```

**Insert 5.**
Since $5 < 30$, $5 < 20$, and $5 < 10$, insert 5 as the left child of 10.

```
      30
     /  \
   20    40
  /  \     \
10   25    50
/
5
```

No rotation is needed.
**Insert 15.**
Since $15 < 30$, $15 < 20$, and $15 > 10$, insert 15 as the right child of 10.

```
      30
     /  \
   20    40
  /  \     \
10   25    50
/ \
5  15
```

No rotation is needed.
**Insert 13.**
Since $13 < 30$, $13 < 20$, $13 > 10$, and $13 < 15$, insert 13 as the left child of 15.
Before rebalancing:

```
      30
     /  \
   20    40
  /  \     \
10   25    50
/ \
5  15
   /
  13
```

The first unbalanced node is 20.
The insertion happened in the right subtree of the left child of 20, so this is a **Left-Right** case.
Repair:

1. Left rotation at 10.
2. Right rotation at 20.

After the left rotation at 10:

```
      30
     /  \
   20    40
  /  \     \
15   25    50
/
10
/ \
5  13
```

After the right rotation at 20:

```
        30
       /  \
     15    40
    /  \     \
  10    20    50
 / \      \
5  13     25
```

Therefore, the final AVL tree is:

```
        30
       /  \
     15    40
    /  \     \
  10    20    50
 / \      \
5  13     25
```

The inorder traversal is:

$$5,\ 10,\ 13,\ 15,\ 20,\ 25,\ 30,\ 40,\ 50$$

This is sorted, so the BST property is preserved. The rotations repaired the height imbalance while keeping the inorder order unchanged.

**Solution 24 [concept]**

A rotation must preserve the BST ordering property; otherwise the result is no longer a BST. It must preserve the same set of values because rotation is not insertion or deletion. It also preserves inorder traversal because the sorted order of values must stay the same. The purpose of rotations is to repair a local height imbalance while keeping the ordering invariant intact.

**Solution 25 [implementation]**

```python
class AVLNode:
    def __init__(self, value, left=None, right=None, height=1):
        self.value = value
        self.left = left
```

```
        self.right = right
        self.height = height


def height(node):
    return 0 if node is None else node.height


def update_height(node):
    if node is not None:
        node.height = 1 + max(height(node.left), height(node.right))


def balance_factor(node):
    return 0 if node is None else height(node.left) - height(node.right)


def right_rotate(y):
    x = y.left
    t2 = x.right
    x.right = y
    y.left = t2
    update_height(y)
    update_height(x)
    return x


def left_rotate(x):
    y = x.right
    t2 = y.left
    y.left = x
    x.right = t2
    update_height(x)
    update_height(y)
    return y


def avl_insert(root, x):
    if root is None:
        return AVLNode(x)

    if x < root.value:
        root.left = avl_insert(root.left, x)
    elif x > root.value:
        root.right = avl_insert(root.right, x)
    else:
        return root

    update_height(root)
    balance = balance_factor(root)

```

```
    # Left-left case.
    if balance > 1 and x < root.left.value:
        return right_rotate(root)

    # Right-right case.
    if balance < -1 and x > root.right.value:
        return left_rotate(root)

    # Left-right case.
    if balance > 1 and x > root.left.value:
        root.left = left_rotate(root.left)
        return right_rotate(root)

    # Right-left case.
    if balance < -1 and x < root.right.value:
        root.right = right_rotate(root.right)
        return left_rotate(root)

    return root
```

The height is updated before checking the balance factor. During a rotation, the lower node is updated first, then the new root of the rotated subtree.

**Solution 26 [reflection]**

A binary tree adds branching structure: each node can have a left and right child. A BST adds ordering: values smaller than a node go to the left subtree and values larger than the node go to the right subtree. That ordering gives search direction. An AVL tree adds balance control: after updates, it uses heights, balance factors, and rotations to prevent the BST from becoming too tall. AVL trees are needed because a normal BST can degenerate into a chain under bad insertion orders, making search $O(n)$ instead of $O(\log n)$.

Chapter 13

# Binary Heaps and Priority Queues: Review Questions

This chapter reviews Lecture 7 of COMP102 and the binary heaps project tasks. The focus is on priority queues, min-heaps and max-heaps, complete-tree shape, array representation, sift-up and sift-down repairs, Python's `heapq`, priority scheduling, top- $k$ selection, and heap operation complexity.

Solutions are intentionally placed in the following chapter.

## 13.1 Priority queues and heap motivation

**Learning outcome 13.1: Define the priority queue ADT**

Students should be able to define a priority queue as an abstract data type and distinguish it from an ordinary FIFO queue.

**Question 1 [concept] Normal queue versus priority queue**

Explain the difference between a normal queue and a priority queue.
Your answer should mention:

1. what determines the next item removed from a normal queue,
2. what determines the next item removed from a priority queue,
3. one real situation where FIFO order is not enough.

**Question 2 [definition] Priority queue operations**

A priority queue stores items together with priorities.
For each operation below, write what it should do.

1. `push(item, priority)`
2. `peek()`
3. `pop()`
4. `is_empty()`

Then answer: why is this an ADT rather than a specific implementation?

**Question 3 [comparison] Could we just use a list?**

Complete the comparison below.

| Implementation | Push | Pop best | Main problem |
|---|---|---|---|
| Unsorted list | | | |
| Sorted list | | | |
| Binary heap | | | |

Use Big-O notation for the costs. Then explain why a binary heap is a compromise rather than a perfect structure.

## 13.2 Heap property and array representation

**Learning outcome 13.2: Define min-heaps, max-heaps, and complete-tree shape**

Students should be able to state the heap property, distinguish min-heaps from max-heaps, and explain why complete-tree shape matters.

**Question 4 [definition] The two heap rules**

A binary heap must satisfy two rules.

1. State the heap property for a min-heap.
2. State the heap property for a max-heap.
3. State the complete-tree shape property.
4. Explain what each rule gives us: which rule gives access to the best item, and which rule keeps the height small?

**Question 5 [classification] Valid min-heaps**

For each list, say whether it is a valid min-heap. Use zero-based array representation.

1. `[1, 3, 2, 7, 5, 8]`
2. `[3, 1, 2, 7, 5, 8]`
3. `[1, 7, 2, 3, 5, 8]`
4. `[1, 3, 8, 7, 5, 2]`

For every invalid heap, identify one parent-child violation.

**Question 6 [concept] A heap is not a BST**

Explain why a binary heap is not a binary search tree.
Your answer should mention:

1. the BST ordering rule,
2. the heap ordering rule,

3. why the left and right subtrees of a heap are not globally ordered against each other.

**Question 7 [trace] Tree view and list view**

Consider this heap list:

```
[2, 5, 3, 9, 7, 8]
```

1. Draw the corresponding complete binary tree.
2. Is the list sorted?
3. Is it still a valid min-heap? Justify using parent-child comparisons.

**Learning outcome 13.3: Represent a binary heap using a Python list**

Students should be able to use zero-based index formulas for parent and child positions in a heap array.

**Question 8 [calculation] Index formulas**

For a node at index `i` in a zero-based heap array:

1. Give the index of the left child.
2. Give the index of the right child.
3. Give the index of the parent.

Then compute:

1. the parent index of `6`,
2. the left and right child indexes of `4`,
3. the children of index `1` in a heap of length `6`.

**Question 9 [implementation] Heap index helpers**

Implement the helper functions for a zero-based heap array.

```
def parent(i):
    pass


def left(i):
    pass


def right(i):
    pass


def has_left(heap, i):
    pass
```

```
def has_right(heap, i):
    pass
```

**Question 10 [implementation] Check the min-heap property**

Implement `is_heap(heap)`.
Rules:

- Return `True` if the list satisfies the min-heap property.
- Return `True` for an empty list and a one-element list.
- Check only valid child indexes.

```
def is_heap(heap):
    pass
```

## 13.3 Insertion, removal, and heap repairs

**Learning outcome 13.4: Trace insertion using sift up**

Students should be able to explain how heap insertion appends a value and restores the heap property by moving upward.

**Question 11 [trace] Trace heap insertion**

Start with this min-heap:

```
[2, 5, 3, 9, 7, 8]
```

Insert `1`.

1. Write the list immediately after appending `1`.
2. Show each swap made by sift up.
3. Write the final heap list.
4. Explain why sift up follows only one path.

**Question 12 [implementation] Sift up and heap push**

Implement `sift_up(heap, index)` and `heap_push(heap, value)` for a min-heap.

```
def sift_up(heap, index):
    pass


def heap_push(heap, value):
```

```
    pass
```

The functions should modify `heap` in place.

**Learning outcome 13.5: Trace removal using sift down**

Students should be able to explain how removing the minimum moves the last value to the root and restores the heap property by moving downward.

**Question 13 [trace] Trace heap pop**

Start with this min-heap:

```
[1, 5, 2, 9, 7, 8, 3]
```

Remove the minimum.

1. Which value is returned?
2. What list do we get after moving the last value to the root and removing the last position?
3. Show each swap made by sift down.
4. Write the final heap list.

**Question 14 [implementation] Sift down and heap pop**

Implement `sift_down(heap, index)` and `heap_pop(heap)` for a min-heap.
Rules for `heap_pop`:

- Raise `IndexError` if the heap is empty.
- Return the removed minimum value.
- Modify the heap in place.

```
def sift_down(heap, index):
    pass


def heap_pop(heap):
    pass
```

## 13.4 Python heapq and priority scheduling

**Learning outcome 13.6: Use Python's heapq module to implement priority queues**

Students should be able to use `heapq` as a min-heap over Python lists and understand tuple-based priority entries.

**Question 15 [code reading] Predict heapq output**

What does the following code print?

```
import heapq

heap = []
heapq.heappush(heap, 5)
heapq.heappush(heap, 2)
heapq.heappush(heap, 8)

print(heap[0])
print(heapq.heappop(heap))
print(heap)
```

For the last line, write one possible valid heap list and explain why it may not be sorted.

**Question 16 [code reading] Tuples as priority entries**

Given this code:

```
import heapq

tasks = []
heapq.heappush(tasks, (3, "write report"))
heapq.heappush(tasks, (1, "fix server"))
heapq.heappush(tasks, (2, "answer email"))

while tasks:
    priority, task = heapq.heappop(tasks)
    print(task)
```

1. What is the output order?
2. Why does the priority control the order?
3. If larger numeric priority should mean more urgent, what trick can we use with `heapq`?

**Question 17 [concept] Stable tie-breaking**

A common priority-queue entry is:

```
(priority, counter, task)
```

Explain why the `counter` is useful. Your answer should mention what can go wrong when two entries have the same priority.

**Learning outcome 13.7: Model a small priority scheduling problem**

Students should be able to represent jobs with priorities and produce the correct execution order.

**Question 18 [trace] Priority scheduling by hand**

A scheduler receives these jobs. Lower number means more urgent.

| Job | Priority |
|---|---|
| backup files | 3 |
| restart server | 1 |
| security alert | 0 |
| clean logs | 5 |

Write the execution order. Then explain why this is not FIFO order.

**Question 19 [implementation] PriorityQueue class**

Implement a `PriorityQueue` class backed by a min-heap.
Rules:

- Smaller priority values come first.
- If priorities tie, earlier insertion comes first.
- Store entries as `(priority, counter, item)`.
- Raise `IndexError` from `peek` or `pop` if the queue is empty.
- You may use your own heap functions; do not use `heapq` in this exercise.

```
class PriorityQueue:
    def __init__(self):
        pass

    def push(self, item, priority):
        pass

    def peek(self):
        pass

    def pop(self):
        pass

    def is_empty(self):
        pass
```

## 13.5 Heapify, top-k, complexity, and limits

**Learning outcome 13.8: Analyze the time complexity of heap operations**

Students should be able to connect heap costs to complete-tree height and to distinguish `heapify` from repeated insertion.

**Question 20 [comparison] Complexity summary**

Complete the table.

| Operation | Cost | Reason |
|---|---|---|
| `heap[0]` / peek | | |
| `heap_push` | | |
| `heap_pop` | | |
| `heapify` | | |
| Pop all items | | |

Then explain why `heap_push` and `heap_pop` are logarithmic.

**Question 21 [implementation] Bottom-up heapify**

Implement bottom-up `heapify(values)` without using `heapq`.
Rules:

- Modify `values` in place.
- Start from the last parent index and move backward to index `0`.
- Call `sift_down(values, i)` at each parent.

```
def heapify(values):
    pass
```

After your code, explain why bottom-up heapify is $O(n)$, not $O(n \log n)$.

**Question 22 [implementation] Top-k with a heap**

Implement `top_k(values, k)` using a min-heap of size at most `k`.
Rules:

- Return the `k` largest values sorted from largest to smallest.
- If `k <= 0`, return `[]`.
- If `k >= len(values)`, return all values sorted from largest to smallest.
- Do not sort the whole list unless `k >= len(values)`.

```
def top_k(values, k):
    pass
```

Example:

```
print(top_k([5, 1, 9, 3, 7, 2], 3))
# [9, 7, 5]
```

**Question 23 [concept] What heaps are bad at**

List four operations or questions that binary heaps are not good at.
Then choose one of them and explain why the heap property does not give enough information to solve it quickly.

**Question 24 [synthesis] Choosing the right structure**

For each situation, choose one structure from this list: stack, queue, BST, AVL tree, binary heap, priority queue.

1. Process tasks strictly by arrival order.
2. Undo the most recent editor action.
3. Repeatedly execute the most urgent pending task.
4. Search for a specific value while maintaining reliable logarithmic performance.
5. Repeatedly extract the smallest timestamp in a simulation.
6. Store values so that inorder traversal gives sorted order, but without guaranteed balance.

Briefly justify each choice.

Chapter 14

# Solutions: Binary Heaps and Priority Queues

This chapter gives solutions to the review questions from the previous chapter. The solutions emphasize the difference between priority-queue behavior and heap implementation, the heap property, array indexing, sift-up and sift-down repairs, tuple priorities, stable scheduling, and heap complexity.

## 14.1 Solutions for priority queues and heap motivation

**Solution 1 [concept]**

A normal queue removes the item that arrived first; it follows FIFO order. A priority queue removes the item with the best priority, regardless of arrival order. FIFO is not enough in situations such as emergency scheduling, operating-system task scheduling, printer queues with urgent jobs, or simulation events where the earliest timestamp must be processed first.

**Solution 2 [definition]**

1. `push(item, priority)` inserts an item together with its priority.
2. `peek()` returns the current best-priority item without removing it.
3. `pop()` removes and returns the current best-priority item.
4. `is_empty()` returns whether the priority queue contains no items.

It is an ADT because it specifies behavior, not representation. It can be implemented with an unsorted list, a sorted list, a binary heap, or another structure.

**Solution 3 [comparison]**

| Implementation | Push | Pop best | Main problem |
|---|---|---|---|
| Unsorted list | $O(1)$ | $O(n)$ | finding the best item is expensive |
| Sorted list | $O(n)$ | $O(1)$ | insertion is expensive |
| Binary heap | $O(\log n)$ | $O(\log n)$ | neither operation is constant |

A heap is a compromise because it does not make every operation cheap. Instead, it keeps both insertion and best-item removal efficient enough for repeated use.

## 14.2 Solutions for heap property and array representation

**Solution 4 [definition]**

1. In a min-heap, every parent is less than or equal to its children.
2. In a max-heap, every parent is greater than or equal to its children.
3. A complete binary tree has all levels full except possibly the last, and the last level is filled from left to right.
4. The heap property puts the best item at the root. The complete-tree shape keeps the height $O(\log n)$.

**Solution 5 [classification]**

1. `[1, 3, 2, 7, 5, 8]` is a valid min-heap.
2. `[3, 1, 2, 7, 5, 8]` is invalid because parent `3` at index `0` is greater than child `1` at index `1`.
3. `[1, 7, 2, 3, 5, 8]` is invalid because parent `7` at index `1` is greater than child `3` at index `3`.
4. `[1, 3, 8, 7, 5, 2]` is invalid because parent `8` at index `2` is greater than child `2` at index `5`.

**Solution 6 [concept]**

A BST requires every value in the left subtree to be smaller than the node and every value in the right subtree to be larger. A heap only requires each parent to be better than its direct children. A heap does not say that all values in the left subtree must be smaller than all values in the right subtree. Therefore a valid heap can be invalid as a BST.

**Solution 7 [trace]**

The tree represented by `[2, 5, 3, 9, 7, 8]` is:

```
      2
     / \
    5   3
   / \ /
  9  7 8
```

The list is not sorted because `3` appears after `5`. It is still a valid min-heap: `2 <= 5`, `2 <= 3`, `5 <= 9`, `5 <= 7`, and `3 <= 8`. Only parent-child comparisons matter.

**Solution 8 [calculation]**

For index `i`:

1. left child: `2*i + 1`;

2. right child: `2*i + 2`;
3. parent: `(i - 1) // 2`.

The parent of index `6` is `(6 - 1) // 2 = 2`. The children of index `4` are `9` and `10`. In a heap of length `6`, index `1` has children `3` and `4`, and both exist.

**Solution 9 [implementation]**

```
def parent(i):
    return (i - 1) // 2


def left(i):
    return 2*i + 1


def right(i):
    return 2*i + 2


def has_left(heap, i):
    return left(i) < len(heap)


def has_right(heap, i):
    return right(i) < len(heap)
```

**Solution 10 [implementation]**

```
def is_heap(heap):
    n = len(heap)
    for i in range(n):
        l = 2*i + 1
        r = 2*i + 2
        if l < n and heap[i] > heap[l]:
            return False
        if r < n and heap[i] > heap[r]:
            return False
    return True
```

It is enough to check parent-child pairs. The list does not need to be sorted.

## 14.3 Solutions for insertion, removal, and heap repairs

**Solution 11 [trace]**

After appending `1`:

```
[2, 5, 3, 9, 7, 8, 1]
```

The new value is at index `6`; its parent is index `2`, value `3`. Since `1 < 3`, swap:

```
[2, 5, 1, 9, 7, 8, 3]
```

Now `1` is at index `2`; its parent is index `0`, value `2`. Since `1 < 2`, swap:

```
[1, 5, 2, 9, 7, 8, 3]
```

Sift up follows one path because a newly inserted value can only violate the heap property with its ancestors, not with unrelated nodes.

**Solution 12 [implementation]**

```
def sift_up(heap, index):
    while index > 0:
        p = (index - 1) // 2
        if heap[p] <= heap[index]:
            break
        heap[p], heap[index] = heap[index], heap[p]
        index = p


def heap_push(heap, value):
    heap.append(value)
    sift_up(heap, len(heap) - 1)
```

Both functions modify the heap in place.

**Solution 13 [trace]**

The returned value is `1`, the root.
After moving the last value `3` to the root and removing the last position:

```
[3, 5, 2, 9, 7, 8]
```

Compare `3` with its children `5` and `2`. The smaller child is `2`, so swap:

```
[2, 5, 3, 9, 7, 8]
```

Now `3` has child `8`; since `3 <= 8`, stop. The final heap is `[2, 5, 3, 9, 7, 8]`.

**Solution 14 [implementation]**

```
def sift_down(heap, index):
    n = len(heap)
    while True:
        l = 2*index + 1
        r = 2*index + 2
        smallest = index

        if l < n and heap[l] < heap[smallest]:
            smallest = l
        if r < n and heap[r] < heap[smallest]:
            smallest = r

        if smallest == index:
            break

        heap[index], heap[smallest] = heap[smallest], heap[index]
        index = smallest


def heap_pop(heap):
    if not heap:
        raise IndexError("pop from empty heap")

    result = heap[0]
    last = heap.pop()

    if heap:
        heap[0] = last
        sift_down(heap, 0)

    return result
```

The special case for one element avoids writing back into an empty list.

## 14.4 Solutions for Python heapq and priority scheduling

**Solution 15 [code reading]**

The first line prints:

```
2
```

The second line also prints:

```
2
```

After popping 2, one possible remaining heap is:

```
[5, 8]
```

The list need not be sorted in general. A heap only guarantees that the root is the minimum and that every parent is less than or equal to its children.

**Solution 16 [code reading]**

The output order is:

```
fix server
answer email
write report
```

Python compares tuples lexicographically, so the first component, the priority number, controls the heap order. If larger numeric priority should mean more urgent, store negative priorities, for example `(-priority, task)`.

**Solution 17 [concept]**

The counter gives stable tie-breaking. If two entries have the same priority, Python compares the next tuple field. If the next field is a task object that is not comparable, the heap may raise an error. Even when tasks are comparable strings, the result may be alphabetical rather than insertion order. A counter makes equal-priority tasks come out in the order they were inserted.

**Solution 18 [trace]**

The execution order is:

```
security alert
restart server
backup files
clean logs
```

This is not FIFO order because the first arriving job, `backup files`, does not execute first. The scheduler chooses the smallest priority number first.

**Solution 19 [implementation]**

One complete implementation is:

```
class PriorityQueue:
    def __init__(self):
        self._heap = []
        self._counter = 0

    def _sift_up(self, index):
        heap = self._heap
        while index > 0:
            p = (index - 1) // 2
            if heap[p] <= heap[index]:
                break
```

```
            heap[p], heap[index] = heap[index], heap[p]
            index = p

    def _sift_down(self, index):
        heap = self._heap
        n = len(heap)
        while True:
            l = 2*index + 1
            r = 2*index + 2
            smallest = index

            if l < n and heap[l] < heap[smallest]:
                smallest = l
            if r < n and heap[r] < heap[smallest]:
                smallest = r

            if smallest == index:
                break

            heap[index], heap[smallest] = heap[smallest], heap[index]
            index = smallest

    def push(self, item, priority):
        entry = (priority, self._counter, item)
        self._counter += 1
        self._heap.append(entry)
        self._sift_up(len(self._heap) - 1)

    def peek(self):
        if self.is_empty():
            raise IndexError("peek from empty priority queue")
        return self._heap[0][2]

    def pop(self):
        if self.is_empty():
            raise IndexError("pop from empty priority queue")

        result = self._heap[0]
        last = self._heap.pop()
        if self._heap:
            self._heap[0] = last
            self._sift_down(0)
        return result[2]

    def is_empty(self):
        return len(self._heap) == 0
```

The heap compares entries by priority first, then by counter, then by item only if both earlier fields tie. Since the counter is unique, the item is never needed for comparison.

## 14.5 Solutions for heapify, top-k, complexity, and limits

**Solution 20 [comparison]**

| Operation | Cost | Reason |
|---|---|---|
| `heap[0]` / peek | $O(1)$ | best item is at the root |
| `heap_push` | $O(\log n)$ | sift up follows one root-to-leaf path upward |
| `heap_pop` | $O(\log n)$ | sift down follows one root-to-leaf path downward |
| `heapify` | $O(n)$ | bottom-up construction does less work near leaves |
| Pop all items | $O(n \log n)$ | $n$ removals, each logarithmic |

`heap_push` and `heap_pop` are logarithmic because a complete binary tree with $n$ nodes has height $O(\log n)$, and each repair touches at most one node per level.

**Solution 21 [implementation]**

```
def heapify(values):
    last_parent = len(values) // 2 - 1
    for i in range(last_parent, -1, -1):
        sift_down(values, i)
```

Bottom-up heapify is $O(n)$ because most nodes are near the bottom and can move only a small distance. Only a few nodes near the root can move $O(\log n)$ levels. Adding the work over all levels gives a linear total, not $n$ independent logarithmic repairs.

**Solution 22 [implementation]**

```
def top_k(values, k):
    if k <= 0:
        return []
    if k >= len(values):
        return sorted(values, reverse=True)

    heap = []
    for value in values:
        if len(heap) < k:
            heap_push(heap, value)
        elif value > heap[0]:
            heap_pop(heap)
            heap_push(heap, value)

    result = []
    while heap:
        result.append(heap_pop(heap))

    result.reverse()
    return result
```

The heap stores only the best $k$ values seen so far. The root is the smallest among those

$k$ values, so a new value only matters if it is larger than the root.

**Solution 23 [concept]**

Heaps are bad at:

1. searching for a specific value,
2. printing all values in sorted order without popping,
3. removing an arbitrary internal value,
4. updating the priority of an arbitrary item already inside the heap,
5. answering general order queries such as the second smallest value.

For example, searching for a specific value is not efficient because the heap property only compares parents with children. If a target is larger than the root, it could be almost anywhere below the root. The heap gives no BST-style direction for search.

**Solution 24 [synthesis]**

1. Process tasks strictly by arrival order: queue. FIFO is exactly arrival order.
2. Undo the most recent editor action: stack. Undo uses the most recent unfinished action, so LIFO fits.
3. Repeatedly execute the most urgent pending task: priority queue. The next task is chosen by priority, not arrival.
4. Search for a specific value while maintaining reliable logarithmic performance: AVL tree. It is a balanced BST.
5. Repeatedly extract the smallest timestamp in a simulation: binary heap or priority queue. This is repeated best-item removal.
6. Store values so that inorder traversal gives sorted order, but without guaranteed balance: BST. Inorder is sorted because of the BST invariant, but the tree may degenerate.

Chapter 15

# Graphs I: Representation: Review Questions

This chapter reviews Lecture 8 of COMP102. The focus is on modeling relationships as graphs, choosing vertices and edges, distinguishing directed and undirected graphs, handling edge weights, representing graphs with adjacency lists and adjacency matrices, and choosing the right representation for a problem.

Solutions are intentionally placed in the following chapter.

## 15.1 Graphs as models

**Learning outcome 15.1: Define a graph and identify its parts**

Students should be able to define a graph as a set of vertices and edges, and identify vertices, edges, neighbors, degree, and paths in a small graph.

**Question 1 [definition] Basic graph vocabulary**

Answer each question using this undirected graph:

```
A -- B -- D
|         |
C --------
```

1. What are the vertices?
2. What are the edges?
3. What are the neighbors of `A`?
4. What are the neighbors of `D`?
5. What is the degree of each vertex?
6. Give one path from `A` to `D` of length 2.

**Question 2 [concept] Why graphs are not just drawings**

A graph can model roads, friendships, web pages, course prerequisites, or game states. For each situation below, say what the vertices and edges could represent.

1. A road map between cities.

2. A social network where users follow each other.
3. A website where pages link to other pages.
4. A university curriculum with prerequisite relationships.
5. A chessboard where a knight can move from one square to another.

Then answer: why does the same graph drawing not have one fixed meaning?

**Question 3 [calculation] Degree and path length**

Consider this undirected graph:

```
A -- B -- C
|    |    |
D -- E -- F
```

Assume the edges are:

$$(A, B), (B, C), (A, D), (B, E), (C, F), (D, E), (E, F).$$

1. Compute $\deg(A)$, $\deg(B)$, $\deg(E)$, and $\deg(F)$.
2. Give a path from `A` to `F` of length 3.
3. Give a different path from `A` to `F`.
4. Is `A -> B -> E -> F` a valid path? Explain.

## 15.2 Direction and weight

**Learning outcome 15.2: Distinguish directed, undirected, weighted, and unweighted graphs**

Students should be able to decide whether edges should have direction, whether they should have weights, and what those weights mean in the model.

**Question 4 [classification] Direction in graph models**

For each relation, decide whether the graph should usually be directed or undirected. Justify briefly.

1. Two cities connected by a two-way road.
2. User `A` follows user `B` on a social media platform.
3. Course `COMP101` is a prerequisite for `COMP102`.
4. File `main.py` imports file `utils.py`.
5. Two people are siblings.

**Question 5 [concept] Weights depend on the question**

Suppose a graph models a campus map. The vertices are buildings and the edges are walkable paths.
For each question below, say what the edge weights should represent.

1. Find the physically shortest route.
2. Find the fastest route at noon.
3. Find the safest route at night.
4. Find the route with the fewest stairs.

Then answer: why is it wrong to assume that every graph weight means distance?

**Question 6 [representation] Directed and weighted edge lists**

You are given these directed weighted edges:

$$(A, B, 5), \quad (A, C, 2), \quad (B, D, 7), \quad (C, D, 1).$$

1. Write the directed weighted adjacency list.
2. Should `A` appear automatically in `B`'s neighbor list? Explain.
3. What are the outgoing neighbors of `A`?
4. What are the incoming neighbors of `D`?

## 15.3 Adjacency lists

**Learning outcome 15.3: Represent graphs using adjacency lists**

Students should be able to read, build, and interpret adjacency lists for directed, undirected, weighted, and unweighted graphs.

**Question 7 [conversion] Build an undirected adjacency list by hand**

Let

$$V = \{A, B, C, D, E\}$$

and let the undirected edges be:

$$(A, B), \quad (A, C), \quad (B, D), \quad (C, D).$$

1. Write the adjacency list as a Python dictionary.
2. Which vertex is isolated?
3. Why should the isolated vertex still appear in the dictionary?
4. For each undirected edge, how many list entries are stored?

### Question 8 [implementation] Build an undirected graph

Implement `build_undirected_graph(vertices, edges)`.
Rules:

- `vertices` is a list of vertex names.
- `edges` is a list of pairs `(u, v)`.
- Return an adjacency-list dictionary.
- Preserve isolated vertices.

```
def build_undirected_graph(vertices, edges):
    pass
```

Example:

```
vertices = ["A", "B", "C", "D", "E"]
edges = [("A", "B"), ("A", "C"), ("B", "D")]
```

### Question 9 [implementation] Build a directed graph

Implement `build_directed_graph(vertices, edges)`.
Rules:

- For an edge `u -> v`, store `v` in `graph[u]`.
- Do not automatically store `u` in `graph[v]`.
- Preserve vertices with no outgoing edges.

```
def build_directed_graph(vertices, edges):
    pass
```

### Question 10 [implementation] Build a weighted adjacency list

Implement `build_weighted_undirected_graph(vertices, edges)`.
Each edge is a triple `(u, v, w)`, where `w` is the weight.

```
def build_weighted_undirected_graph(vertices, edges):
    pass
```

For this input:

```
vertices = ["A", "B", "C", "D"]
edges = [("A", "B", 5), ("A", "C", 2), ("C", "D", 1)]
```

1. What should `graph["A"]` contain?
2. What should `graph["D"]` contain?

## 15.4 Adjacency matrices

**Learning outcome 15.4: Represent graphs using adjacency matrices**

Students should be able to read and build adjacency matrices, interpret symmetry, and explain how weighted matrices differ from unweighted matrices.

**Question 11 [conversion] Read an adjacency matrix**

Use the vertex order:

$$A, B, C, D$$

and the matrix:

| | $A$ | $B$ | $C$ | $D$ |
|---|---|---|---|---|
| $A$ | 0 | 1 | 1 | 0 |
| $B$ | 1 | 0 | 0 | 1 |
| $C$ | 1 | 0 | 0 | 1 |
| $D$ | 0 | 1 | 1 | 0 |

1. List all edges represented by the matrix.
2. Is this graph directed or undirected? Explain using symmetry.
3. What are the neighbors of `A`?
4. What is the degree of `D`?

**Question 12 [implementation] Build an undirected adjacency matrix**

Implement `build_undirected_matrix(vertices, edges)`.
Rules:

- Use the order of `vertices` as the row and column order.
- Use `1` when an edge exists and `0` otherwise.
- Since the graph is undirected, update both `matrix[i][j]` and `matrix[j][i]`.

```
def build_undirected_matrix(vertices, edges):
    pass
```

**Question 13 [implementation] Check whether an edge exists**

Suppose a graph is represented by an adjacency matrix and a dictionary `index` mapping each vertex to its matrix position.
Implement `has_edge(matrix, index, u, v)`.

```
def has_edge(matrix, index, u, v):
    pass
```

Then answer: why is checking whether `u -> v` exists usually $O(1)$ with an adjacency matrix?

**Question 14 [concept] Weighted matrices and missing edges**

A weighted graph can be represented with this matrix:

| | $A$ | $B$ | $C$ | $D$ |
|---|---|---|---|---|
| $A$ | 0 | 5 | 2 | $\infty$ |
| $B$ | 5 | 0 | $\infty$ | 7 |
| $C$ | 2 | $\infty$ | 0 | 1 |
| $D$ | $\infty$ | 7 | 1 | 0 |

1. What is the weight of edge `A - C`?
2. Is there an edge between `A` and `D`?
3. Why is $\infty$ used for missing edges?
4. Why can using `0` for missing edges be dangerous?

## 15.5 Representation tradeoffs

**Learning outcome 15.5: Choose between adjacency lists and adjacency matrices**

Students should be able to compare adjacency lists and adjacency matrices using memory cost, edge lookup cost, neighbor iteration cost, and graph density.

**Question 15 [comparison] Adjacency list versus adjacency matrix**

Let $n = |V|$ and $m = |E|$.
Complete the table.

| Operation | Adjacency list | Adjacency matrix |
|---|---|---|
| Memory | | |
| Check edge $u \to v$ | | |
| Iterate neighbors of $u$ | | |
| Add edge | | |

Then answer: which representation is usually better for sparse graphs, and why?

**Question 16 [classification] Choose the representation**

For each situation, choose adjacency list or adjacency matrix. Justify briefly.

1. A road map with thousands of cities, where each city connects to only a few nearby cities.
2. A dense graph with 500 vertices where almost every pair of vertices is connected.
3. A graph algorithm that frequently asks: “is there an edge from `u` to `v`?”
4. A graph algorithm that frequently loops over all neighbors of the current vertex.
5. A course-prerequisite graph where most courses have only a few prerequisites.

**Question 17 [concept] Sparse and dense graphs**

Explain the difference between a sparse graph and a dense graph.
Your answer should mention:

1. the number of possible edges compared with the number of actual edges,
2. why many real-world graphs are sparse,
3. why adjacency matrices can waste memory on sparse graphs.

## 15.6 Modeling practice

**Learning outcome 15.6: Translate a problem into a graph model**

Students should be able to identify vertices, edges, direction, weights, and representation choices before writing graph code.

**Question 18 [modeling] Graph detective**

For each problem, identify the vertices, the edges, whether the graph is directed, and whether it should be weighted.

1. **Word ladder.** Words have the same length. One move changes exactly one letter.
2. **Maze.** Open cells are connected if one can move between them by one step up, down, left, or right.
3. **Course planning.** Some courses must be completed before others.
4. **Package imports.** One Python file imports another Python file.
5. **Campus navigation.** Buildings are connected by paths with walking times.

**Question 19 [implementation] Convert an adjacency list to an edge list**

Implement `edge_list_from_directed_graph(graph)`.
The input is a directed adjacency list, such as:

```
graph = {
    "A": ["B", "C"],
    "B": ["D"],
    "C": ["D"],
    "D": [],
}
```

The function should return a list of directed edges `(u, v)`.

```
def edge_list_from_directed_graph(graph):
    pass
```

**Question 20 [implementation] Convert an adjacency matrix to an adjacency list**

Implement `matrix_to_adjacency_list(vertices, matrix)` for an unweighted directed graph.
Rules:

- `vertices[i]` is the vertex for row `i` and column `i`.
- If `matrix[i][j] == 1`, then there is an edge from `vertices[i]` to `vertices[j]`.
- Preserve vertices with no outgoing edges.

```
def matrix_to_adjacency_list(vertices, matrix):
    pass
```

Chapter 16

# Solutions: Graphs I: Representation

This chapter gives solutions to the review questions from the previous chapter. The solutions emphasize graph modeling, direction, weights, adjacency lists, adjacency matrices, representation tradeoffs, and basic graph-building code.

## 16.1 Solutions for graphs as models

**Solution 1 [definition]**

1. The vertices are $\{A, B, C, D\}$.
2. The edges are $(A, B), (A, C), (B, D), (C, D)$.
3. The neighbors of `A` are `B` and `C`.
4. The neighbors of `D` are `B` and `C`.
5. The degrees are $\deg(A) = 2$, $\deg(B) = 2$, $\deg(C) = 2$, and $\deg(D) = 2$.
6. One path from `A` to `D` of length 2 is `A -> B -> D`. Another is `A -> C -> D`.

**Solution 2 [concept]**

1. Road map: vertices are cities or intersections; edges are roads.
2. Social network: vertices are users; edges are follows or friendships.
3. Website: vertices are pages; edges are links from one page to another.
4. Curriculum: vertices are courses; edges are prerequisite relationships.
5. Knight moves: vertices are board squares; edges connect squares when a knight can legally move between them.

The same drawing does not have one fixed meaning because vertices and edges are chosen by the modeler. A node can mean a city, a person, a file, a course, or a game state depending on the problem.

**Solution 3 [calculation]**

1. $\deg(A) = 2$, because `A` touches `B` and `D`. $\deg(B) = 3$, because `B` touches `A`, `C`, and `E`. $\deg(E) = 3$, because `E` touches `B`, `D`, and `F`. $\deg(F) = 2$, because `F` touches `C` and `E`.
2. One path from `A` to `F` of length 3 is `A -> B -> E -> F`.
3. A different path is `A -> D -> E -> F`.

4. Yes. Each consecutive pair is connected by an edge: $(A, B)$, $(B, E)$, and $(E, F)$.

## 16.2 Solutions for direction and weight

**Solution 4 [classification]**

1. Usually undirected, because a two-way road can be traveled in both directions.
2. Directed, because `A` following `B` does not imply `B` follows `A`.
3. Directed, because the prerequisite relation has direction: prerequisite course → later course.
4. Directed, because `main.py` importing `utils.py` does not imply `utils.py` imports `main.py`.
5. Usually undirected, because if one person is a sibling of another, the reverse relation also holds.

**Solution 5 [concept]**

1. For the physically shortest route, weights should represent physical distance.
2. For the fastest route at noon, weights should represent expected walking time, possibly including crowding.
3. For the safest route at night, weights could represent risk or lack of safety, so lower weight means safer.
4. For the fewest stairs, weights could represent number of stairs or stair difficulty.

It is wrong to assume every weight means distance because a weight is just a value attached to an edge. Its meaning comes from the problem: distance, time, cost, risk, capacity, or difficulty.

**Solution 6 [representation]**

The directed weighted adjacency list is:

```
graph = {
    "A": [("B", 5), ("C", 2)],
    "B": [("D", 7)],
    "C": [("D", 1)],
    "D": [],
}
```

1. Since the graph is directed, `A` should not automatically appear in `B`'s neighbor list. The edge `A -> B` does not imply `B -> A`.
2. The outgoing neighbors of `A` are `B` and `C`.
3. The incoming neighbors of `D` are `B` and `C`.

## 16.3 Solutions for adjacency lists

**Solution 7 [conversion]**

One valid adjacency list is:

```
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C"],
    "E": [],
}
```

The isolated vertex is `E`. It should still appear so that the graph representation preserves the full vertex set, not only vertices that have edges. Each undirected edge is stored twice: once in each endpoint's neighbor list.

**Solution 8 [implementation]**

```
def build_undirected_graph(vertices, edges):
    graph = {v: [] for v in vertices}

    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    return graph
```

The initialization step creates an empty list for every vertex, including isolated vertices.

**Solution 9 [implementation]**

```
def build_directed_graph(vertices, edges):
    graph = {v: [] for v in vertices}

    for u, v in edges:
        graph[u].append(v)

    return graph
```

Only the outgoing direction is stored. A vertex with no outgoing edges still appears with an empty list.

**Solution 10 [implementation]**

```
def build_weighted_undirected_graph(vertices, edges):
    graph = {v: [] for v in vertices}

    for u, v, w in edges:
```

```
        graph[u].append((v, w))
        graph[v].append((u, w))

    return graph
```

For the given input, `graph["A"]` should contain `[("B", 5), ("C", 2)]`, possibly in that order. `graph["D"]` should contain `[("C", 1)]`.

## 16.4 Solutions for adjacency matrices

**Solution 11 [conversion]**

The edges are:

$$(A, B), (A, C), (B, D), (C, D).$$

The graph is undirected because the matrix is symmetric: whenever entry $(i, j)$ is 1, entry $(j, i)$ is also 1. The neighbors of `A` are `B` and `C`. The degree of `D` is 2, because `D` is connected to `B` and `C`.

**Solution 12 [implementation]**

```
def build_undirected_matrix(vertices, edges):
    n = len(vertices)
    index = {v: i for i, v in enumerate(vertices)}
    matrix = [[0] * n for _ in range(n)]

    for u, v in edges:
        i = index[u]
        j = index[v]
        matrix[i][j] = 1
        matrix[j][i] = 1

    return matrix
```

The dictionary `index` converts vertex names into row and column positions.

**Solution 13 [implementation]**

```
def has_edge(matrix, index, u, v):
    i = index[u]
    j = index[v]
    return matrix[i][j] != 0
```

The lookup is usually $O(1)$ because once `u` and `v` are converted into indexes, checking `matrix[i][j]` is a direct table access.

**Solution 14 [concept]**

1. The weight of `A - C` is 2.
2. There is no edge between `A` and `D`; the entry is $\infty$.
3. $\infty$ means the cost is not finite because the edge does not exist.
4. Using `0` for missing edges is dangerous because a graph may allow a real edge with weight `0`. Then `0` would be ambiguous: it could mean "no edge" or "zero-cost edge."

## 16.5 Solutions for representation tradeoffs

**Solution 15 [comparison]**

| Operation | Adjacency list | Adjacency matrix |
|---|---|---|
| Memory | $O(n + m)$ | $O(n^2)$ |
| Check edge $u \to v$ | $O(\deg(u))$ | $O(1)$ |
| Iterate neighbors of $u$ | $O(\deg(u))$ | $O(n)$ |
| Add edge | usually $O(1)$ | $O(1)$ |

Adjacency lists are usually better for sparse graphs because they store only existing edges. A matrix reserves space for every possible pair of vertices, even when most edges are absent.

**Solution 16 [classification]**

1. Use an adjacency list. The graph is sparse because each city connects to only a few nearby cities.
2. Use an adjacency matrix. The graph is dense, so the $O(n^2)$ space is less wasteful and edge lookups are fast.
3. Use an adjacency matrix if edge lookup is the dominant operation, because checking $u \to v$ is $O(1)$.
4. Use an adjacency list if neighbor iteration is frequent, because it scans only actual neighbors.
5. Use an adjacency list. Course-prerequisite graphs are usually sparse.

**Solution 17 [concept]**

A sparse graph has far fewer actual edges than possible edges: $m \ll n^2$. A dense graph has many of the possible edges, with $m$ close to $n^2$. Many real-world graphs are sparse because roads, imports, prerequisites, and social connections usually connect each object to only a limited number of others. An adjacency matrix can waste memory on sparse graphs because it stores entries for all vertex pairs, including pairs with no edge.

## 16.6 Solutions for modeling practice

**Solution 18 [modeling]**

1. Word ladder: vertices are words. Edges connect words that differ by exactly one letter. Usually undirected and unweighted if each move costs one step.
2. Maze: vertices are open cells. Edges connect cells that can be reached by one legal move. Usually undirected and unweighted for ordinary movement.
3. Course planning: vertices are courses. Edges go from prerequisite to dependent course. Directed and usually unweighted.
4. Package imports: vertices are files or modules. Edges go from importing file to imported file. Directed and usually unweighted.
5. Campus navigation: vertices are buildings or locations. Edges are walkable paths. Usually undirected if paths are two-way, and weighted if walking time matters.

**Solution 19 [implementation]**

```
def edge_list_from_directed_graph(graph):
    edges = []

    for u in graph:
        for v in graph[u]:
            edges.append((u, v))

    return edges
```

For the example graph, one valid output is:

```
[("A", "B"), ("A", "C"), ("B", "D"), ("C", "D")]
```

assuming dictionary iteration follows the insertion order shown.

**Solution 20 [implementation]**

```
def matrix_to_adjacency_list(vertices, matrix):
    graph = {v: [] for v in vertices}

    for i in range(len(vertices)):
        u = vertices[i]
        for j in range(len(vertices)):
            if matrix[i][j] == 1:
                v = vertices[j]
                graph[u].append(v)

    return graph
```

This preserves vertices with no outgoing edges because every vertex is initialized before reading the matrix.

Chapter 17

# Graphs II: Algorithms: Review Questions

This chapter reviews Lecture 9 of COMP102. The focus is on graph traversal, visited sets, BFS, shortest paths in unweighted graphs, parent reconstruction, DFS, connected components, traversal complexity, and the basic idea of Dijkstra's algorithm for non-negative weighted graphs.

Solutions are intentionally placed in the following chapter.

## 17.1 Traversal, frontier, and visited sets

**Learning outcome 17.1: Explain graph traversal and the role of the frontier**

Students should be able to explain what graph traversal does, why a visited set is needed, and how the frontier controls the order in which vertices are explored.

**Question 1 [concept] Traversal vocabulary**

Consider this adjacency list:

```
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "E"],
    "D": ["B"],
    "E": ["C"],
}
```

Answer the following questions.

1. What does `graph["C"]` represent?
2. What does it mean for a vertex to be *unseen*?
3. What does it mean for a vertex to be in the *frontier*?
4. What does it mean for a vertex to be *done*?
5. Why can a traversal get stuck forever if it does not remember visited vertices?

**Question 2 [concept] BFS versus DFS frontier discipline**

Both BFS and DFS repeatedly choose one vertex from the frontier and inspect its neighbors.
Explain the difference between these two choices:

1. BFS chooses the oldest discovered vertex first.
2. DFS chooses the newest discovered vertex first.

Your answer should mention the queue used by BFS and the stack or recursion used by DFS.

## 17.2 Breadth-first search

**Learning outcome 17.2: Trace BFS on directed and undirected graphs**

Students should be able to trace BFS using a queue, mark vertices when they are inserted into the queue, and produce both queue states and traversal order.

**Question 3 [trace] BFS queue trace on an undirected graph**

Consider the following undirected graph represented as an adjacency list:

```
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "E"],
    "D": ["B", "F"],
    "E": ["C"],
    "F": ["D"],
}
```

Run BFS starting from `A`.
Rules:

- Process neighbors in alphabetical order.
- Mark a vertex as visited when it is inserted into the queue.

1. Write the content of the queue after each vertex is processed.
2. Write the BFS traversal order.

**Question 4 [trace] BFS queue trace on a directed graph**

Consider the following directed graph:

```
graph = {
    "S": ["A", "B", "C"],
    "A": ["D", "E"],
    "B": ["E", "F"],
    "C": ["B", "G"],
    "D": ["H"],
```

```
    "E": ["H"],
    "F": ["I"],
    "G": ["I"],
    "H": ["T"],
    "I": ["T"],
    "T": [],
}
```

Run BFS starting from `S`.
Rules:

- Follow only outgoing edges.
- Process outgoing neighbors in alphabetical order.
- Mark a vertex as visited when it is inserted into the queue.

1. Write the content of the queue after each vertex is processed.
2. Write the BFS traversal order.

**Question 5 [implementation] Implement BFS traversal order**

Implement `bfs_order(graph, start)`.
The function should return the BFS traversal order as a list, not print it.
Rules:

- Use a queue.
- Mark a vertex as visited when it is inserted into the queue.
- Assume every vertex appears as a key in `graph`.

```
from collections import deque


def bfs_order(graph, start):
    pass
```

## 17.3 BFS distances and path reconstruction

**Learning outcome 17.3: Use BFS for shortest paths in unweighted graphs**

Students should be able to compute BFS distances in an unweighted graph, explain why BFS gives shortest paths by number of edges, and reconstruct a path using parent pointers.

**Question 6 [trace] BFS distance table**

Use this undirected graph:

```
graph = {
    "A": ["B", "C"],
```

```
    "B": ["A", "D"],
    "C": ["A", "E"],
    "D": ["B", "F"],
    "E": ["C", "F"],
    "F": ["D", "E"],
}
```

Run BFS from `A` and give the distance from `A` to every reachable vertex.
Distance means the minimum number of edges from `A`.

### Question 7 [implementation] Implement BFS distances

Implement `bfs_distances(graph, start)`.
The function should return a dictionary mapping each reachable vertex to its distance from `start`.

```
from collections import deque


def bfs_distances(graph, start):
    pass
```

### Question 8 [implementation] Implement shortest path reconstruction

Implement `shortest_path(graph, start, target)` for an unweighted graph.
The function should:

- use BFS,
- store a parent dictionary,
- return a list of vertices from `start` to `target`,
- return `None` if `target` is unreachable.

```
from collections import deque


def shortest_path(graph, start, target):
    pass
```

### Question 9 [concept] Why BFS gives shortest paths in unweighted graphs

Explain why BFS gives shortest paths in an unweighted graph.
Your answer should mention:

1. layers from the start vertex,
2. why all distance-$k$ vertices are processed before distance-$k+1$ vertices,
3. why this assumes every edge has the same cost,
4. why the same reasoning fails for weighted graphs.

## 17.4 Depth-first search

**Learning outcome 17.4: Trace and implement DFS**

Students should be able to trace recursive DFS, implement DFS with recursion or a stack, and explain why DFS does not guarantee shortest paths.

**Question 10 [trace] Recursive DFS trace on an undirected graph**

Consider the following undirected graph:

```
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "E"],
    "D": ["B"],
    "E": ["C"],
}
```

Run recursive DFS starting from `A`.
Rules:

- Process neighbors in alphabetical order.
- Mark a vertex as visited when DFS first enters it.

1. Write the recursive calls in order.
2. Write the DFS traversal order.

**Question 11 [trace] Recursive DFS trace on a directed graph**

Consider the following directed graph:

```
graph = {
    "A": ["B", "C", "D"],
    "B": ["E", "F"],
    "C": ["F", "G"],
    "D": ["C", "H"],
    "E": [],
    "F": ["H"],
    "G": ["B"],
    "H": [],
}
```

Run recursive DFS starting from `A`.
Rules:

- Follow only outgoing edges.
- Process outgoing neighbors in alphabetical order.
- Mark a vertex as visited when DFS first enters it.

1. Write the recursive calls in order.
2. Write the DFS traversal order.

**Question 12 [implementation] Implement recursive DFS order**

Implement `dfs_recursive_order(graph, start)`.
The function should return the DFS order as a list. Use a helper function if needed.

```
def dfs_recursive_order(graph, start):
    pass
```

**Question 13 [implementation] Implement iterative DFS order**

Implement `dfs_iterative_order(graph, start)` using a stack.
Assume the adjacency lists are already in the desired neighbor order. To obtain the same order as recursive DFS, push neighbors onto the stack in reverse order.

```
def dfs_iterative_order(graph, start):
    pass
```

**Question 14 [concept] DFS and shortest paths**

Explain why DFS does not guarantee the shortest path in an unweighted graph.
Give a small graph example where DFS can reach a target through a longer path before discovering a shorter path.

## 17.5 Connected components

**Learning outcome 17.5: Find connected components using repeated traversal**

Students should be able to define connected components in an undirected graph and compute them by running traversal from every unvisited vertex.

**Question 15 [trace] Identify connected components**

Consider this undirected graph:

```
graph = {
    "A": ["B"],
    "B": ["A", "C"],
    "C": ["B"],
    "D": ["E"],
    "E": ["D"],
    "F": [],
}
```

1. List the connected components.
2. How many connected components are there?
3. Why is `F` its own component?

**Question 16 [implementation] Implement connected components**

Implement `connected_components(graph)` for an undirected graph.
The function should return a list of components, where each component is a list of vertices discovered by a traversal.

```
def connected_components(graph):
    pass
```

**Question 17 [concept] Weak and strong connectivity**

For directed graphs, explain the difference between weak connectivity and strong connectivity.
Use this directed graph in your explanation:

```
A -> B
B -> C
C -> A
C -> D
```

Which vertices form a strongly connected group? Why is `D` different?

## 17.6 Complexity

**Learning outcome 17.6: Analyze BFS, DFS, and component complexity**

Students should be able to explain why BFS and DFS are $O(V + E)$ on adjacency lists, and why adjacency matrices change neighbor-scanning cost.

**Question 18 [analysis] Traversal complexity**

Answer the following questions.

1. Why is each vertex marked visited at most once in BFS or DFS?
2. Why does scanning adjacency lists across the whole traversal cost $O(E)$?
3. Why is the total time $O(V + E)$?
4. What extra space does BFS use?
5. What extra space does DFS use?
6. What changes if the graph is stored as an adjacency matrix?

**Question 19 [debugging] Visited when inserted versus visited when popped**

In BFS, the lecture and lab use the rule: mark a vertex as visited when it is inserted into the queue.
Consider this graph:

```
graph = {
```

```
    "A": ["B", "C"],
    "B": ["D"],
    "C": ["D"],
    "D": [],
}
```

Explain what can go wrong if `D` is not marked visited until it is popped from the queue.

## 17.7 Dijkstra's algorithm

**Learning outcome 17.7: Apply Dijkstra's algorithm on non-negative weighted graphs**

Students should be able to trace Dijkstra's algorithm, explain relaxation, use a priority queue, and distinguish weighted shortest paths from BFS shortest paths.

**Question 20 [trace] Dijkstra trace on an undirected weighted graph**

Consider this undirected weighted graph:

```
A--B: 2
A--C: 5
B--C: 1
B--D: 4
C--D: 2
D--E: 1
```

Use Dijkstra's algorithm to find the shortest path from `A` to `E`.
Rules:

- Use a priority queue ordered by distance.
- When distances are tied, remove the alphabetically smaller vertex first.
- Process neighbors in alphabetical order.
- If a new path has the same distance as the current best distance, do not update the parent.

1. Write the order in which vertices are settled.
2. Write the final distance table.
3. Write the shortest path from `A` to `E`.

**Question 21 [trace] Dijkstra trace on a directed weighted graph**

Consider this directed weighted graph:

```
S -> A: 2
S -> B: 1
S -> C: 5
A -> D: 4
```

```
A -> E: 7
B -> A: 1
B -> D: 2
B -> C: 2
C -> E: 2
D -> E: 1
D -> F: 5
E -> F: 1
F -> T: 2
E -> T: 6
```

Use Dijkstra's algorithm to find the shortest path from `S` to `T`.
Rules:

- Follow only outgoing edges.
- Use a priority queue ordered by `(distance, vertex name)`.
- Process outgoing neighbors in alphabetical order.
- If a new path has the same distance as the current best distance, do not update the parent.

1. Write the order in which vertices are settled.
2. Write the final distance table.
3. Write the shortest path from `S` to `T`.

### Question 22 [implementation] Implement Dijkstra distances

Implement `dijkstra(graph, start)`.
The graph is a weighted adjacency list where `graph[u]` contains pairs `(neighbor, weight)`.
Rules:

- All weights are non-negative.
- Return a dictionary of shortest distances from `start`.
- Use `heapq` as a priority queue.

```python
import heapq


def dijkstra(graph, start):
    pass
```

### Question 23 [comparison] BFS versus Dijkstra

Explain when to use BFS and when to use Dijkstra.
Your answer should mention:

1. unweighted graphs,
2. weighted graphs,

3. non-negative weights,
4. the queue in BFS,
5. the priority queue in Dijkstra,
6. why Dijkstra is not covered here for negative edge weights.

Chapter 18

# Solutions: Graphs II: Algorithms

This chapter gives solutions to the review questions from the previous chapter. The solutions emphasize traversal state, BFS queue behavior, shortest paths in unweighted graphs, parent reconstruction, DFS order, connected components, complexity, and Dijkstra's algorithm.

## 18.1 Solutions for traversal, frontier, and visited sets

**Solution 1 [concept]**

1. `graph["C"]` is the list of vertices directly reachable from `C`. Here, the only neighbor of `C` is `E`.
2. A vertex is unseen if the traversal has not discovered it yet.
3. A vertex is in the frontier if it has been discovered but its neighbors have not yet been fully explored.
4. A vertex is done after the algorithm has removed it from the frontier and inspected its neighbors.
5. Without a visited set, a traversal can loop around a cycle forever. For example, in an undirected edge `A - B`, the algorithm can move from `A` to `B`, then back to `A`, then back to `B`, and so on.

**Solution 2 [concept]**

BFS and DFS differ mainly by how they choose the next vertex from the frontier.
BFS uses a queue. The oldest discovered vertex is processed first, so the search expands layer by layer from the start vertex.
DFS uses a stack or recursion. The newest discovered vertex is processed first, so the search follows one branch deeply before returning to older alternatives.

## 18.2 Solutions for breadth-first search

**Solution 3 [trace]**

Initial queue:

```
[A]
```

Queue after each processed vertex:

| Processed vertex | Queue after processing |
|---|---|
| A | [B, C] |
| B | [C, D] |
| C | [D, E] |
| D | [E, F] |
| E | [F] |
| F | [] |

The BFS traversal order is:

```
A, B, C, D, E, F
```

**Solution 4 [trace]**

Initial queue:

```
[S]
```

Queue after each processed vertex:

| Processed vertex | Queue after processing |
|---|---|
| S | [A, B, C] |
| A | [B, C, D, E] |
| B | [C, D, E, F] |
| C | [D, E, F, G] |
| D | [E, F, G, H] |
| E | [F, G, H] |
| F | [G, H, I] |
| G | [H, I] |
| H | [I, T] |
| I | [T] |
| T | [] |

The BFS traversal order is:

```
S, A, B, C, D, E, F, G, H, I, T
```

**Solution 5 [implementation]**

```python
from collections import deque


def bfs_order(graph, start):
    visited = {start}
    queue = deque([start])
    order = []

    while queue:
        node = queue.popleft()
        order.append(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return order
```

The vertex is marked visited when it is inserted into the queue. This prevents the same vertex from being inserted multiple times through different incoming edges.

## 18.3 Solutions for BFS distances and path reconstruction

**Solution 6 [trace]**

Starting from `A`, the BFS layers are:

- distance 0: `A`
- distance 1: `B`, `C`
- distance 2: `D`, `E`
- distance 3: `F`

So the distance table is:

| Vertex | Distance from A |
|---|---|
| A | 0 |
| B | 1 |
| C | 1 |
| D | 2 |
| E | 2 |
| F | 3 |

### Solution 7 [implementation]

```python
from collections import deque


def bfs_distances(graph, start):
    distance = {start: 0}
    queue = deque([start])

    while queue:
        node = queue.popleft()

        for neighbor in graph[node]:
            if neighbor not in distance:
                distance[neighbor] = distance[node] + 1
                queue.append(neighbor)

    return distance
```

The dictionary `distance` also acts as the visited set: if a vertex already has a distance, it has already been discovered.

### Solution 8 [implementation]

```python
from collections import deque


def shortest_path(graph, start, target):
    parent = {start: None}
    queue = deque([start])

    while queue:
        node = queue.popleft()

        if node == target:
            break

        for neighbor in graph[node]:
            if neighbor not in parent:
                parent[neighbor] = node
                queue.append(neighbor)

    if target not in parent:
        return None

    path = []
    current = target
    while current is not None:
        path.append(current)
        current = parent[current]

```

```
    path.reverse()
    return path
```

The parent dictionary records how each vertex was first reached. Reconstruction starts at the target and walks backward through parents, then reverses the result.

**Solution 9 [concept]**

BFS gives shortest paths in unweighted graphs because it explores by layers. First it processes the start vertex at distance 0, then all vertices at distance 1, then all vertices at distance 2, and so on.
Before BFS begins processing distance $k + 1$, all vertices at distance $k$ have already been discovered. Therefore, the first time BFS discovers a vertex, it has found a path with the fewest possible number of edges.
This reasoning assumes every edge has equal cost. In a weighted graph, a path with fewer edges may cost more than a path with more edges. For weighted shortest paths with non-negative weights, Dijkstra's algorithm is the right tool.

## 18.4 Solutions for depth-first search

**Solution 10 [trace]**

Starting from `A`, recursive DFS processes neighbors alphabetically.
Recursive calls in order:

```
DFS(A)
DFS(B)
DFS(D)
DFS(C)
DFS(E)
```

The DFS traversal order is:

```
A, B, D, C, E
```

**Solution 11 [trace]**

Starting from `A`, recursive DFS follows outgoing edges and processes neighbors alphabetically.
Recursive calls in order:

```
DFS(A)
DFS(B)
DFS(E)
DFS(F)
DFS(H)
DFS(C)
```

```
DFS(G)
DFS(D)
```

The DFS traversal order is:

```
A, B, E, F, H, C, G, D
```

Explanation: after `A`, DFS enters `B`. From `B`, it enters `E`, then returns and enters `F`, then `H`. After finishing `B`'s branch, it returns to `A` and enters `C`, then `G`. Finally it enters `D`; both of `D`'s outgoing neighbors were already visited.

### Solution 12 [implementation]

```python
def dfs_recursive_order(graph, start):
    visited = set()
    order = []

    def visit(node):
        visited.add(node)
        order.append(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                visit(neighbor)

    visit(start)
    return order
```

The helper function represents the recursive DFS call. A vertex is marked visited as soon as DFS enters it.

### Solution 13 [implementation]

```python
def dfs_iterative_order(graph, start):
    visited = set()
    stack = [start]
    order = []

    while stack:
        node = stack.pop()

        if node in visited:
            continue

        visited.add(node)
        order.append(node)

        for neighbor in reversed(graph[node]):
            if neighbor not in visited:
                stack.append(neighbor)
```

```
    return order
```

The neighbors are pushed in reverse order because a stack pops the last inserted item first. This can make the iterative DFS match the usual recursive order.

**Solution 14 [concept]**

DFS does not guarantee shortest paths because it commits to one branch before checking all closer alternatives.
Example:

```
A: B, C
B: D
D: E
C: E
E:
```

If DFS starts at `A` and processes `B` before `C`, it may find `E` through:

```
A -> B -> D -> E
```

which has length 3. But the shorter path:

```
A -> C -> E
```

has length 2. BFS would find the shorter path first because it explores by distance layers.

## 18.5 Solutions for connected components

**Solution 15 [trace]**

The connected components are:

```
{A, B, C}
{D, E}
{F}
```

There are three connected components.
`F` is its own component because it has no edges. It can reach only itself, and no other vertex can reach it.

**Solution 16 [implementation]**

```
def connected_components(graph):
    visited = set()
    components = []

    for start in graph:
        if start in visited:
```

```
            continue

        component = []
        stack = [start]

        while stack:
            node = stack.pop()

            if node in visited:
                continue

            visited.add(node)
            component.append(node)

            for neighbor in graph[node]:
                if neighbor not in visited:
                    stack.append(neighbor)

        components.append(component)

    return components
```

The outer loop is necessary because one traversal only reaches one connected component. Repeated traversal finds the remaining components.

**Solution 17 [concept]**

Weak connectivity ignores edge directions. A directed graph is weakly connected if the graph would be connected after replacing every directed edge by an undirected edge. Strong connectivity respects edge directions. A group is strongly connected if every vertex in the group can reach every other vertex in the group by following directed edges. In the given graph, `A`, `B`, and `C` form a strongly connected group because:

```
A -> B -> C -> A
```

allows movement around the cycle. `D` is different because it is reachable from `C`, but there is no outgoing edge from `D` back to `A`, `B`, or `C`.

## 18.6 Solutions for complexity

**Solution 18 [analysis]**

1. Each vertex is marked visited at most once because the algorithm checks whether a vertex has already been visited before processing it again.
2. With adjacency lists, scanning neighbors across the traversal means scanning the stored edge entries. Across the whole graph, this is proportional to the number of edges.
3. The total time is $O(V + E)$: $O(V)$ for visiting vertices and $O(E)$ for scanning

adjacency lists.

4. BFS uses $O(V)$ extra space for the queue and visited set.
5. DFS uses $O(V)$ extra space for the visited set and either the recursion stack or an explicit stack.
6. With an adjacency matrix, finding the neighbors of one vertex requires scanning an entire row of length $V$. Traversing all vertices can therefore cost $O(V^2)$, even if the graph has few edges.

**Solution 19 [debugging]**

If vertices are marked visited only when they are popped, the same vertex can be inserted into the queue multiple times.
In the example, BFS starts from `A` and inserts `B` and `C`. When `B` is processed, it inserts `D`. If `D` has not been marked visited yet, then when `C` is processed, `C` can insert `D` again. The queue can contain duplicate copies of `D`.
Marking a vertex when it is inserted prevents this duplication. It means "this vertex is already scheduled for processing."

## 18.7 Solutions for Dijkstra's algorithm

**Solution 20 [trace]**

The shortest-path computation from `A` gives the following settled order:

```
A, B, C, D, E
```

Distance table:

| Vertex | Distance from A |
|---|---|
| A | 0 |
| B | 2 |
| C | 3 |
| D | 5 |
| E | 6 |

Parent relationships:

```
B <- A
C <- B
D <- C
E <- D
```

Therefore, the shortest path from `A` to `E` is:

```
A -> B -> C -> D -> E
```

Its total cost is:

$$2 + 1 + 2 + 1 = 6.$$

**Solution 21 [trace]**

The settled order is:

```
S, B, A, C, D, E, F, T
```

Distance table:

| Vertex | Distance from S |
|---|---|
| S | 0 |
| A | 2 |
| B | 1 |
| C | 3 |
| D | 3 |
| E | 4 |
| F | 5 |
| T | 7 |

Parent relationships on the shortest path tree include:

```
B <- S
A <- S
C <- B
D <- B
E <- D
F <- E
T <- F
```

Therefore, the shortest path from `S` to `T` is:

```
S -> B -> D -> E -> F -> T
```

Its total cost is:

$$1 + 2 + 1 + 1 + 2 = 7.$$

**Solution 22 [implementation]**

```python
import heapq


def dijkstra(graph, start):
    distance = {start: 0}
    pq = [(0, start)]

    while pq:
        current_dist, node = heapq.heappop(pq)

        if current_dist != distance[node]:
            continue

        for neighbor, weight in graph[node]:
            new_dist = current_dist + weight

```

```
            if neighbor not in distance or new_dist < distance[neighbor]:
                distance[neighbor] = new_dist
                heapq.heappush(pq, (new_dist, neighbor))

    return distance
```

The priority queue always expands the unsettled vertex with the smallest known distance so far. The check `current_dist != distance[node]` skips stale priority-queue entries created before a better path was found.

**Solution 23 [comparison]**

Use BFS for shortest paths when the graph is unweighted, or when every edge has the same cost. BFS uses a normal queue because all vertices in the same layer have the same distance from the start.
Use Dijkstra for weighted shortest paths when all edge weights are non-negative. Dijkstra uses a priority queue because the next vertex to expand should be the one with the smallest known total distance, not simply the one discovered earliest.
Dijkstra is not used here for negative edge weights because a later negative edge can invalidate a distance that looked final. Handling negative weights requires different algorithms, which are outside this lecture.

Chapter 19

# Sorting Algorithms: Review Questions

This chapter reviews Lecture 10 of COMP102. The focus is on efficient sorting by divide and conquer: merge sort, quicksort, partitioning, recurrence analysis, the Master Theorem, best/average/worst-case complexity, stability, and memory tradeoffs.

Solutions are intentionally placed in the following chapter.

## 19.1 Sorting baselines and divide and conquer

**Learning outcome 19.1: Explain why efficient sorting matters**

Students should be able to compare quadratic sorting with $n \log n$ sorting and explain why selection sort and insertion sort become inadequate on large inputs.

**Question 1 [concept] Quadratic sorting versus $n \log n$**

Selection sort and insertion sort are useful baselines, but their average and worst-case running times are quadratic.
Answer the following questions.

1. What does $O(n^2)$ mean informally for a sorting algorithm?
2. What does $O(n \log n)$ mean informally for a sorting algorithm?
3. Why is $O(n^2)$ not merely a small slowdown compared with $O(n \log n)$ when $n$ is large?
4. Give one reason insertion sort can still be useful despite its quadratic worst case.

**Learning outcome 19.2: Describe divide-and-conquer sorting**

Students should be able to describe the divide, solve, and combine structure and explain how merge sort and quicksort place their linear work in different parts of the algorithm.

**Question 2 [concept] Divide and conquer in merge sort and quicksort**

Both merge sort and quicksort are divide-and-conquer sorting algorithms.
Compare them by answering the following questions.

1. How does merge sort divide the input?
2. How does quicksort divide the input?
3. In merge sort, when does the main linear work happen?
4. In quicksort, when does the main linear work happen?
5. Why does quicksort not need a final merge step?

## 19.2 Merge sort

**Learning outcome 19.3: Trace merge sort**

Students should be able to trace the splitting and merging phases of merge sort and identify the base case.

**Question 3 [trace] Merge sort trace**

Trace merge sort on the following list:

```
[7, 2, 9, 4, 3, 8]
```

Answer the following questions.

1. Show the recursive splitting until all sublists have size one.
2. Show the merging steps that rebuild the sorted list.
3. What is the base case of merge sort?
4. What is the final sorted list?

**Learning outcome 19.4: Implement the merge operation**

Students should be able to merge two sorted lists using two indexes and explain why the merge step is linear.

**Question 4 [implementation] Implement merge**

Implement `merge(left, right)`.
The function receives two sorted lists and returns one sorted list containing all elements from both inputs.
Rules:

- Use two indexes, one for `left` and one for `right`.
- Do not call `sort` or `sorted`.
- If two values are equal, take the value from `left` first.

```
def merge(left, right):
    pass
```

**Question 5 [implementation] Implement merge sort**

Implement `merge_sort(a)` using the clear slicing-based version from the lecture.
Rules:

- If the list has length zero or one, return it.
- Split the list into two halves.
- Recursively sort the two halves.
- Return the result of merging them.
- Do not mutate the input list.

```
def merge(left, right):
    # Assume your merge function is available.
    pass


def merge_sort(a):
    pass
```

**Question 6 [concept] Why merge is linear**

Explain why merging two sorted lists of total length $n$ costs $\Theta(n)$.
Your answer should mention:

1. the two indexes,
2. why each comparison advances at least one index,
3. why each element is copied once,
4. why the remaining suffix can be appended directly.

## 19.3 The Master Theorem and merge sort complexity

**Learning outcome 19.5: Apply the Master Theorem to simple divide-and-conquer recurrences**

Students should be able to identify $a$, $b$, and $f(n)$, compare $f(n)$ with $n^{\log_b a}$, and choose the correct Master Theorem case.

**Question 7 [analysis] Master Theorem practice**

For each recurrence below, identify $a$, $b$, $f(n)$, compute $n^{\log_b a}$, and give the asymptotic solution using the Master Theorem.

1. $T(n) = 2T(n/2) + n$
2. $T(n) = 2T(n/2) + 1$
3. $T(n) = 2T(n/2) + n^2$
4. $T(n) = 3T(n/2) + n$

**Learning outcome 19.6: Analyze merge sort complexity**

Students should be able to derive the recurrence for merge sort and explain why its best, average, and worst-case time are all $\Theta(n \log n)$.

**Question 8 [analysis] Merge sort recurrence**

Derive the recurrence for merge sort and solve it.
Your answer should include:

1. why there are two recursive calls,
2. why each subproblem has size $n/2$,
3. why the merge work is $\Theta(n)$,
4. the recurrence $T(n) = 2T(n/2) + cn$,
5. the Master Theorem case,
6. the final time complexity,
7. the extra memory complexity.

## 19.4 Quicksort and partitioning

**Learning outcome 19.7: Trace quicksort partitioning**

Students should be able to explain the role of the pivot, trace partitioning, and identify the pivot's final position.

**Question 9 [trace] Quicksort with extra lists**

Use the intuition version of quicksort, where we choose a pivot and build three lists: smaller, equal, and larger.
For the input below, choose pivot `4`.

```
[7, 2, 9, 4, 3, 8]
```

Answer the following questions.

1. What goes into the `smaller` list?
2. What goes into the `equal` list?
3. What goes into the `larger` list?
4. Why is the pivot in its final sorted position after partitioning?
5. What recursive calls remain after this partition?

**Question 10 [trace] In-place partition trace**

Trace the following in-place partition procedure on this list:

```
a = [7, 2, 9, 4, 3, 8]
```

```
low = 0
high = 5
```

Use this partition rule:

- the pivot is `a[low]`,
- `i` marks where the next value less than or equal to the pivot should go,
- `j` scans from `low + 1` to `high`,
- at the end, swap the pivot with `a[i - 1]`.

Answer the following questions.

1. What is the pivot?
2. Show the array after each successful swap during the scan.
3. What is the final pivot index?
4. What is the array after the final pivot swap?

**Learning outcome 19.8: Implement in-place partition and quicksort**

Students should be able to implement the lecture's in-place partition and use it inside recursive quicksort.

**Question 11 [implementation] Implement partition**

Implement `partition(a, low, high)` using the lecture version.
Rules:

- The pivot is the first element: `a[low]`.
- Values less than or equal to the pivot should be moved before the final pivot position.
- Values greater than the pivot should remain after the final pivot position.
- The function should mutate `a` in place.
- The function should return the final pivot index.

```
def partition(a, low, high):
    pass
```

**Question 12 [implementation] Implement in-place quicksort**

Implement `quicksort(a, low=0, high=None)` using `partition`.
Rules:

- Sort the list in place.
- If `high` is `None`, set it to the last valid index.
- Stop when the segment has zero or one element.
- Recursively sort the left and right sides of the pivot.
- Do not return a new sorted list.

```python
def partition(a, low, high):
    # Assume your partition function is available.
    pass


def quicksort(a, low=0, high=None):
    pass
```

## 19.5 Quicksort complexity

**Learning outcome 19.9: Analyze quicksort best, average, and worst cases**

Students should be able to explain how pivot quality controls the quicksort recursion tree and why the best and average cases are $\Theta(n \log n)$ while the worst case is $\Theta(n^2)$.

**Question 13 [analysis] Quicksort recurrence cases**

Answer the following questions about quicksort complexity.

1. What recurrence describes the best case when every pivot splits the array into two equal halves?
2. What is the best-case complexity?
3. What recurrence describes the worst case when the pivot is always the smallest or largest element?
4. Why does the Master Theorem not apply directly to the worst-case recurrence?
5. Expand the worst-case recurrence enough to justify $\Theta(n^2)$.
6. What is the average-case complexity when pivot ranks are reasonably balanced on average?

**Question 14 [concept] Avoiding quicksort's worst case**

The lecture suggests shuffling the array before running quicksort with a fixed pivot rule such as choosing the first element.
Explain why shuffling helps.
Your answer should mention:

1. what goes wrong on already sorted or adversarial input,
2. why a fixed pivot rule becomes risky,
3. how shuffling makes the pivot behave more like a random pivot,
4. why shuffling does not change the theoretical worst case but makes it unlikely in practice.

## 19.6 Comparison and algorithm choice

**Learning outcome 19.10: Compare merge sort and quicksort**

Students should be able to compare merge sort and in-place quicksort by time complexity, memory use, stability, and practical risk.

**Question 15 [concept] Merge sort versus quicksort**

Complete the comparison between merge sort and in-place quicksort.

| Feature | Merge sort | In-place quicksort |
|---|---|---|
| Best-case time | | |
| Average-case time | | |
| Worst-case time | | |
| Main operation | | |
| Extra memory | | |
| Stability | | |
| Main risk | | |

**Question 16 [concept] Choosing a sorting algorithm**

For each situation, choose merge sort or in-place quicksort and justify your choice.

1. You need guaranteed $\Theta(n \log n)$ worst-case time and can afford extra memory.
2. You want an in-place algorithm that is usually fast in practice.
3. The input may be adversarially ordered and the pivot rule is fixed as the first element.
4. Equal elements must preserve their original relative order.
5. Memory is tight and occasional bad pivots are acceptable.

Chapter 20

# Solutions: Sorting Algorithms

This chapter gives solutions to the review questions from the previous chapter. The solutions emphasize divide-and-conquer structure, merge sort tracing, implementation details, Master Theorem analysis, quicksort partitioning, quicksort complexity, and the tradeoff between reliability and in-place performance.

## 20.1 Solutions for sorting baselines and divide and conquer

**Solution 1 [concept]**

1. $O(n^2)$ means the work grows roughly like the square of the input size. If the input becomes ten times larger, the work can become about one hundred times larger.
2. $O(n \log n)$ means the work grows a little more than linearly, but much less than quadratically.
3. For large input, the gap is enormous. For example, when $n = 1{,}000{,}000$, $n^2 = 10^{12}$, while $n \log_2 n$ is about 20,000,000. That is not a small constant-factor difference.
4. Insertion sort can still be useful on very small inputs or nearly sorted inputs because it is simple and has low overhead.

**Solution 2 [concept]**

1. Merge sort divides the input by position: it splits the list into a left half and a right half.
2. Quicksort divides the input by pivot value: values less than or equal to the pivot go to one side, and values greater than the pivot go to the other side.
3. In merge sort, the main linear work happens after the recursive calls, when the two sorted halves are merged.
4. In quicksort, the main linear work happens before the recursive calls, when the segment is partitioned around the pivot.
5. Quicksort does not need a final merge because partitioning already puts the pivot in its final sorted position and separates smaller values from larger values.

## 20.2 Solutions for merge sort

**Solution 3 [trace]**

The splitting phase is:

```
[7, 2, 9, 4, 3, 8]
[7, 2, 9]              [4, 3, 8]
[7] [2, 9]             [4] [3, 8]
[7] [2] [9]            [4] [3] [8]
```

The merging phase is:

```
[2] and [9]       -> [2, 9]
[7] and [2, 9]    -> [2, 7, 9]
[3] and [8]       -> [3, 8]
[4] and [3, 8]    -> [3, 4, 8]
[2, 7, 9] and [3, 4, 8] -> [2, 3, 4, 7, 8, 9]
```

The base case is a list of length zero or one, because such a list is already sorted.
The final sorted list is:

```
[2, 3, 4, 7, 8, 9]
```

**Solution 4 [implementation]**

```python
def merge(left, right):
    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Using `<=` means that when equal values are compared, the value from the left list is copied first. This is the condition that makes merge sort stable when the recursive sort is also stable.

**Solution 5 [implementation]**

```python
def merge(left, right):
    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result


def merge_sort(a):
    if len(a) <= 1:
        return a

    middle = len(a) // 2
    left = merge_sort(a[:middle])
    right = merge_sort(a[middle:])

    return merge(left, right)
```

This version is written for clarity. The slices create new lists, so it is not the most memory-minimal implementation.

**Solution 6 [concept]**

Merging two sorted lists uses two indexes, one into `left` and one into `right`. At each step, the algorithm compares the first unused values and copies the smaller one into the result.

Each comparison advances at least one index, so the algorithm never repeatedly compares the same pair forever. Each element is copied into the result exactly once. When one list is exhausted, the remaining suffix of the other list is already sorted, so it can be appended directly.

Therefore, merging lists with total length $n$ costs $\Theta(n)$.

## 20.3 Solutions for the Master Theorem and merge sort complexity

**Solution 7 [analysis]**

1. $T(n) = 2T(n/2) + n$. Here $a = 2$, $b = 2$, and $f(n) = n$. Also $n^{\log_b a} = n^{\log_2 2} = n$. Since $f(n) = \Theta(n)$, this is Case 2. Therefore:

$$T(n) = \Theta(n \log n).$$

2. $T(n) = 2T(n/2) + 1$. Here $a = 2$, $b = 2$, and $f(n) = 1$. Also $n^{\log_b a} = n$. Since 1 is smaller than $n$, this is Case 1. Therefore:

$$T(n) = \Theta(n).$$

3. $T(n) = 2T(n/2) + n^2$. Here $a = 2$, $b = 2$, and $f(n) = n^2$. Also $n^{\log_b a} = n$. Since $n^2$ is larger than $n$, this is Case 3. Therefore:

$$T(n) = \Theta(n^2).$$

4. $T(n) = 3T(n/2) + n$. Here $a = 3$, $b = 2$, and $f(n) = n$. Also:

$$n^{\log_b a} = n^{\log_2 3}.$$

   Since $n$ is smaller than $n^{\log_2 3}$, this is Case 1. Therefore:

$$T(n) = \Theta(n^{\log_2 3}).$$

**Solution 8 [analysis]**

In one call to merge sort on input size $n$:

- the left half is sorted recursively, giving $T(n/2)$,
- the right half is sorted recursively, giving another $T(n/2)$,
- the two sorted halves are merged in linear time, giving $cn$.

Therefore:

$$T(n) = 2T(n/2) + cn.$$

For the Master Theorem:

$$a = 2, \qquad b = 2, \qquad f(n) = cn = \Theta(n).$$

Also:

$$n^{\log_b a} = n^{\log_2 2} = n.$$

So $f(n) = \Theta(n^{\log_b a})$, which is Case 2. Therefore:

$$T(n) = \Theta(n \log n).$$

The recurrence is the same in the best, average, and worst case because merge sort always splits by position, not by the values in the list.
The extra memory is $\Theta(n)$, mainly because merging creates auxiliary lists/results.

## 20.4 Solutions for quicksort and partitioning

**Solution 9 [trace]**

Input:

```
[7, 2, 9, 4, 3, 8]
```

Pivot:

```
4
```

The three lists are:

```
smaller = [2, 3]
equal   = [4]
larger  = [7, 9, 8]
```

After partitioning, every value in `smaller` is less than `4`, and every value in `larger` is greater than `4`. Therefore, the pivot `4` is in its final sorted position relative to the whole list.
The remaining recursive calls are on:

```
[2, 3]
[7, 9, 8]
```

**Solution 10 [trace]**

The input is:

```
[7, 2, 9, 4, 3, 8]
```

The pivot is `7`, because the pivot is `a[low]`.
Initial state:

```
pivot = 7
 i = 1
 j scans from 1 to 5
```

Successful swaps during the scan:

| j | Reason | Array after successful swap |
|---|---|---|
| 1 | `2 <= 7` | `[7, 2, 9, 4, 3, 8]` |
| 3 | `4 <= 7` | `[7, 2, 4, 9, 3, 8]` |
| 4 | `3 <= 7` | `[7, 2, 4, 3, 9, 8]` |

At the end of the scan, `i = 4`, so the final pivot index is:

```
i - 1 = 3
```

After swapping the pivot with `a[3]`, the array becomes:

```
[3, 2, 4, 7, 9, 8]
```

The pivot `7` is now at index `3`. All values before it are less than or equal to `7`, and all values after it are greater than `7`.

### Solution 11 [implementation]

```
def partition(a, low, high):
    pivot = a[low]
    i = low + 1

    for j in range(low + 1, high + 1):
        if a[j] <= pivot:
            a[i], a[j] = a[j], a[i]
            i += 1

    pivot_index = i - 1
    a[low], a[pivot_index] = a[pivot_index], a[low]
    return pivot_index
```

The loop scans the current segment once, so partitioning a segment of size $n$ costs $\Theta(n)$.

### Solution 12 [implementation]

```
def partition(a, low, high):
    pivot = a[low]
    i = low + 1

    for j in range(low + 1, high + 1):
        if a[j] <= pivot:
            a[i], a[j] = a[j], a[i]
            i += 1

    pivot_index = i - 1
    a[low], a[pivot_index] = a[pivot_index], a[low]
    return pivot_index


def quicksort(a, low=0, high=None):
    if high is None:
        high = len(a) - 1

    if low >= high:
        return

    pivot_index = partition(a, low, high)
```

```
    quicksort(a, low, pivot_index - 1)
    quicksort(a, pivot_index + 1, high)
```

The function sorts in place. It does not need to return a new list because the array is modified directly.

## 20.5 Solutions for quicksort complexity

**Solution 13 [analysis]**

1. In the best case, every pivot splits the array into two equal halves, so:

$$T(n) = 2T(n/2) + cn.$$

2. This matches Master Theorem Case 2, so best-case quicksort is:

$$\Theta(n \log n).$$

3. In the worst case, the pivot is always the smallest or largest value. One side has size $n - 1$, and the other side has size 0, so:

$$T(n) = T(n-1) + cn.$$

4. The Master Theorem does not apply directly because the recursive subproblem is size $n - 1$, not $n/b$ for a constant $b$.
5. Expanding the recurrence:

$$\begin{aligned} T(n) &= T(n-1) + cn \\ &= T(n-2) + c(n-1) + cn \\ &= T(n-3) + c(n-2) + c(n-1) + cn \\ &\cdots \\ &= c(1 + 2 + 3 + \cdots + n). \end{aligned}$$

   Since:

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2},$$

   the worst-case time is:

$$\Theta(n^2).$$

6. The average-case complexity of quicksort is $\Theta(n \log n)$, assuming pivot ranks are reasonably balanced on average.

**Solution 14 [concept]**

Quicksort becomes slow when the pivot repeatedly creates extremely unbalanced partitions. For example, if the pivot is always the smallest or largest value, the recurrence becomes:

$$T(n) = T(n-1) + cn,$$

which gives $\Theta(n^2)$.
A fixed pivot rule, such as always choosing the first element, is risky on already sorted input because the first element may repeatedly be the minimum. Shuffling the list before sorting makes the input order random. After shuffling, choosing the first element behaves more like choosing a random pivot.
This does not remove the theoretical worst case. A terrible sequence of pivots is still possible. But it makes that sequence unlikely in practice and pushes behavior closer to the average case.

## 20.6 Solutions for comparison and algorithm choice

**Solution 15 [concept]**

| Feature | Merge sort | In-place quicksort |
|---|---|---|
| Best-case time | $\Theta(n \log n)$ | $\Theta(n \log n)$ |
| Average-case time | $\Theta(n \log n)$ | $\Theta(n \log n)$ |
| Worst-case time | $\Theta(n \log n)$ | $\Theta(n^2)$ |
| Main operation | Merge | Partition |
| Extra memory | $\Theta(n)$ | $\Theta(\log n)$ average recursion stack |
| Stability | Yes, if implemented carefully | Usually no |
| Main risk | Extra memory cost | Bad pivots |

**Solution 16 [concept]**

1. Choose merge sort. It gives guaranteed $\Theta(n \log n)$ worst-case time, and the condition says extra memory is acceptable.
2. Choose in-place quicksort. It is usually fast in practice and uses little extra memory on balanced recursion.
3. Choose merge sort, or at least do not use fixed-first-pivot quicksort without shuffling. A fixed first pivot on adversarial or already sorted input can trigger worst-case $\Theta(n^2)$.
4. Choose merge sort. Merge sort can be stable if equal elements are taken from the left half first during merging.
5. Choose in-place quicksort. It has lower extra memory use, but the tradeoff is sensitivity to pivot quality.

Chapter 21

# Searching and Hashing: Review Questions

This chapter reviews Lecture 11 of COMP102. The focus is on exact lookup by key: linear search, binary search, dynamic ordered search, dictionaries, hash functions, hash tables, collisions, chaining, open addressing, tombstones, and the tradeoffs behind average-case constant-time lookup.

Solutions are intentionally placed in the following chapter.

## 21.1 Searching as an engineering problem

**Learning outcome 21.1: Compare search strategies**

Students should be able to compare linear search, binary search, balanced BST search, and hash-table lookup by their assumptions, strengths, and costs.

**Question 1 [concept] From scanning to direct access**

Lecture 11 moves from scanning all items to computing where a key should be stored. Answer the following questions.

1. What assumption does linear search make about the data?
2. What assumption does binary search require?
3. Why can a balanced BST support dynamic ordered search better than a sorted array?
4. What kind of lookup is hashing designed for?
5. Why is hashing not a replacement for ordered structures when we need minimums, maximums, or range queries?

**Learning outcome 21.2: Analyze linear search**

Students should be able to trace linear search and explain its best, average, and worst-case behavior.

### Question 2 [analysis] Linear search cost

Consider this function:

```
def contains(items, target):
    for item in items:
        if item == target:
            return True
    return False
```

Answer the following questions.

1. What is the best case? Give an example.
2. What is the worst case? Give an example.
3. If the target is equally likely to appear at any position, why is the average number of checks $\Theta(n)$?
4. What happens to the cost when the target is absent?

### Learning outcome 21.3: Trace binary search

Students should be able to trace binary search using `left`, `right`, and `mid`, and explain why the input must be sorted by the same key used for searching.

### Question 3 [trace] Binary search trace

Trace binary search for target `19` in the following sorted list:

```
items = [3, 6, 8, 11, 15, 19, 24, 31, 42]
```

For each iteration, give:

1. `left`, `right`, and `mid`,
2. the value `items[mid]`,
3. whether the algorithm searches left or right next,
4. when the target is found.

### Question 4 [implementation] Implement binary search

Implement `binary_search(items, target)`.
Rules:

- Return `True` if `target` appears in `items`.
- Return `False` otherwise.
- Assume `items` is sorted in increasing order.
- Do not use `in`, `index`, `find`, `bisect`, `sort`, or `sorted`.

```
def binary_search(items, target):
    pass
```

## 21.2 Dictionaries, hash functions, and hashability

**Learning outcome 21.4: Explain the dictionary ADT**

Students should be able to distinguish the dictionary/map ADT from a hash-table implementation.

**Question 5 [concept] Dictionary ADT versus hash table**

Answer the following questions.

1. What does a dictionary/map store?
2. What do `put(key, value)`, `get(key)`, `contains(key)`, and `delete(key)` mean?
3. Why is a list of key-value pairs a valid but inefficient dictionary implementation?
4. What does a hash table add to improve exact lookup?
5. Why must lookup still verify equality after using the hash value?

**Learning outcome 21.5: Reason about hash functions**

Students should be able to explain the role of a hash function, identify weak hash functions, and understand that collisions are unavoidable.

**Question 6 [concept] Hash functions and collisions**

Consider the following three hash functions:

```
def bad_hash(key, m):
    return 0


def weak_hash(text, m):
    total = 0
    for char in text:
        total += ord(char)
    return total % m


def poly_hash(text, m):
    h = 0
    base = 257
    for char in text:
        h = (h * base + ord(char)) % m
    return h
```

Answer the following questions.

1. Why is `bad_hash` a poor hash function?
2. Why do `"AB"` and `"BA"` collide under `weak_hash` for every table size `m`?
3. What improvement does `poly_hash` make compared with `weak_hash`?

4. Does a better hash function eliminate collisions completely? Explain.

**Question 7 [analysis] Modulo and load factor**

A hash table has $M = 10$ slots and stores $N = 7$ keys.
Answer the following questions.

1. What does $h(k)$ mod $M$ compute?
2. What is the load factor $\alpha$?
3. Why does increasing $M$ usually reduce collisions?
4. What is the memory tradeoff of increasing $M$?
5. Why does open addressing require $\alpha < 1$?

**Learning outcome 21.6: Explain Python hashability**

Students should be able to explain the equality-hash contract, custom `__eq__` and `__hash__`, and why mutable keys are dangerous.

**Question 8 [concept] Equality and hashing in Python**

Answer the following questions.

1. State the equality-hash contract.
2. Why does equal objects having equal hashes matter for dictionaries?
3. Why does `hash(a) == hash(b)` not imply `a == b`?
4. What happens in Python if a class defines `__eq__` but not `__hash__`?
5. Why is it dangerous to mutate the field used by `__hash__` while the object is inside a dictionary?

**Question 9 [implementation] Custom hashable student object**

Implement a class `Student` so that two students are considered equal when they have the same `student_id`, regardless of their name.
Rules:

- Store `student_id` and `name`.
- Implement `__eq__`.
- Implement `__hash__`.
- Use the same information for equality and hashing.

```
class Student:
    pass
```

## 21.3 Hash tables with chaining

**Learning outcome 21.7: Trace collision resolution by chaining**

Students should be able to model insertions and retrievals in a chaining hash table using a given dummy hash function.

**Question 10 [trace] Chaining trace with a dummy hash function**

We use a hash table with $M = 5$ buckets. Each bucket is a list of key-value pairs. Use this dummy hash function:

```
def dummy_hash(key):
    values = {
        "A": 1,
        "B": 1,
        "C": 3,
        "D": 1,
        "E": 4,
    }
    return values[key]
```

The table index is:

$$\text{index} = \texttt{dummy\_hash(key)} \bmod 5.$$

Start with an empty chaining table:

```
0: []
1: []
2: []
3: []
4: []
```

Perform the following operations in order:

```
put("A", 10)
put("B", 20)
put("C", 30)
get("B")
put("A", 99)
contains("D")
get("A")
```

Answer the following questions.

1. Show the table after the first three `put` operations.
2. Trace `get("B")` bucket by bucket and pair by pair.
3. Show the table after `put("A", 99)`.
4. What does `contains("D")` return? Explain why.
5. What does `get("A")` return? Explain why.

**Learning outcome 21.8: Implement chaining operations**

Students should be able to implement `put`, `get`, `contains`, and `delete` for a chaining hash table.

**Question 11 [implementation] Implement chaining dictionary operations**

Implement the following functions for a chaining hash table.
Representation:

- `table` is a list of buckets.
- Each bucket is a list of `(key, value)` pairs.
- Use `hash(key) % len(table)` to choose the bucket.

Rules:

- `put` updates the value if the key already exists; otherwise it appends a new pair.
- `get` returns the value or raises `KeyError(key)`.
- `contains` returns a Boolean.
- `delete` removes the pair or raises `KeyError(key)`.

```
def put(table, key, value):
    pass


def get(table, key):
    pass


def contains(table, key):
    pass


def delete(table, key):
    pass
```

**Question 12 [analysis] Chaining complexity**

Let $N$ be the number of keys, $M$ the number of buckets, and $\alpha = N/M$ the load factor. Answer the following questions.

1. Why does a chaining lookup first compute one bucket and then scan only that bucket?
2. Why is the average cost $\Theta(1 + \alpha)$ with good hashing?
3. What is the worst case for chaining?
4. How can resizing help keep chaining efficient?

## 21.4 Hash tables with open addressing

**Learning outcome 21.9: Trace open addressing with linear probing**

Students should be able to model insertion, retrieval, deletion, tombstones, and tombstone reuse using a given dummy hash function.

**Question 13 [trace] Open addressing trace with a dummy hash function**

We use an open-addressing hash table with $M = 7$ slots and linear probing.
Slot states:

- `EMPTY`: no key has ever occupied the slot,
- `TOMBSTONE`: a key was deleted here,
- `(key, value)`: occupied slot.

Use this dummy hash function:

```
def dummy_hash(key):
    values = {
        "A": 2,
        "B": 2,
        "C": 2,
        "D": 5,
        "E": 2,
    }
    return values[key]
```

The first probe index is:

$$\text{start} = \texttt{dummy\_hash(key)} \bmod 7.$$

Linear probing tries:

$$\text{start},\ \text{start} + 1,\ \text{start} + 2, \ldots$$

wrapping around the table when needed.
Start with an empty table:

```
0: EMPTY
1: EMPTY
2: EMPTY
3: EMPTY
4: EMPTY
5: EMPTY
6: EMPTY
```

Perform the following operations in order:

```
put("A", 10)
put("B", 20)
put("C", 30)
get("C")
```

```
delete("B")
get("C")
put("E", 50)
get("B")
```

Answer the following questions.

1. Show the table after inserting `"A"`, `"B"`, and `"C"`.
2. Trace `get("C")` before deletion.
3. Show the table after `delete("B")`.
4. Trace `get("C")` after deletion and explain why the tombstone must not stop the search.
5. Show where `put("E", 50)` inserts the new key.
6. Trace `get("B")` at the end and explain why it fails.

**Learning outcome 21.10: Explain tombstones**

Students should be able to explain why deletion in open addressing cannot simply replace a deleted slot with `EMPTY`.

**Question 14 [concept] Why tombstones are necessary**

Consider this open-addressing table with linear probing:

```
0: EMPTY
1: EMPTY
2: ("A", 10)
3: ("B", 20)
4: ("C", 30)
5: EMPTY
6: EMPTY
```

Assume `"A"`, `"B"`, and `"C"` all have initial index `2`.
Answer the following questions.

1. Why is `"C"` stored at index `4`?
2. If `"B"` is deleted by setting slot `3` to `EMPTY`, what goes wrong when searching for `"C"`?
3. How does a `TOMBSTONE` fix the problem?
4. Why are too many tombstones still bad for performance?

**Learning outcome 21.11: Implement open addressing operations**

Students should be able to implement basic open-addressing operations with linear probing and tombstones.

### Question 15 [implementation] Implement open addressing get and contains

Implement `get(table, key)` and `contains(table, key)` for an open-addressing hash table with linear probing.
Representation:

- `EMPTY = None`
- `TOMBSTONE = object()`
- occupied slots store `(key, value)`

Rules:

- Start at `hash(key) % len(table)`.
- Continue through tombstones.
- Stop at `EMPTY` or after a full loop.
- `get` returns the value or raises `KeyError(key)`.
- `contains` returns `True` or `False`.

```
EMPTY = None
TOMBSTONE = object()


def get(table, key):
    pass


def contains(table, key):
    pass
```

### Question 16 [implementation] Implement open addressing put and delete

Implement `put(table, key, value)` and `delete(table, key)` for open addressing with linear probing.
Rules for `put`:

- Update the key if it already exists.
- Remember the first tombstone found.
- Continue probing after a tombstone, because the same key may appear later in the probe sequence.
- Insert into the first tombstone if available; otherwise insert into the first empty slot.
- Raise `RuntimeError("table is full")` if there is no place to insert.

Rules for `delete`:

- Replace the deleted slot with `TOMBSTONE`, not `EMPTY`.
- Raise `KeyError(key)` if the key is absent.

```
EMPTY = None
TOMBSTONE = object()

```

```
4
5  def put(table, key, value):
6      pass
7
8
9  def delete(table, key):
10     pass
```

**Question 17 [analysis] Open addressing complexity and limitations**

Answer the following questions.

1. Why must the load factor stay below 1 in open addressing?
2. Why does low load factor help keep probes short?
3. What is clustering in linear probing?
4. Why do tombstones make future searches slower?
5. Why do practical hash tables resize or rebuild?

## 21.5 Choosing the right structure

**Learning outcome 21.12: Choose between search data structures**

Students should be able to select an appropriate structure based on whether the problem needs order, exact identity lookup, cheap insertion, or predictable performance.

**Question 18 [synthesis] Pick the right structure**

For each situation, choose one structure from this list:

unsorted list sorted array balanced BST hash table

Then justify your choice briefly.

1. You store a small list of configuration flags and rarely search it.
2. You have a static sorted catalog and perform many membership queries.
3. You maintain dynamic ordered data and often need the next larger value.
4. You store session tokens and need exact lookup by token.
5. You need to print all keys in sorted order frequently.
6. You need average-case fast `put`, `get`, and `contains` by exact key.

Chapter 22

# Solutions: Searching and Hashing

This chapter gives solutions to the review questions from the previous chapter. The solutions emphasize search assumptions, exact lookup by key, hash functions, collision handling, chaining traces, open-addressing traces, tombstones, and complexity tradeoffs.

## 22.1 Solutions for searching as an engineering problem

**Solution 1 [concept]**

1. Linear search makes essentially no ordering assumption. It can work on any list because it checks elements one by one.
2. Binary search requires the data to be sorted by the same key and comparison used during the search.
3. A balanced BST avoids shifting array elements during insertion and deletion. It keeps order through the tree structure and gives $O(\log n)$ search, insert, and delete when balanced.
4. Hashing is designed for exact lookup by key: for example, student ID to student record, sensor ID to latest reading, or session token to active session.
5. Hashing does not preserve useful sorted order. A hash table is poor for minimum, maximum, next-larger, range queries, or sorted traversal. Ordered structures are the right tool for those questions.

**Solution 2 [analysis]**

1. The best case is when the first element is the target. Example: searching for `7` in `[7, 2, 9, 4]` takes one check, so the cost is $O(1)$.
2. The worst case is when the target is at the last position or absent. Example: searching for `8` in `[7, 2, 9, 4, 3, 8]` checks every element, so the cost is $O(n)$.
3. If the target is equally likely to be at any position, the expected number of checks is:

$$\frac{1+2+3+\cdots+n}{n} = \frac{1}{n}\cdot\frac{n(n+1)}{2} = \frac{n+1}{2}.$$

   This grows linearly with $n$, so the average case is $\Theta(n)$.
4. If the target is absent, the loop must scan the entire list before returning `False`. That is $\Theta(n)$.

**Solution 3 [trace]**

The list is:

```
index:  0  1  2  3  4  5  6  7  8
value:  3  6  8  11 15 19 24 31 42
```

Trace:

| Iteration | left | right | mid | action |
|---|---|---|---|---|
| 1 | 0 | 8 | 4 | `items[4] = 15`. Since $15 < 19$, search right. |
| 2 | 5 | 8 | 6 | `items[6] = 24`. Since $24 > 19$, search left. |
| 3 | 5 | 5 | 5 | `items[5] = 19`. Target found. |

The function returns `True` on the third iteration.

**Solution 4 [implementation]**

```python
def binary_search(items, target):
    left = 0
    right = len(items) - 1

    while left <= right:
        mid = (left + right) // 2

        if items[mid] == target:
            return True
        elif items[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return False
```

The key assumption is that `items` is sorted. Without sorted order, comparing with the middle value does not tell us which half can be safely discarded.

## 22.2 Solutions for dictionaries, hash functions, and hashability

**Solution 5 [concept]**

1. A dictionary/map stores key-value pairs.
2. `put(key, value)` inserts or updates a key-value pair. `get(key)` retrieves the value for a key. `contains(key)` checks whether the key exists. `delete(key)` removes the key-value pair.
3. A list of key-value pairs is valid because it can store all pairs and scan for a matching key. It is inefficient because `get`, `contains`, `put` updates, and `delete` may require scanning $n$ pairs.
4. A hash table adds an array and a hash function. The hash function maps the key to

an array index, so lookup can focus on one location or one collision path.

5. Equality must still be checked because different keys can have the same hash index. The hash narrows the search; equality confirms the exact key.

**Solution 6 [concept]**

1. `bad_hash` sends every key to index `0`. This creates maximum collision and reduces the table to a linear scan inside one bucket or one long probe sequence.
2. `weak_hash` adds character codes. Since addition ignores order, `"AB"` and `"BA"` have the same total: `ord('A')` + `ord('B')`. Therefore they collide for every `m`.
3. `poly_hash` updates the hash as `h = h * base + ord(char)`. This makes character position matter, so `"AB"` and `"BA"` usually produce different values.
4. No. Collisions are unavoidable in principle because many possible keys must map into a finite number of table slots. A good hash function only makes collisions less clustered and less predictable.

**Solution 7 [analysis]**

1. $h(k) \bmod M$ converts a hash value into a valid table index from 0 to $M - 1$.
2. The load factor is:
$$\alpha = \frac{N}{M} = \frac{7}{10} = 0.7.$$
3. Increasing $M$ gives keys more possible slots or buckets, so fewer keys are forced to share the same location.
4. Increasing $M$ uses more memory. Hash tables trade memory for speed.
5. Open addressing stores all keys directly inside the array. If $\alpha \geq 1$, the table is full or overfull, so insertion may have nowhere to place a new key.

**Solution 8 [concept]**

1. The equality-hash contract is:
$$a == b \implies \texttt{hash(a)} == \texttt{hash(b)}.$$
2. Dictionaries use the hash to locate a key. If equal objects had different hashes, the dictionary could store them in different places and fail to find an equal key.
3. Hash collisions are allowed. Two different objects can have the same hash, so `hash(a) == hash(b)` does not prove equality.
4. If a class defines `__eq__` but not `__hash__`, Python makes its instances unhashable. They cannot be used as dictionary keys or set elements.
5. If the field used by `__hash__` changes while the object is already inside a dictionary, the object may no longer be found in the place where the dictionary stored it. The key effectively becomes corrupted.

**Solution 9 [implementation]**

```
class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name

    def __eq__(self, other):
        return (
            isinstance(other, Student)
            and self.student_id == other.student_id
        )

    def __hash__(self):
        return hash(self.student_id)
```

Both equality and hashing depend on `student_id`. That satisfies the equality-hash contract: if two students have the same ID, they are equal and have the same hash.

## 22.3 Solutions for hash tables with chaining

**Solution 10 [trace]**

The table has $M = 5$ buckets. The relevant indexes are:

```
A -> dummy_hash(A) % 5 = 1
B -> dummy_hash(B) % 5 = 1
C -> dummy_hash(C) % 5 = 3
D -> dummy_hash(D) % 5 = 1
E -> dummy_hash(E) % 5 = 4
```

After the first three insertions:

```
put("A", 10) goes to bucket 1
put("B", 20) goes to bucket 1
put("C", 30) goes to bucket 3

0: []
1: [("A", 10), ("B", 20)]
2: []
3: [("C", 30)]
4: []
```

Trace `get("B")`:

```
index = dummy_hash("B") % 5 = 1
bucket 1 = [("A", 10), ("B", 20)]
compare "A" with "B" -> not equal
compare "B" with "B" -> equal
return 20
```

Now `put("A", 99)` goes to bucket 1. The key `"A"` already exists, so its value is updated instead of appending a duplicate:

```
0: []
1: [("A", 99), ("B", 20)]
2: []
3: [("C", 30)]
4: []
```

Trace `contains("D")`:

```
index = dummy_hash("D") % 5 = 1
bucket 1 = [("A", 99), ("B", 20)]
compare "A" with "D" -> not equal
compare "B" with "D" -> not equal
no match found
return False
```

Trace `get("A")`:

```
index = dummy_hash("A") % 5 = 1
bucket 1 = [("A", 99), ("B", 20)]
compare "A" with "A" -> equal
return 99
```

**Solution 11 [implementation]**

```python
def put(table, key, value):
    index = hash(key) % len(table)
    bucket = table[index]

    for i in range(len(bucket)):
        existing_key, existing_value = bucket[i]
        if existing_key == key:
            bucket[i] = (key, value)
            return

    bucket.append((key, value))


def get(table, key):
    index = hash(key) % len(table)
    bucket = table[index]

    for existing_key, value in bucket:
        if existing_key == key:
            return value

    raise KeyError(key)


def contains(table, key):
    index = hash(key) % len(table)
    bucket = table[index]
```

```
    for existing_key, value in bucket:
        if existing_key == key:
            return True

    return False


def delete(table, key):
    index = hash(key) % len(table)
    bucket = table[index]

    for i in range(len(bucket)):
        existing_key, value = bucket[i]
        if existing_key == key:
            bucket.pop(i)
            return

    raise KeyError(key)
```

**Solution 12 [analysis]**

1. A chaining lookup computes the bucket index with $h(k)$ mod $M$. Since every key with that hash index must be in that bucket, the algorithm scans only that bucket and not the whole table.
2. With good hashing, keys are spread roughly evenly. The average bucket length is close to $\alpha = N/M$, so the expected lookup cost is the cost of computing the index plus scanning a short bucket: $\Theta(1 + \alpha)$.
3. The worst case is $\Theta(n)$, when many or all keys land in the same bucket.
4. Resizing increases $M$, reducing $\alpha$. Rehashing into the larger table spreads keys over more buckets, making buckets shorter on average.

## 22.4 Solutions for hash tables with open addressing

**Solution 13 [trace]**

The table has $M = 7$. All of `"A"`, `"B"`, `"C"`, and `"E"` start at index 2.
Insert `"A"`:

```
start = 2
slot 2 is EMPTY -> insert A at 2
```

Insert `"B"`:

```
start = 2
slot 2 has A -> probe 3
slot 3 is EMPTY -> insert B at 3
```

Insert `"C"`:

```
start = 2
slot 2 has A -> probe 3
slot 3 has B -> probe 4
slot 4 is EMPTY -> insert C at 4
```

After inserting `"A"`, `"B"`, and `"C"`:

```
0: EMPTY
1: EMPTY
2: ("A", 10)
3: ("B", 20)
4: ("C", 30)
5: EMPTY
6: EMPTY
```

Trace `get("C")` before deletion:

```
start = 2
slot 2 has A -> not C, continue
slot 3 has B -> not C, continue
slot 4 has C -> found, return 30
```

Now delete `"B"`. Search starts at 2, finds `"B"` at slot 3, and replaces that slot with `TOMBSTONE`:

```
0: EMPTY
1: EMPTY
2: ("A", 10)
3: TOMBSTONE
4: ("C", 30)
5: EMPTY
6: EMPTY
```

Trace `get("C")` after deletion:

```
start = 2
slot 2 has A -> not C, continue
slot 3 is TOMBSTONE -> continue
slot 4 has C -> found, return 30
```

The tombstone must not stop the search. If slot 3 were treated as `EMPTY`, the search would incorrectly conclude that `"C"` is absent.
Now insert `"E"`. It also starts at 2:

```
start = 2
slot 2 has A -> continue
slot 3 is TOMBSTONE -> remember first tombstone = 3, continue
slot 4 has C -> continue
slot 5 is EMPTY -> insert into first remembered tombstone, slot 3
```

After inserting `"E"`:

```
0: EMPTY
1: EMPTY
```

```
2: ("A", 10)
3: ("E", 50)
4: ("C", 30)
5: EMPTY
6: EMPTY
```

Trace `get("B")` at the end:

```
start = 2
slot 2 has A -> not B, continue
slot 3 has E -> not B, continue
slot 4 has C -> not B, continue
slot 5 is EMPTY -> stop
raise KeyError("B")
```

The search fails because the first genuinely empty slot ends the probe chain.

### Solution 14 [concept]

1. `"C"` is stored at index `4` because its first index is `2`, but slots `2` and `3` are already occupied by `"A"` and `"B"`. Linear probing continues to the next available slot.
2. If deleting `"B"` sets slot `3` to `EMPTY`, then a future search for `"C"` starts at `2`, checks `"A"`, reaches slot `3`, sees `EMPTY`, and stops too early. It incorrectly reports that `"C"` is absent.
3. A `TOMBSTONE` tells search that a key used to occupy this slot, so the probe sequence may continue beyond it. Search skips tombstones instead of stopping.
4. Too many tombstones make probe sequences longer. They preserve correctness, but they act like clutter that search must step through. Rebuilding or resizing removes accumulated tombstones.

### Solution 15 [implementation]

```python
EMPTY = None
TOMBSTONE = object()


def get(table, key):
    start = hash(key) % len(table)
    index = start

    while table[index] is not EMPTY:
        if table[index] is not TOMBSTONE:
            existing_key, value = table[index]
            if existing_key == key:
                return value

        index = (index + 1) % len(table)
        if index == start:
            break
```

```
    raise KeyError(key)


def contains(table, key):
    start = hash(key) % len(table)
    index = start

    while table[index] is not EMPTY:
        if table[index] is not TOMBSTONE:
            existing_key, value = table[index]
            if existing_key == key:
                return True

        index = (index + 1) % len(table)
        if index == start:
            break

    return False
```

Both functions skip tombstones and stop only at `EMPTY` or after returning to the starting index.

**Solution 16 [implementation]**

```
EMPTY = None
TOMBSTONE = object()


def put(table, key, value):
    start = hash(key) % len(table)
    index = start
    first_tombstone = None

    while table[index] is not EMPTY:
        if table[index] is TOMBSTONE:
            if first_tombstone is None:
                first_tombstone = index
        else:
            existing_key, old_value = table[index]
            if existing_key == key:
                table[index] = (key, value)
                return

        index = (index + 1) % len(table)
        if index == start:
            break

    if first_tombstone is not None:
        table[first_tombstone] = (key, value)
    elif table[index] is EMPTY:
```

```
        table[index] = (key, value)
    else:
        raise RuntimeError("table is full")


def delete(table, key):
    start = hash(key) % len(table)
    index = start

    while table[index] is not EMPTY:
        if table[index] is not TOMBSTONE:
            existing_key, value = table[index]
            if existing_key == key:
                table[index] = TOMBSTONE
                return

        index = (index + 1) % len(table)
        if index == start:
            break

    raise KeyError(key)
```

The subtle point in `put` is that the first tombstone is remembered but probing continues. This avoids inserting a duplicate key if the key already appears later in the probe sequence.

**Solution 17 [analysis]**

1. Open addressing stores all keys directly inside the table. If the load factor reaches 1, there may be no empty slot for a new key.
2. Low load factor means there are many empty slots, so probe sequences are likely to stop quickly.
3. Clustering is the formation of long runs of occupied slots. With linear probing, once a cluster forms, new colliding keys tend to attach to it and make it longer.
4. Tombstones must be inspected and skipped during search. Many tombstones make successful and unsuccessful searches take more probes.
5. Practical hash tables resize to reduce crowding and rebuild to remove tombstones. Both actions keep average probe lengths short.

## 22.5 Solutions for choosing the right structure

**Solution 18 [synthesis]**

1. **Unsorted list.** The data is small and rarely searched, so simplicity matters more than asymptotic search speed.
2. **Sorted array.** The catalog is static and sorted, so binary search gives $O(\log n)$ membership queries without insertion costs.

3. **Balanced BST.** The data changes dynamically and order matters. A balanced BST supports ordered operations such as next larger value.
4. **Hash table.** Session token lookup is exact identity lookup by key. Average-case $O(1)$ lookup is the target.
5. **Balanced BST.** Printing keys in sorted order frequently requires maintaining order. A hash table does not naturally store keys sorted.
6. **Hash table.** Average-case fast `put`, `get`, and `contains` by exact key is precisely the hash-table use case.

Chapter 23

# Introduction to Databases: Review Questions

This chapter reviews Lecture 12 of COMP102. The focus is on why databases are needed, the relational model, tables, keys, integrity constraints, ER modeling, ER-to-table mapping, and basic schema design.

Solutions are intentionally placed in the following chapter.

## 23.1 Why databases?

**Learning outcome 23.1: Explain why databases are needed**

Students should be able to explain why in-memory data structures and ad-hoc files are not enough for persistent, shared, large, and integrity-sensitive data.

**Question 1 [concept] In-memory structures versus databases**

For eleven lectures, the course focused on in-memory data structures such as lists, stacks, queues, trees, heaps, hash tables, and graphs.
Answer the following questions.

1. What assumption do these structures usually make about where the data lives?
2. What happens to the data when the process ends, unless the program explicitly saves it?
3. Name the four pressures that motivate using a database instead of only in-memory structures.
4. Why is a DBMS more than just a file on disk?

**Learning outcome 23.2: Distinguish database concepts**

Students should be able to distinguish a database, a DBMS, a data model, a schema, an instance, and a query language.

**Question 2 [concept] Database vocabulary**

Define each term in your own words.

1. Database
2. DBMS
3. Data model
4. Schema
5. Instance
6. Query language

Then classify each item below as one of those terms.

1. PostgreSQL
2. SQL
3. The table design `Student(id, name, major)`
4. The actual rows currently stored in `Student`
5. The relational model

**Question 3 [analysis] Why not a Python dictionary?**

Consider this toy storage design:

```
students = {
    1: {"name": "Amal", "major": "CS"},
    2: {"name": "Yassine", "major": "Math"},
}

import pickle
pickle.dump(students, open("students.pkl", "wb"))
```

Answer the following questions.

1. Why is this acceptable for a toy example?
2. What can go wrong if two programs write to the file at the same time?
3. Why is querying by content hard when the file is very large?
4. What can happen if the program crashes mid-write?
5. Which DBMS guarantees address these problems?

## 23.2 The relational model

**Learning outcome 23.3: Use relational terminology precisely**

Students should be able to use the terms relation, tuple, attribute, degree, cardinality, domain, schema, and instance.

### Question 4 [concept] Relational terminology

Consider this relation instance:

```
Student
id | name     | major
---+----------+------
1  | Amal     | CS
2  | Yassine | Math
3  | Salma    | CS
```

Answer the following questions.

1. What is the relation?
2. Give one tuple.
3. Give one attribute.
4. What is the degree?
5. What is the cardinality?
6. Give a possible domain for `id`.

### Learning outcome 23.4: Explain the consequences of relations as sets

Students should be able to explain why a relation has no duplicate rows, no meaningful row order, and atomic cells.

### Question 5 [analysis] Is this a valid relation?

Consider this table:

```
id | name     | phones
---+----------+--------------------
1  | Amal     | 0600..., 0611...
2  | Yassine | 0622...
1  | Amal     | 0600..., 0611...
```

Answer the following questions.

1. Which rule is violated by the `phones` cell in the first row?
2. Which rule is violated by rows 1 and 3?
3. Why should row order not be treated as part of the relation?
4. Rewrite the design so that phone numbers are atomic.

### Question 6 [concept] Schema versus instance

Answer the following questions.

1. What is the difference between a schema and an instance?
2. Which one usually changes rarely?
3. Which one changes constantly?

4. Give one example of a schema for students.
5. Give one example of an instance of that schema.

## 23.3 Keys and integrity

**Learning outcome 23.5: Reason about keys**

Students should be able to distinguish superkeys, candidate keys, primary keys, natural keys, and surrogate keys.

**Question 7 [analysis] Superkeys, candidate keys, and primary keys**

Consider:

```
Student(id, email, name, major)
```

Assume `id` is unique and `email` is unique. `name` is not unique.
Answer the following questions.

1. Give two candidate keys.
2. Give one superkey that is not a candidate key.
3. Why is `name` not a candidate key?
4. If we choose `id` as the primary key, what role can `email` still play?
5. Why is a small, stable surrogate key often preferred as a primary key?

**Learning outcome 23.6: Explain foreign keys and referential integrity**

Students should be able to identify foreign keys and explain how they prevent dangling references.

**Question 8 [analysis] Foreign-key check**

Consider these tables:

```
Student
id | name
---+--------
1  | Amal
2  | Yassine

Enrollment
id | student_id | course
---+------------+-------
10 | 1          | CS200
11 | 1          | MA101
12 | 2          | CS200
```

Answer the following questions.

1. Which attribute in `Enrollment` is a foreign key?
2. Which primary key does it reference?
3. Would `Enrollment(13, 99, "CS200")` violate referential integrity? Explain.
4. What problem can occur if `Student` row `id = 1` is deleted while enrollments still reference it?
5. What does it mean to declare this rule once and let the DBMS enforce it?

**Question 9 [concept] Null is not an ordinary value**

Answer the following questions.

1. What does `null` represent?
2. Why is `null` not the same as `0`?
3. Why is `null` not the same as the empty string?
4. Why must a primary key never be `null`?
5. Why can treating `null` as an ordinary value create bugs?

## 23.4 Entity-relationship modeling

**Learning outcome 23.7: Build an ER model from a problem statement**

Students should be able to identify entities, attributes, relationships, relationship attributes, and cardinalities before writing tables.

**Question 10 [modeling] University ER model**

A university records students, courses, and the grade each student receives in each course.
Answer the following questions.

1. What are the entity sets?
2. What relationship connects them?
3. What is the cardinality of the relationship?
4. Where does the attribute `grade` belong: on `Student`, on `Course`, or on the relationship? Explain.
5. Give one key attribute for each entity set.

**Question 11 [modeling] Library ER model**

A library lends books to members. A member can borrow many books. A book can be borrowed by many members over time. Each loan has a `borrowed_on` date and a `due_date`.
Answer the following questions.

1. What are the entity sets?

2. What relationship connects them?
3. What is the cardinality over time?
4. Where should `borrowed_on` and `due_date` belong?
5. Why might a real library model `BookCopy` separately from `Book`?

**Question 12 [concept] Attribute flavors**

Classify each attribute as key, simple, composite, multivalued, or derived. Some attributes may fit more than one category depending on the model.

1. `student_id`
2. `birthdate`
3. `age`, computed from `birthdate`
4. `address`, made of street, city, and zip code
5. `phone_numbers`, where a student may have several phone numbers
6. `course_code`

## 23.5 From ER to tables

**Learning outcome 23.8: Translate ER models into relational schemas**

Students should be able to map entity sets, one-to-many relationships, many-to-many relationships, and multivalued attributes into tables with primary and foreign keys.

**Question 13 [modeling] Map an M:N relationship to tables**

Translate the university model into relations:

```
Student -- Enrolls -- Course
             grade
```

The relationship is many-to-many.
Answer the following questions.

1. Which tables are needed?
2. What attributes should each table contain?
3. What is the primary key of `Enrollment`?
4. Which attributes in `Enrollment` are foreign keys?
5. Why is `grade` stored in `Enrollment`?

**Question 14 [modeling] Map a 1:N relationship to tables**

A department has many employees. Each employee belongs to exactly one department.
Answer the following questions.

1. What are the two entity sets?
2. What is the cardinality?
3. Which table receives the foreign key?
4. Write a relational schema for `Department` and `Employee`.
5. Why does the foreign key go on the many side?

**Question 15 [modeling] Map a multivalued attribute**

A student can have several phone numbers. Assume the base table is:

```
Student(id, name, major)
```

Answer the following questions.

1. Why should we not store all phone numbers in one `phones` cell?
2. Create a separate table for phone numbers.
3. What is the foreign key in that table?
4. Suggest a primary key for the phone-number table.

## 23.6 Designing good schemas

**Learning outcome 23.9: Detect redundancy and anomalies**

Students should be able to explain why one large table can create update, insertion, and deletion anomalies.

**Question 16 [analysis] The fat-table trap**

Consider this table:

```
sid | sname   | course | ctitle     | grade
----+---------+--------+------------+------
1   | Amal    | CS200  | Algorithms | A
1   | Amal    | MA101  | Calculus   | B
2   | Yassine | CS200  | Algorithms | A
```

Answer the following questions.

1. Identify at least two repeated facts.
2. Explain the update anomaly using `Amal`.
3. Explain the insertion anomaly using a new course with no students yet.
4. Explain the deletion anomaly using the last enrollment in `CS200`.
5. Decompose the table into cleaner relations.

**Learning outcome 23.10: Explain normalization and denormalization**

Students should be able to state the intuition behind 1NF, 2NF, 3NF, and explain when denormalization may be used deliberately.

**Question 17 [concept] Normalization in plain words**

Answer the following questions.

1. What is the basic goal of normalization?
2. State 1NF in plain words.
3. State 2NF in plain words.
4. State 3NF in plain words.
5. Explain the slogan: "the key, the whole key, and nothing but the key."
6. When might denormalization be justified?

**Question 18 [synthesis] Choose the right design**

For each situation, say whether the design should likely use a normalized relational schema, controlled denormalization, or a non-relational/document-style design. Briefly justify each answer.

1. A university registration system where enrollments, courses, students, and grades must remain consistent.
2. A reporting dashboard that reads huge precomputed analytics tables but rarely updates them.
3. A small prototype script storing ten records for a one-time experiment.
4. A production system where many users update shared records at the same time.

Chapter 24

# Solutions: Introduction to Databases

This chapter gives solutions to the review questions from the previous chapter. The solutions emphasize persistence, sharing, scale, integrity, relational terminology, keys, ER modeling, ER-to-table mapping, and schema design.

## 24.1 Solutions for why databases matter

**Solution 1 [concept]**

1. In-memory data structures usually assume that the data lives inside one running program, in RAM.
2. Unless the program explicitly saves it, the data disappears when the process ends.
3. The four pressures are persistence, sharing, scale, and integrity.
4. A DBMS is not only storage. It manages persistent data, concurrent access, large data, crash safety, constraints, and safe updates. It is a guarantee engine, not just a file format.

**Solution 2 [concept]**

Definitions:

1. A database is an organized, persistent collection of related data that can be stored, queried, updated, and shared safely.
2. A DBMS is the software system that stores and manages the database, such as PostgreSQL, MySQL, SQLite, Oracle, or MongoDB.
3. A data model is the abstract set of rules for structuring data. The relational model is one example.
4. A schema is the structure of a particular database: its tables, attributes, domains, keys, and constraints.
5. An instance is the actual data currently stored in the schema.
6. A query language is the language used to ask questions and modify data. SQL is the standard language for relational databases.

Classification:

1. PostgreSQL is a DBMS.
2. SQL is a query language.

3. `Student(id, name, major)` is part of a schema.
4. The current rows in `Student` are an instance.
5. The relational model is a data model.

**Solution 3 [analysis]**

1. It is acceptable for a toy example because the data is small, controlled by one program, and does not need serious concurrent access, crash safety, indexing, or integrity constraints.
2. If two programs write at the same time, one write may overwrite the other, interleave with it, or leave the file inconsistent unless explicit locking is implemented correctly.
3. Querying by content is hard because a pickle file is not organized for queries. To find all CS students, the program may need to load or scan all records itself.
4. A crash mid-write can leave a partially written or corrupt file.
5. A DBMS addresses these with transactions, isolation, atomic durable commits, indexes, buffer management, and declared integrity constraints.

## 24.2 Solutions for the relational model

**Solution 4 [concept]**

1. The relation is `Student`.
2. One tuple is `(1, "Amal", "CS")`.
3. One attribute is `name`. Other valid answers include `id` and `major`.
4. The degree is 3 because there are three attributes.
5. The cardinality is 3 because there are three tuples.
6. A possible domain for `id` is positive integers.

**Solution 5 [analysis]**

1. The `phones` cell violates atomicity, also called First Normal Form. A cell should not contain a list of values.
2. Rows 1 and 3 violate the set property: a relation cannot contain duplicate tuples.
3. Row order is not part of a relation because a relation is a set. Ordering is produced by a query, not stored as a property of the table.
4. A cleaner design separates phone numbers:

```
Student(id, name)
StudentPhone(student_id, phone)
```

Example instance:

```
Student
id | name
```

```
---+--------
1  | Amal
2  | Yassine

StudentPhone
student_id | phone
-----------+---------
1          | 0600...
1          | 0611...
2          | 0622...
```

**Solution 6 [concept]**

1. The schema is the structure: table names, attributes, domains, keys, and rules. The instance is the actual rows stored at a particular time.
2. The schema usually changes rarely.
3. The instance changes constantly as rows are inserted, updated, and deleted.
4. Example schema: `Student(id, name, major)`.
5. Example instance:

```
id | name  | major
---+-------+------
1  | Amal  | CS
2  | Salma | Math
```

## 24.3 Solutions for keys and integrity

**Solution 7 [analysis]**

1. Two candidate keys are `{id}` and `{email}`, assuming each is unique and minimal.
2. `{id, name}` is a superkey but not a candidate key, because `id` alone already identifies a tuple.
3. `name` is not a candidate key because different students can have the same name.
4. If `id` is the primary key, `email` can still be declared as a unique candidate key.
5. A small, stable surrogate key is often preferred because it has no business meaning, is compact, is not reused, and should not change when real-world attributes change.

**Solution 8 [analysis]**

1. `Enrollment.student_id` is the foreign key.
2. It references `Student.id`.
3. Yes. `student_id = 99` violates referential integrity because no student with `id = 99` exists.

4. Deleting `Student.id = 1` while rows still reference it would create dangling enrollments unless the DBMS blocks the deletion or applies a specified action such as cascade.
5. It means the rule is part of the database schema. Every application using the database is subject to the same rule, so correctness does not depend on remembering to check it manually in every program.

**Solution 9 [concept]**

1. `null` represents an unknown, missing, or not applicable value.
2. `null` is not `0`; zero is a known numeric value.
3. `null` is not the empty string; the empty string is a known text value of length zero.
4. A primary key must never be `null` because it is supposed to identify each row. An unknown identifier does not identify anything reliably.
5. Treating `null` as ordinary can create bugs because comparisons involving `null` do not behave like normal true/false comparisons. SQL uses special tests such as `IS NULL`.

## 24.4 Solutions for ER modeling

**Solution 10 [modeling]**

1. The entity sets are `Student` and `Course`.
2. The relationship is `Enrolls`.
3. The relationship is many-to-many: a student can enroll in many courses, and a course can contain many students.
4. `grade` belongs on the relationship. It is not a property of the student alone or the course alone. It describes a particular student's enrollment in a particular course.
5. Example key attributes: `Student.id` and `Course.code`.

**Solution 11 [modeling]**

1. The entity sets are at least `Member` and `Book`. In a more realistic model, `BookCopy` should also appear.
2. The relationship is `Borrows` or `Loan`.
3. Over time, the cardinality is many-to-many: a member can borrow many books, and a book can be borrowed by many members over time.
4. `borrowed_on` and `due_date` belong on the relationship, because they describe a specific loan event.
5. A real library may own several physical copies of the same book title. `Book` describes the title, while `BookCopy` describes a specific physical copy that can be borrowed.

**Solution 12 [concept]**

1. `student_id`: key attribute if it uniquely identifies a student.
2. `birthdate`: simple attribute if stored as one atomic date.
3. `age`: derived attribute if computed from `birthdate`.
4. `address`: composite attribute if decomposed into street, city, and zip code.
5. `phone_numbers`: multivalued attribute because a student may have several phone numbers.
6. `course_code`: key attribute if it uniquely identifies a course.

## 24.5 Solutions for ER-to-table mapping

**Solution 13 [modeling]**

1. The needed tables are `Student`, `Course`, and `Enrollment`.
2. A reasonable schema is:

```
Student(id, name, major)
Course(code, title, credits)
Enrollment(student_id, course_code, grade)
```

3. The primary key of `Enrollment` is usually the composite key `(student_id, course_code)`.
4. `Enrollment.student_id` references `Student.id`. `Enrollment.course_code` references `Course.code`.
5. `grade` is stored in `Enrollment` because it belongs to the student-course pair, not to the student alone and not to the course alone.

**Solution 14 [modeling]**

1. The entity sets are `Department` and `Employee`.
2. The cardinality is 1:$N$: one department has many employees, and each employee belongs to one department.
3. The foreign key goes in `Employee`, the many side.
4. A reasonable schema is:

```
Department(dept_id, name)
Employee(emp_id, name, dept_id)
```

Here, `Employee.dept_id` is a foreign key referencing `Department.dept_id`.

5. The foreign key goes on the many side because many employee rows can store the same `dept_id`. Putting many employees inside one department cell would violate atomicity.

**Solution 15 [modeling]**

1. We should not store all phone numbers in one cell because that violates atomicity and First Normal Form.
2. A separate table is:

```
StudentPhone(student_id, phone)
```

3. `student_id` is a foreign key referencing `Student.id`.
4. A reasonable primary key is `(student_id, phone)` if the same phone should not be repeated for the same student. Another design could use a surrogate `phone_id` plus a foreign key to `Student`.

## 24.6 Solutions for schema design

**Solution 16 [analysis]**

1. Repeated facts include the student name `Amal` and the course title `Algorithms`. The course code `CS200` is also repeated because several students enroll in it.
2. Update anomaly: if Amal changes her name, every row containing Amal must be updated. If one row is missed, the database contradicts itself.
3. Insertion anomaly: if a new course has no students yet, there may be no row in this table where the course can be stored without inventing enrollment data.
4. Deletion anomaly: deleting the last enrollment in `CS200` would also delete the only stored fact that `CS200` is `Algorithms`.
5. A cleaner design is:

```
Student(id, name)
Course(code, title)
Enrollment(student_id, course_code, grade)
```

**Solution 17 [concept]**

1. The goal of normalization is to reduce redundancy and store each independent fact in one place.
2. 1NF means every cell is atomic: no lists, repeating groups, or nested tables inside a cell.
3. 2NF means that when a table has a composite key, non-key attributes should depend on the whole key, not only part of it.
4. 3NF means non-key attributes should not depend on other non-key attributes.
5. The slogan means every non-key attribute should describe the whole primary key and nothing else. It should not describe only part of a composite key or another non-key attribute.

6. Denormalization may be justified for measured read performance, analytics, warehouses, or document-style systems where controlled duplication is intentional and maintained carefully.

**Solution 18 [synthesis]**

1. A university registration system should use a normalized relational schema. The data has clear entities and relationships, and correctness is critical.
2. A reporting dashboard may use controlled denormalization. If the workload is mostly reads over precomputed analytics, wide redundant tables can be acceptable for performance.
3. A small one-time prototype may not need a full database. A simple file or in-memory structure may be enough if there is no concurrency, scale, or long-term integrity requirement.
4. A production system with many concurrent users should use a DBMS with transactions and constraints. A normalized relational design is usually the starting point.

Chapter 25

# SQL Foundations: Review Questions

This chapter reviews Lecture 13 of COMP102. The focus is on reading and writing SQL commands: table creation, insertion, selection, filtering, ordering, joins, aggregation, grouping, and basic indexing.

Solutions are intentionally placed in the following chapter.

## 25.1 Reference schema and data

Use the following university schema for the exercises unless a question says otherwise.

```
Student(id, name, major)
Course(code, title, credits, department)
Enrollment(student_id, course_code, grade)
```

The intended keys are:

- `Student.id` is the primary key of `Student`.
- `Course.code` is the primary key of `Course`.
- `Enrollment(student_id, course_code)` is the primary key of `Enrollment`.
- `Enrollment.student_id` references `Student.id`.
- `Enrollment.course_code` references `Course.code`.

Sample data:

```
Student
id | name    | major
---+---------+------
1  | Amal    | CS
2  | Yassine | Math
3  | Salma   | CS
4  | Nora    | NULL

Course
code   | title       | credits | department
-------+-------------+---------+-----------
CS200 | Algorithms  | 4       | CS
MA101 | Calculus    | 3       | Math
CS150 | Programming | 4       | CS
```

```
PH100 | Physics     | 3       | Physics

Enrollment
student_id | course_code | grade
-----------+-------------+------
1          | CS200       | 17.0
1          | MA101       | 14.0
2          | MA101       | 16.0
3          | CS200       | 18.0
3          | CS150       | 15.5
```

## 25.2 From schema to SQL tables

**Learning outcome 25.1: Write table-definition SQL**

Students should be able to write `CREATE TABLE` commands with column types, primary keys, `NOT NULL`, foreign keys, and composite primary keys.

**Question 1 [syntax] Reading a CREATE TABLE command**

Consider the following SQL command.

```
CREATE TABLE Student (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    major TEXT
);
```

Answer the following questions.

1. What is the table name?
2. What are the three column names?
3. Which column is the primary key?
4. Which column is forbidden from being `NULL`?
5. Which column may be `NULL`?
6. What does the semicolon do?

**Question 2 [sql] Create the course table**

Write one SQL command that creates the `Course` table with the following columns and constraints:

- `code`: text primary key,
- `title`: text, required,
- `credits`: integer, required,
- `department`: text, required.

**Question 3 [sql] Create the relationship table**

Write one SQL command that creates the `Enrollment` table with:

- `student_id`: integer, references `Student(id)`,
- `course_code`: text, references `Course(code)`,
- `grade`: real number,
- composite primary key `(student_id, course_code)`.

The command should express the many-to-many relationship between students and courses.

**Learning outcome 25.2: Write insertion SQL**

Students should be able to write `INSERT INTO ... VALUES` commands and match values to the column list in the correct order.

**Question 4 [sql] Insert rows**

Write SQL commands for the following insertions.

1. Insert student `(5, 'Imane', 'Physics')` into `Student`.
2. Insert course `('DS100', 'Data Systems', 3, 'CS')` into `Course`.
3. Insert enrollment `(5, 'DS100', 16.5)` into `Enrollment`.

Use explicit column lists in each command.

**Question 5 [analysis] Integrity from SQL constraints**

Using the reference schema, answer the following questions.

1. What happens if we try to insert two students with `id = 1`?
2. What happens if we try to insert an enrollment with `student_id = 99` when no student has that id?
3. Which SQL clauses or constraints are responsible for these two protections?

## 25.3 Basic querying

**Learning outcome 25.3: Use SELECT, FROM, and WHERE**

Students should be able to retrieve selected columns from a table and filter rows using `WHERE` conditions.

**Question 6 [sql] Basic SELECT commands**

Write one SQL query for each task.

1. Show all columns from `Student`.

2. Show only student names and majors.
3. Show all course codes and titles.
4. Show all enrollment rows.

### Question 7 [sql] Filtering with WHERE

Write one SQL query for each task.

1. Show the names of students whose major is `'CS'`.
2. Show the titles of courses in the `'Math'` department.
3. Show the titles and credits of courses worth at least 4 credits.
4. Show the student id and course code for enrollments whose grade is at least 16.

### Question 8 [sql] Combining conditions

Write one SQL query for each task.

1. Show the titles of courses where `department = 'CS'` and `credits = 4`.
2. Show student names and majors where the major is `'CS'` or `'Math'`.
3. Show enrollments where `course_code = 'CS200'` and `grade > 17`.

### Learning outcome 25.4: Handle NULL correctly in SQL

Students should be able to explain why `= NULL` is wrong and use `IS NULL` and `IS NOT NULL` instead.

### Question 9 [sql] Filtering NULL values

Write one SQL query for each task.

1. Show the names of students whose major is missing.
2. Show the names and majors of students whose major is not missing.
3. Explain why this query returns no rows in the reference data:

```
SELECT name
FROM Student
WHERE major = NULL;
```

## 25.4 Ordering and limiting results

**Learning outcome 25.5: Use ORDER BY and LIMIT**

Students should be able to sort query results explicitly and keep only the first rows after sorting.

**Question 10 [sql] Ordering commands**

Write one SQL query for each task.

1. Show student names and majors sorted alphabetically by name.
2. Show course titles and credits sorted from highest credits to lowest credits.
3. Show course titles and credits sorted by credits descending, then title ascending.
4. Show enrollment rows sorted from highest grade to lowest grade.

**Question 11 [sql] LIMIT commands**

Write one SQL query for each task.

1. Show the two courses with the highest number of credits.
2. Show the three best grades in `Enrollment`.
3. Show the alphabetically first two students by name.

Each query should use `ORDER BY` before `LIMIT`.

**Question 12 [concept] Why ORDER BY matters**

Answer the following questions.

1. Why should we not rely on the stored order of rows in a table?
2. What does `ORDER BY` change: the table itself or the query result?
3. Why is `LIMIT` without `ORDER BY` usually not a reliable way to ask for the “top” rows?

## 25.5 Joins

**Learning outcome 25.6: Write JOIN ... ON queries**

Students should be able to recombine normalized tables using `JOIN`, `ON`, qualified column names, and table aliases.

**Question 13 [sql] Two-table join**

Write a SQL query that shows each enrolled student’s name, course code, and grade. Required output columns:

```
student name | course code | grade
```

Use `Enrollment` and `Student`. Use aliases `e` and `s`. Sort by student name, then course code.

**Question 14 [sql] Three-table join**

Write a SQL query that shows each enrolled student's name, course title, and grade. Required output columns:

```
student name | course title | grade
```

Use `Enrollment`, `Student`, and `Course`. Use aliases `e`, `s`, and `c`. Sort by student name, then course title.

**Question 15 [sql] Filtering after joins**

Write one SQL query for each task.

1. Show the names of students enrolled in `'Algorithms'`.
2. Show the titles and grades of courses taken by `'Amal'`.
3. Show student names, course titles, and grades for enrollments in the `'CS'` department.

**Question 16 [analysis] Avoiding Cartesian products**

Consider this query.

```
SELECT Student.name, Course.title
FROM Student, Course;
```

Answer the following questions.

1. What does this query produce if `Student` has 4 rows and `Course` has 4 rows?
2. Why are most of those rows meaningless for the university schema?
3. Which table is missing from the query?
4. Rewrite the query so that it returns only real student-course enrollments.

## 25.6 Aggregation and grouping

**Learning outcome 25.7: Use aggregate functions**

Students should be able to use `COUNT`, `AVG`, `MIN`, and `MAX` to summarize rows.

**Question 17 [sql] Whole-table aggregates**

Write one SQL query for each task.

1. Count all rows in `Enrollment`.
2. Compute the average grade across all enrollments.

3. Show the lowest and highest grade in `Enrollment`.
4. Count all students in `Student`.

**Question 18 [sql] Aggregation after filtering**

Write one SQL query for each task.

1. Compute the average grade in `'CS200'`.
2. Count enrollments with grade at least 16.
3. Compute the highest grade in `'MA101'`.

**Learning outcome 25.8: Use GROUP BY**

Students should be able to compute one summary row per group using `GROUP BY`.

**Question 19 [sql] Grouped summaries**

Write one SQL query for each task.

1. For each course code, show the number of enrolled students.
2. For each course code, show the average grade.
3. For each course title, show the average grade, sorted from highest average to lowest average.
4. For each student name, show how many courses the student is enrolled in.

**Question 20 [analysis] WHERE and GROUP BY order**

Consider this incorrect query.

```
SELECT course_code, COUNT(*) AS n
FROM Enrollment
WHERE COUNT(*) >= 2
GROUP BY course_code;
```

Answer the following questions.

1. Why is `WHERE COUNT(*) >= 2` wrong?
2. What does `WHERE` filter?
3. What does `GROUP BY` create?
4. Write a correct query that shows course codes with at least two enrollments.

## 25.7 Indexing basics

**Learning outcome 25.9: Reason about indexes from SQL queries**

Students should be able to identify when an index may help, write `CREATE INDEX`, and explain the tradeoff between faster reads and extra write/storage cost.

**Question 21 [concept] Reading a query plan**

Consider this command.

```
EXPLAIN QUERY PLAN
SELECT id, name, major
FROM Student
WHERE major = 'CS';
```

Answer the following questions.

1. What is the purpose of `EXPLAIN QUERY PLAN`?
2. What does a plan containing `SCAN Student` suggest?
3. What does a plan containing `SEARCH Student USING INDEX idx_student_major` suggest?

**Question 22 [sql] Create indexes for queries**

For each query, decide whether an index may help. If yes, write a suitable `CREATE INDEX` command. If no, explain why not.

1. `SELECT * FROM Student WHERE major = 'CS';`
2. `SELECT * FROM Enrollment WHERE student_id = 3;`
3. `SELECT * FROM Course;`
4. `SELECT * FROM Course WHERE department = 'CS';`
5. `SELECT * FROM Enrollment WHERE course_code = 'CS200';`

**Question 23 [analysis] Index tradeoffs**

Answer the following questions.

1. What is one benefit of an index?
2. What are two costs of an index?
3. Why should we not index every column blindly?
4. Why should the decision to create an index start from a real query?

## 25.8 Mixed SQL practice

**Learning outcome 25.10: Choose SQL clauses based on the question**

Students should be able to translate a natural-language data question into the SQL clauses needed to answer it.

**Question 24 [sql] Full-query practice**

Write one SQL query for each task.

1. Show the names of CS students, sorted alphabetically.
2. Show the titles of the two highest-credit courses.
3. Show each course title with its number of enrolled students.
4. Show the name of each student with their average grade, sorted from highest average to lowest average.
5. Show the course title with the highest average grade among courses that have at least one enrollment.

Chapter 26

# Solutions: SQL Foundations

This chapter gives solutions to the review questions from the previous chapter. The solutions emphasize SQL commands themselves rather than the Python wrapper used to execute them.

## 26.1 Solutions for table definition and insertion

**Solution 1 [syntax]**

1. The table name is `Student`.
2. The columns are `id`, `name`, and `major`.
3. `id` is the primary key.
4. `name` is forbidden from being `NULL` because it has the `NOT NULL` constraint.
5. `major` may be `NULL` because it has no `NOT NULL` constraint.
6. The semicolon ends the SQL statement.

**Solution 2 [sql]**

```sql
CREATE TABLE Course (
    code TEXT PRIMARY KEY,
    title TEXT NOT NULL,
    credits INTEGER NOT NULL,
    department TEXT NOT NULL
);
```

**Solution 3 [sql]**

```sql
CREATE TABLE Enrollment (
    student_id INTEGER REFERENCES Student(id),
    course_code TEXT REFERENCES Course(code),
    grade REAL,
    PRIMARY KEY (student_id, course_code)
);
```

The composite primary key prevents the same student from being enrolled in the same course twice. The two foreign keys connect each enrollment row to one student and one course.

**Solution 4 [sql]**

```sql
INSERT INTO Student(id, name, major)
VALUES (5, 'Imane', 'Physics');

INSERT INTO Course(code, title, credits, department)
VALUES ('DS100', 'Data Systems', 3, 'CS');

INSERT INTO Enrollment(student_id, course_code, grade)
VALUES (5, 'DS100', 16.5);
```

**Solution 5 [analysis]**

1. The insertion is rejected because `Student.id` is a primary key and must be unique.
2. The insertion is rejected because `Enrollment.student_id` references `Student.id`, and there is no matching student with `id = 99`.
3. The first protection comes from `PRIMARY KEY`. The second protection comes from `REFERENCES Student(id)`, assuming foreign-key enforcement is enabled by the DBMS.

## 26.2 Solutions for basic querying

**Solution 6 [sql]**

1. Show all columns from `Student`:

```sql
SELECT *
FROM Student;
```

2. Show only student names and majors:

```sql
SELECT name, major
FROM Student;
```

3. Show all course codes and titles:

```sql
SELECT code, title
FROM Course;
```

4. Show all enrollment rows:

```sql
SELECT *
FROM Enrollment;
```

**Solution 7 [sql]**

1. CS students:

```sql
SELECT name
FROM Student
WHERE major = 'CS';
```

2. Math courses:

```sql
SELECT title
FROM Course
WHERE department = 'Math';
```

3. Courses worth at least 4 credits:

```sql
SELECT title, credits
FROM Course
WHERE credits >= 4;
```

4. Enrollments with grade at least 16:

```sql
SELECT student_id, course_code
FROM Enrollment
WHERE grade >= 16;
```

**Solution 8 [sql]**

1. CS courses worth 4 credits:

```sql
SELECT title
FROM Course
WHERE department = 'CS' AND credits = 4;
```

2. Students in CS or Math:

```sql
SELECT name, major
FROM Student
WHERE major = 'CS' OR major = 'Math';
```

3. CS200 enrollments with grade greater than 17:

```sql
SELECT student_id, course_code, grade
FROM Enrollment
WHERE course_code = 'CS200' AND grade > 17;
```

**Solution 9 [sql]**

1. Students whose major is missing:

```sql
SELECT name
FROM Student
WHERE major IS NULL;
```

2. Students whose major is not missing:

```sql
SELECT name, major
FROM Student
WHERE major IS NOT NULL;
```

3. The query using `major = NULL` returns no rows because `NULL` is not compared using `=`. Missing values must be tested with `IS NULL` or `IS NOT NULL`.

## 26.3 Solutions for ordering and limiting

**Solution 10 [sql]**

1. Students sorted by name:

```sql
SELECT name, major
FROM Student
ORDER BY name;
```

2. Courses from highest credits to lowest credits:

```sql
SELECT title, credits
FROM Course
ORDER BY credits DESC;
```

3. Credits descending, then title ascending:

```sql
SELECT title, credits
FROM Course
ORDER BY credits DESC, title ASC;
```

4. Enrollments from highest grade to lowest grade:

```sql
SELECT student_id, course_code, grade
FROM Enrollment
ORDER BY grade DESC;
```

**Solution 11 [sql]**

1. Two courses with highest credits:

```
SELECT title, credits
FROM Course
ORDER BY credits DESC, title ASC
LIMIT 2;
```

2. Three best grades:

```
SELECT student_id, course_code, grade
FROM Enrollment
ORDER BY grade DESC
LIMIT 3;
```

3. Alphabetically first two students:

```
SELECT name, major
FROM Student
ORDER BY name ASC
LIMIT 2;
```

**Solution 12 [concept]**

1. A table is a relation, and a relation is a set of rows. Sets do not have inherent order.
2. `ORDER BY` changes the order of the query result. It does not rearrange the stored table.
3. `LIMIT` without `ORDER BY` keeps some rows, but not necessarily the top rows by any meaningful criterion. To ask for the top rows, define the ordering first.

## 26.4 Solutions for joins

**Solution 13 [sql]**

```
SELECT s.name, e.course_code, e.grade
FROM Enrollment AS e
JOIN Student AS s ON e.student_id = s.id
ORDER BY s.name, e.course_code;
```

**Solution 14 [sql]**

```
SELECT s.name, c.title, e.grade
FROM Enrollment AS e
JOIN Student AS s ON e.student_id = s.id
JOIN Course AS c ON e.course_code = c.code
```

```
ORDER BY s.name, c.title;
```

**Solution 15 [sql]**

1. Students enrolled in Algorithms:

```
SELECT s.name
FROM Enrollment AS e
JOIN Student AS s ON e.student_id = s.id
JOIN Course AS c ON e.course_code = c.code
WHERE c.title = 'Algorithms'
ORDER BY s.name;
```

2. Courses taken by Amal:

```
SELECT c.title, e.grade
FROM Enrollment AS e
JOIN Student AS s ON e.student_id = s.id
JOIN Course AS c ON e.course_code = c.code
WHERE s.name = 'Amal'
ORDER BY c.title;
```

3. Enrollments in the CS department:

```
SELECT s.name, c.title, e.grade
FROM Enrollment AS e
JOIN Student AS s ON e.student_id = s.id
JOIN Course AS c ON e.course_code = c.code
WHERE c.department = 'CS'
ORDER BY c.title, s.name;
```

**Solution 16 [analysis]**

1. It produces a Cartesian product with $4 \times 4 = 16$ rows.
2. Most rows are meaningless because the query pairs every student with every course, regardless of whether the student is actually enrolled in the course.
3. The missing table is `Enrollment`, the relationship table.
4. Correct query:

```
SELECT s.name, c.title
FROM Enrollment AS e
JOIN Student AS s ON e.student_id = s.id
JOIN Course AS c ON e.course_code = c.code
ORDER BY s.name, c.title;
```

## 26.5 Solutions for aggregation and grouping

**Solution 17 [sql]**

1. Count all enrollments:

```sql
SELECT COUNT(*)
FROM Enrollment;
```

2. Average grade:

```sql
SELECT AVG(grade)
FROM Enrollment;
```

3. Lowest and highest grade:

```sql
SELECT MIN(grade), MAX(grade)
FROM Enrollment;
```

4. Count all students:

```sql
SELECT COUNT(*)
FROM Student;
```

**Solution 18 [sql]**

1. Average grade in CS200:

```sql
SELECT AVG(grade)
FROM Enrollment
WHERE course_code = 'CS200';
```

2. Count enrollments with grade at least 16:

```sql
SELECT COUNT(*)
FROM Enrollment
WHERE grade >= 16;
```

3. Highest grade in MA101:

```sql
SELECT MAX(grade)
FROM Enrollment
WHERE course_code = 'MA101';
```

**Solution 19 [sql]**

1. Number of enrolled students per course code:

```sql
SELECT course_code, COUNT(*) AS number_of_students
```

```
FROM Enrollment
GROUP BY course_code;
```

2. Average grade per course code:

```
SELECT course_code, AVG(grade) AS average_grade
FROM Enrollment
GROUP BY course_code;
```

3. Average grade per course title:

```
SELECT c.title, AVG(e.grade) AS average_grade
FROM Enrollment AS e
JOIN Course AS c ON e.course_code = c.code
GROUP BY c.code, c.title
ORDER BY average_grade DESC;
```

4. Number of courses per student name:

```
SELECT s.name, COUNT(*) AS number_of_courses
FROM Enrollment AS e
JOIN Student AS s ON e.student_id = s.id
GROUP BY s.id, s.name
ORDER BY s.name;
```

**Solution 20 [analysis]**

1. `WHERE COUNT(*) >= 2` is wrong because `WHERE` is applied before groups exist. `COUNT(*)` is computed after grouping.
2. `WHERE` filters individual rows before grouping.
3. `GROUP BY` creates groups of rows that share the same grouping value.
4. A correct query uses `HAVING` to filter grouped results:

```
SELECT course_code, COUNT(*) AS n
FROM Enrollment
GROUP BY course_code
HAVING COUNT(*) >= 2;
```

## 26.6 Solutions for indexing basics

**Solution 21 [concept]**

1. `EXPLAIN QUERY PLAN` asks SQLite to describe how it plans to execute the query. It is for inspecting the strategy, not for returning the query result.
2. `SCAN Student` suggests that SQLite scans the table rows.

3. `SEARCH Student USING INDEX idx_student_major` suggests that SQLite uses the named index to look up rows more directly for the `major` condition.

**Solution 22 [sql]**

1. This query filters by `major`, so an index may help on a large table:

```
CREATE INDEX idx_student_major
ON Student(major);
```

2. This query filters by `student_id`, so an index may help:

```
CREATE INDEX idx_enrollment_student
ON Enrollment(student_id);
```

3. `SELECT * FROM Course;` scans all rows and has no filter. An index is unlikely to help because the query asks for the whole table.
4. This query filters by `department`, so an index may help on a large course table:

```
CREATE INDEX idx_course_department
ON Course(department);
```

5. This query filters by `course_code`, so an index may help:

```
CREATE INDEX idx_enrollment_course
ON Enrollment(course_code);
```

**Solution 23 [analysis]**

1. A benefit of an index is faster lookup for queries that filter, join, or sort using the indexed column.
2. Two costs are extra storage space and slower inserts, updates, or deletes because the index must also be maintained.
3. Indexing every column blindly can create useless overhead. Some indexes will not support real queries, and some queries return so many rows that an index may not help much.
4. The decision should start from a real query because an index is useful only when it matches how the data is actually accessed.

## 26.7 Solutions for mixed SQL practice

**Solution 24 [sql]**

1. Names of CS students, sorted alphabetically:

```
SELECT name
FROM Student
WHERE major = 'CS'
ORDER BY name;
```

2. Titles of the two highest-credit courses:

```
SELECT title
FROM Course
ORDER BY credits DESC, title ASC
LIMIT 2;
```

3. Each course title with its number of enrolled students:

```
SELECT c.title, COUNT(*) AS number_of_students
FROM Enrollment AS e
JOIN Course AS c ON e.course_code = c.code
GROUP BY c.code, c.title
ORDER BY c.title;
```

4. Each student with average grade:

```
SELECT s.name, AVG(e.grade) AS average_grade
FROM Enrollment AS e
JOIN Student AS s ON e.student_id = s.id
GROUP BY s.id, s.name
ORDER BY average_grade DESC;
```

5. Course title with the highest average grade:

```
SELECT c.title, AVG(e.grade) AS average_grade
FROM Enrollment AS e
JOIN Course AS c ON e.course_code = c.code
GROUP BY c.code, c.title
ORDER BY average_grade DESC
LIMIT 1;
```

Chapter 27

# NoSQL and Document Stores: Review Questions

This chapter reviews Lecture 14 of COMP102. The focus is conceptual: why document databases exist, how documents differ from relational tables, what "schema-less" really means, and when SQL or NoSQL is the better fit.

Solutions are intentionally placed in the following chapter.

## 27.1 Reference examples

Use the following relational university schema when a question mentions SQL.

```
Student(id, name, major)
Course(code, title, credits, department)
Enrollment(student_id, course_code, grade)
```

Use the following document-shaped student profile when a question mentions a document store.

```
{
    "_id": 1,
    "name": "Amal",
    "major": "CS",
    "phones": ["0600000000", "0611111111"],
    "skills": ["Python", "SQL", "Git"],
    "address": {
        "city": "Rabat",
        "street": "Avenue Mohammed V"
    },
    "preferences": {
        "theme": "dark",
        "language": "en"
    }
}
```

## 27.2 Why NoSQL after SQL?

**Learning outcome 27.1: Explain the motivation for document databases**

Students should be able to explain why flat relational tables are not always the most natural shape for object-like data with nested fields, arrays, and optional attributes.

**Question 1 [concept] Flat tables and object-shaped data**

Explain why the following information does not fit naturally into one clean relational table:

- a student has one name;
- a student has several phone numbers;
- a student has several skills;
- a student has an address with city and street;
- some students have optional preferences and others do not.

Your answer should mention atomic cells and repeated or optional information.

**Question 2 [design] Bad relational design**

Consider this table design.

```
Student(
    id,
    name,
    major,
    phones,
    skills,
    address
)
```

Assume `phones`, `skills`, and `address` are stored as long text strings such as `'0600,0611'` or `'Python,SQL,Git'`.
Answer the following questions.

1. Which relational design rule is being violated?
2. Why is storing lists inside text fields a problem for querying?
3. What would a cleaner relational design do instead?

**Question 3 [concept] Relational fix versus document fit**

A relational design may split a student profile into tables such as:

```
Student(id, name, major)
StudentPhone(student_id, phone)
StudentSkill(student_id, skill)
StudentAddress(student_id, city, street)
StudentPreference(student_id, key, value)
```

Explain why this design is cleaner than stuffing everything into one table, but also why a document store may feel more natural when the application usually reads one complete student profile at a time.

**Question 4 [concept] NoSQL is not automatically better**

Explain why the sentence "NoSQL is better than SQL" is too vague to be useful.
Your answer should mention data shape, query patterns, relationships, integrity, and flexibility.

## 27.3 NoSQL models and document-store vocabulary

**Learning outcome 27.2: Distinguish document stores from other NoSQL models**

Students should be able to identify document stores as one NoSQL model among others, and avoid treating NoSQL as a single technology.

**Question 5 [matching] NoSQL model matching**

Match each NoSQL model to its main idea.

| Model | Main idea |
|---|---|
| A. Key-value store | 1. Data is stored as nodes and edges. |
| B. Document store | 2. A key maps directly to a value. |
| C. Graph database | 3. Data is stored as JSON-like objects. |
| D. Wide-column store | 4. Data is organized for very large distributed column-family workloads. |

**Question 6 [concept] Why this lecture focuses on document stores**

Give two reasons document stores are a reasonable NoSQL model to study after Python dictionaries and SQL tables.

**Learning outcome 27.3: Use document-store vocabulary**

Students should be able to map relational vocabulary to document-store vocabulary: table, row, column, cell, primary key, collection, document, field, value, and `_id`.

**Question 7 [matching] Relational vocabulary versus document vocabulary**

Fill in the missing document-store terms.

| Relational database | Document store |
|---|---|
| Table | _____ |
| Row | _____ |
| Column | _____ |
| Cell value | _____ |
| Primary key | _____ |

**Question 8 [concept] Document anatomy**

Using the reference student document, answer the following questions.

1. What is the document identifier?
2. Name two fields whose values are arrays.
3. Name one field whose value is a nested object.
4. Which field inside `address` stores the city?
5. Why is this shape closer to a Python dictionary than to a SQL row?

**Question 9 [concept] Collections are not exactly tables**

A collection is often compared to a table. Explain why that comparison is useful, but also why it is incomplete.

## 27.4 Documents, JSON, and flexible fields

**Learning outcome 27.4: Explain JSON-like structure**

Students should be able to recognize JSON-like objects, arrays, nested objects, and the difference between JSON notation and Python notation.

**Question 10 [concept] JSON-like values**

For each field below, identify whether the value is a string, number, boolean, array, nested object, or null-like value.

```
{
    "name": "Amal",
    "year": 2,
    "active": true,
    "skills": ["Python", "SQL"],
    "address": {"city": "Rabat"},
    "advisor": null
}
```

**Question 11 [concept] Python versus JSON**

Correct the following Python-like object so that it is valid JSON notation.

```
{
    'name': 'Amal',
    'active': True,
    'scholarship': None
}
```

**Learning outcome 27.5: Explain schema-less discipline**

Students should be able to explain that schema-less design allows flexibility but still requires consistent field names, predictable types, and deliberate handling of optional fields.

**Question 12 [classification] Reasonable flexibility or careless inconsistency**

Classify each case as either **reasonable flexibility** or **careless inconsistency**. Briefly justify each answer.

1. Some student documents have a `skills` field; others do not.
2. Some documents use `name`; others use `full_name` for the same concept.
3. Some students have an `address` object; others do not.
4. Some documents store `year` as `2`; others store `year` as `"second"`.
5. A new optional field `scholarship` is added for students who have scholarships.

**Question 13 [concept] Where the schema moves**

In a relational database, the schema is strongly declared in tables. In a document store, the database may allow documents in the same collection to vary.
Explain what responsibility shifts to the application developer in a schema-less or flexible-schema design.

## 27.5 Embedding, referencing, and tradeoffs

**Learning outcome 27.6: Compare embedding and referencing**

Students should be able to explain the basic tradeoff between embedding related data in one document and referencing data stored elsewhere.

**Question 14 [concept] Embedding**

A student profile document embeds phones, skills, address, and preferences inside one document.

Answer the following questions.

1. What is convenient about embedding this information?
2. When might embedding create duplication problems?
3. What kind of application read pattern makes embedding attractive?

**Question 15 [concept] Referencing**

Suppose course information is shared by many students. For example, many student profiles mention the same course `CS200`.
Explain why storing the full course title, credits, and department inside every student document may be risky. What would referencing try to avoid?

**Question 16 [design] Student profile versus university enrollment**

For each case, choose the better likely fit: **SQL** or **document store**. Justify briefly.

1. A university enrollment system where students take many courses, courses contain many students, and grades belong to the enrollment relationship.
2. A user profile page with optional settings, phone numbers, skills, and nested address information.
3. A reporting system that computes averages across courses, departments, and enrollments.
4. A form-submission system where each form version may have slightly different optional fields.

## 27.6 SQL versus document stores

**Learning outcome 27.7: Choose SQL or NoSQL based on the problem**

Students should be able to choose between SQL and a document store using data shape, query pattern, integrity requirements, and acceptable duplication.

**Question 17 [comparison] When SQL is better**

List four reasons SQL is often the better choice for a system.

**Question 18 [comparison] When a document store is better**

List four reasons a document store may be the better choice for a system.

**Question 19 [analysis] Scenario classification**

For each scenario, choose **SQL** or **document store**. Give one sentence of justification.

1. Bank transfers where consistency rules are strict and contradictions are unacceptable.
2. Blog posts where each post has title, body, tags, author metadata, comments, and optional settings usually loaded together.
3. Product catalog pages where each product has different optional attributes depending on the category.
4. Course registration with prerequisites, students, courses, departments, grades, and many queries across relationships.
5. Application settings for users, where new optional settings are added frequently.

**Question 20 [concept] Final comparison**

Complete the comparison table.

| Question | SQL databases | Document stores |
|---|---|---|
| Main structure | | |
| Schema style | | |
| Relationship strategy | | |
| Common strength | | |
| Main risk | | |

**Question 21 [reflection] Short answer**

Write a short answer to this prompt:

> The database model should follow the problem, not the trend.

Explain what this means using one SQL example and one document-store example.

Chapter 28

# Solutions: NoSQL and Document Stores

This chapter gives solutions to the review questions from the previous chapter. The solutions stay conceptual and avoid unnecessary MongoDB method detail.

## 28.1 Solutions for NoSQL motivation

**Solution 1 [concept]**

This information does not fit naturally into one clean relational table because several fields are not atomic. A phone-number list and a skill list contain multiple values, not one indivisible value. An address has internal structure, such as city and street. Optional preferences may exist for some students but not others.
A single flat table would either hide lists inside text fields or create many nullable columns. Both designs are weak. A clean relational design would split repeated or structured facts into separate tables. A document design can instead store the profile as one nested object.

**Solution 2 [design]**

1. The design violates First Normal Form because cells should hold atomic values. Lists and structured objects should not be hidden inside one text cell.
2. Querying becomes awkward. For example, finding all students who know `Python` requires parsing the `skills` text field instead of checking a clean value. The database cannot easily enforce consistency inside those strings.
3. A cleaner relational design would split phones, skills, and address information into related tables such as `StudentPhone`, `StudentSkill`, and `StudentAddress`.

**Solution 3 [concept]**

The relational design is cleaner because each fact is stored in a structured place. Phone numbers are rows in `StudentPhone`; skills are rows in `StudentSkill`; address fields are real attributes rather than hidden text.
However, if the application usually loads one whole student profile, the relational design may require several joins or several queries. A document store may be more natural

because the profile can be stored and read as one object containing arrays and nested fields.

**Solution 4 [concept]**

The sentence is too vague because SQL and NoSQL solve different modeling problems. SQL is usually better when data has stable structure, strong relationships, integrity constraints, and many queries across entities. A document store is often better when data is object-shaped, nested, flexible, and usually read as a whole object.
The right choice depends on the data shape, the queries, the need for consistency, and whether duplication is acceptable.

## 28.2 Solutions for NoSQL models and vocabulary

**Solution 5 [matching]**

| | |
|---|---|
| A. Key-value store | 2. A key maps directly to a value. |
| B. Document store | 3. Data is stored as JSON-like objects. |
| C. Graph database | 1. Data is stored as nodes and edges. |
| D. Wide-column store | 4. Data is organized for very large distributed column-family workloads. |

**Solution 6 [concept]**

Document stores are reasonable to study here because they are close to Python dictionaries and lists: documents have fields, arrays, and nested objects. They also provide a clear contrast with SQL tables, because documents do not have to be flat rows with the same fixed columns.

**Solution 7 [matching]**

| Relational database | Document store |
|---|---|
| Table | Collection |
| Row | Document |
| Column | Field |
| Cell value | Field value |
| Primary key | `_id` |

**Solution 8 [concept]**

1. The document identifier is `_id: 1`.
2. Two array fields are `phones` and `skills`.
3. One nested object field is `address`. Another is `preferences`.
4. The city is stored in `address.city`.

5. The shape is closer to a Python dictionary because it uses named fields, arrays, and nested dictionaries. A SQL row is flat and normally stores one atomic value per cell.

**Solution 9 [concept]**

The comparison is useful because both tables and collections group related records. A SQL table groups rows about the same kind of entity, and a document collection groups related documents.
The comparison is incomplete because SQL rows in a table follow the same columns and are flat. Documents in a collection may have nested fields, arrays, and optional fields, so they are not forced to be identical flat rows.

## 28.3 Solutions for documents, JSON, and flexible fields

**Solution 10 [concept]**

| Field | Value type |
|---|---|
| `name` | string |
| `year` | number |
| `active` | boolean |
| `skills` | array |
| `address` | nested object |
| `advisor` | null-like value |

**Solution 11 [concept]**

Valid JSON notation uses double quotes, `true`, and `null`.

```
{
    "name": "Amal",
    "active": true,
    "scholarship": null
}
```

**Solution 12 [classification]**

1. Reasonable flexibility. `skills` may be optional.
2. Careless inconsistency. `name` and `full_name` represent the same concept with different field names.
3. Reasonable flexibility. Some students may not have an address recorded.
4. Careless inconsistency. The same field should not randomly switch between number and string.
5. Reasonable flexibility. A new optional field for scholarship holders is a legitimate

evolution of the document shape.

**Solution 13 [concept]**

The responsibility shifts toward the application developer. The database may accept documents with missing fields, inconsistent field names, or inconsistent types. The application must decide expected fields, optional fields, allowed types, nested structure, and how missing values are handled. Schema-less means the database is less strict by default; it does not mean design rules disappear.

## 28.4 Solutions for embedding, referencing, and tradeoffs

**Solution 14 [concept]**

1. Embedding is convenient because related profile data can be read in one document without joins.
2. Embedding may create duplication problems when the embedded information is shared by many documents and may change. For example, embedding the same course title in many student documents can lead to inconsistent copies.
3. Embedding is attractive when the application usually reads or writes the whole object together, such as loading a full profile page.

**Solution 15 [concept]**

Storing full course information inside every student document duplicates facts. If the course title or credits change, every copy must be updated. Some copies may be missed, creating contradictions. Referencing tries to keep shared facts in one place and store only a reference, such as a course code, in the student-related data.

**Solution 16 [design]**

1. SQL. This is a many-to-many relationship with grades on the relationship, so tables such as `Student`, `Course`, and `Enrollment` fit naturally.
2. Document store. The data is object-like, nested, optional, and usually loaded as one profile.
3. SQL. Reporting across courses, departments, and enrollments benefits from structured tables, joins, and aggregation.
4. Document store. Form submissions may vary by version, and optional fields are natural in documents.

## 28.5 Solutions for SQL versus document stores

**Solution 17 [comparison]**

SQL is often better when:

- the data has a stable structure;
- relationships between entities are important;
- integrity rules must be strongly enforced;
- queries often combine several entities;
- duplicate facts would cause serious update problems.

**Solution 18 [comparison]**

A document store may be better when:

- the data is naturally object-shaped;
- records contain nested fields or arrays;
- different records have different optional fields;
- the application usually reads or writes the whole object;
- the data shape changes frequently.

**Solution 19 [analysis]**

1. SQL. Bank transfers need strict consistency and strong integrity guarantees.
2. Document store. A blog post with metadata, tags, comments, and optional settings is naturally object-shaped and often loaded as one object.
3. Document store. Product attributes may vary widely by category, so flexible fields can help.
4. SQL. Course registration is relationship-heavy and needs structured querying across students, courses, departments, and grades.
5. Document store. User settings often evolve and contain optional fields that are read as one object.

**Solution 20 [concept]**

One possible completion is:

| Question | SQL databases | Document stores |
|---|---|---|
| Main structure | Tables | JSON-like documents |
| Schema style | Strong, declared schema | Flexible schema |
| Relationship strategy | Foreign keys and joins | Embedding or references |
| Common strength | Integrity and relationships | Object-shaped data |
| Main risk | Heavy joins or rigid schema | Duplication and inconsistent fields |

**Solution 21 [reflection]**

The statement means that we should not choose a database because it sounds modern or safe. We should choose based on the problem.
For example, a university enrollment system is a SQL fit because students, courses, and enrollments form structured relationships. Grades belong to the enrollment relationship, and duplication would create correctness problems.
A student profile or user settings page may be a document-store fit because the application often needs one whole object with optional fields, arrays, and nested data. The model follows the shape of the data and the way it is queried.